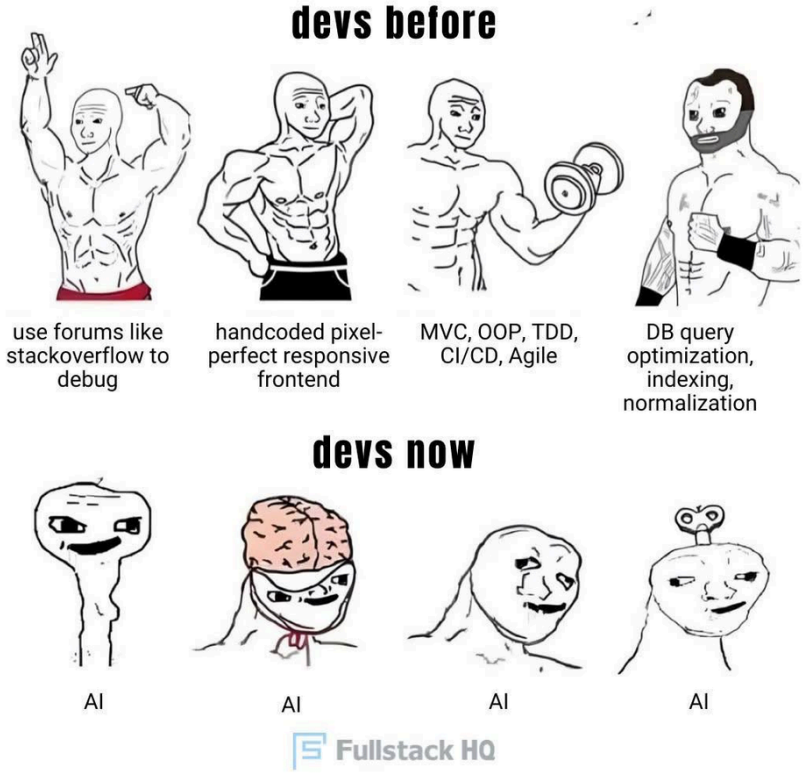
ملخص كتاب The Object Oriented Thought Process

بسم الله و الحمد لله و الصلاة و السلام على رسول الله صل الله عليه و سلم
الحمد لله الذي علم بالقلم علم الإنسان ما لم يعلم و الصلاة و السلام على خير معلم الناس الخير محمد و بعد :

تعريف بالشخصيات الرئيسية
1- أبو السيد : ده راجل software engineer خبرة 20 سنة (شخص محنك)
2- سيد: ده راجل غلبان و على الله حكايته لسه متخرج في زمن ال Ai

المخلص هيكون عبارة عن حوار ما بين اتنين أبو السيد و سيد



الفصل الأول Introduction to Object-Oriented Concepts :

أهلاً بكم
النهاردة هنفتح مع بعض الفصل الأول من الكتاب **The Object Oriented Thought Process(الإصدارالخامس).**
هدفنا هو فهم الفكرة الأساسية ورا **البرمجة الشيئية OOP**،
أو زي ما بيقدمها الفصل ده.
يعني هنحاول نبسّطها خالص للي لسه بيبدأ.

- جاهز يا سيد؟
- جاهز طبعاً.

هو الـ OOP كمفهوم يعني مش جديد خالص،
موجود من زمان، يمكن من الستينات.

- آه فعلاً؟
- أيوة.

بس انتشاره بجد كان في التسعينات،
والنهاردة بقى أساسي في حاجات كتير قوي:
تطبيقات ويب، موبايل، أنظمة معقدة.

زي ما الكتاب بيقول: هي مش بس طريقة كتابة كود وخلاص،
لأ، دي إيه؟ دي **طريقة تفكير**.
طريقة مختلفة في بناء البرامج وحل المشاكل.

- طريقة تفكير؟
- مم، يعني بنشوف المشكلة ونقسمها بشكل مختلف، يمكن أقرب للطبيعة.

خلينا نشوف كده إيه الفرق بينها وبين **البرمجة الإجرائية (Procedural Programming)** اللي كانت قبلها؟

خلينا نبدأ باللي يمكن الناس تعرفه أكتر،
اللي هي البرمجة الإجرائية(Procedural Programming).

زي لغات C أو Visual Basic زمان.
دي كانت بتشتغل إزاي؟

في البرمجة الإجرائية الدنيا كانت متقسمة،
كنا بنفصل بين البيانات (الـ Data يعني) وبين الوظايف أو الإجراءات (الـ Functions) اللي بتتعامل مع البيانات دي.
يعني ده في حتة وده في حتة.

- بالظبط.
- يعني لو عندك بيانات تبعتها على الشبكة مثلاً، بتبعت البيانات كده لوحدها وتتوقع إن البرنامج اللي هيستقبلها هناك عارف المفروض يعمل بيها إيه.

البيانات لوحدها، والأوامر لوحدها.

- تمام.
- طب والبرمجة الشيئية أو الـ OOP بتغيّر ده إزاي بقى؟

التغيير الجذري هنا هو مفهوم الـ **Encapsulation** (التغليف).

- التغليف؟
- التغليف! أيوة.

هنا بقى ما بنفصلش.
بنلم البيانات (اللي هنسميها **Attributes**) والسلوكيات أو الأفعال اللي بتشتغل عليها (اللي هنسميها

Methods) ونلمّهم مع بعض في كيان واحد.

حاجة واحدة بنسميها **كائن Object**.

الكلمة دي بتتكرر كتير.

- طب ممكن نفصصها أكتر؟ إيه هو الكائن ده بالظبط؟

لو خدنا مثال الموظف مثلاً: الكائن هو الوحدة الأساسية خالص في أي برنامج OOP.

أي كائن يكون له حاجتين رئيسيتين:

1. أول حاجة: البيانات بتاعته (Attributes).
   - زي إيه في مثال الموظف؟ زي مثلاً الرقم القومي، تاريخ ميلاده، قسمه، مرتبه، نمرته... أي معلومة بتوصف الكائن ده وتميّزه عن غيره.
   - دي اللي بنسميها **حالة الكائن** (Object State).
2. الحاجة التانية: السلوكيات (Methods).
   - دي بقى الأفعال اللي الكائن ده يقدر يعملها.
   - زمان كنا بنقول عليها Functions، أو Procedural هنا بنقول Methods بالظبط.
   - زي مثلاً: Method اسمها calculateSalary تحسب المرتب، أو عشان updateAdress تغيّر العنوان، أو حتى حاجات أبسط زي Method تجيب لك القسم، أو تغيّر رقم التليفون.

يعني الكائن ده بقى عامل زي كبسولة، جواه معلوماته والأفعال اللي بيعرف يعملها.
كله على بعضه ؟
مظبوط وده بيودينا لنقطة مهمة قوي مرتبطة بالتغليف، اسمها **إخفاء البيانات** (Data Hiding).

- إخفاء البيانات يعني إيه؟
- يعني الكائن بيحافظ على بياناته، مش أي حد كده من برّه يقدر يدخل يغيّر فيها بمزاجه ويبوظ الدنيا. الكائن نفسه هو اللي بيتحكم مين يقدر يوصل لبياناته وإزاي.
- طب بيعمل كده إزاي؟
- غالباً عن طريق الـ Methods برضه. بدل ما تسيب مثلاً خاصية زي المرتب متاحة للتغيير المباشر، بتعمل Method اسمها Setter. دي ممكن جواها تتأكد الأول إن المرتب الجديد ده منطقي قبل ما تغيّره فعلاً.
- آه فهمت.
- ونفس الكلام لو عايز تقرأ معلومة، بتوفر Method تانية زي Getter. اللي بتقرأ واللي بتكتب هي اللي بتمثل الواجهة الآمنة للتعامل مع اللي جوه الكائن.

يعني زي حارس البوابة كده بينظم مين يدخل ومين يخرج للبيانات.

- بالظبط.
  ده بيضمن سلامة البيانات واتساقها.

صحيح إن اللغات الحديثة زي C# مثلاً بتقدّم طرق أسهل زي الـ Properties، بس المبدأ واحد وهو التحكم في الوصول للبيانات.

- تمام.
- يبقى عرفنا إن البرنامج بيتكوّن من كائنات، وكل كائن جواه بياناته وأفعاله ومتحكم فيها.

بس لو أنا عندي برنامج فيه آلاف الموظفين، يعني آلاف الكائنات... دول بييجوا منين؟ مش معقول هكتب كل كائن لوحده!

- سؤال حلو جداً.
- لا طبعاً. هنا بقى بييجي دور مفهوم تاني مهم قوي، وهو **الفئة أو الـ Class**.

الكلاس.

وبكده قطّاعة البسكوت هي الكلاس، وأيوة، وكل بسكوتة بنعملها بالقطّاعة دي هي كائن. أو الكائن الواحد بنسميه **نسخة (Instance)** من الكلاس.
و عملية إنتاج الكائن ده بنسميه **Instantiation**.

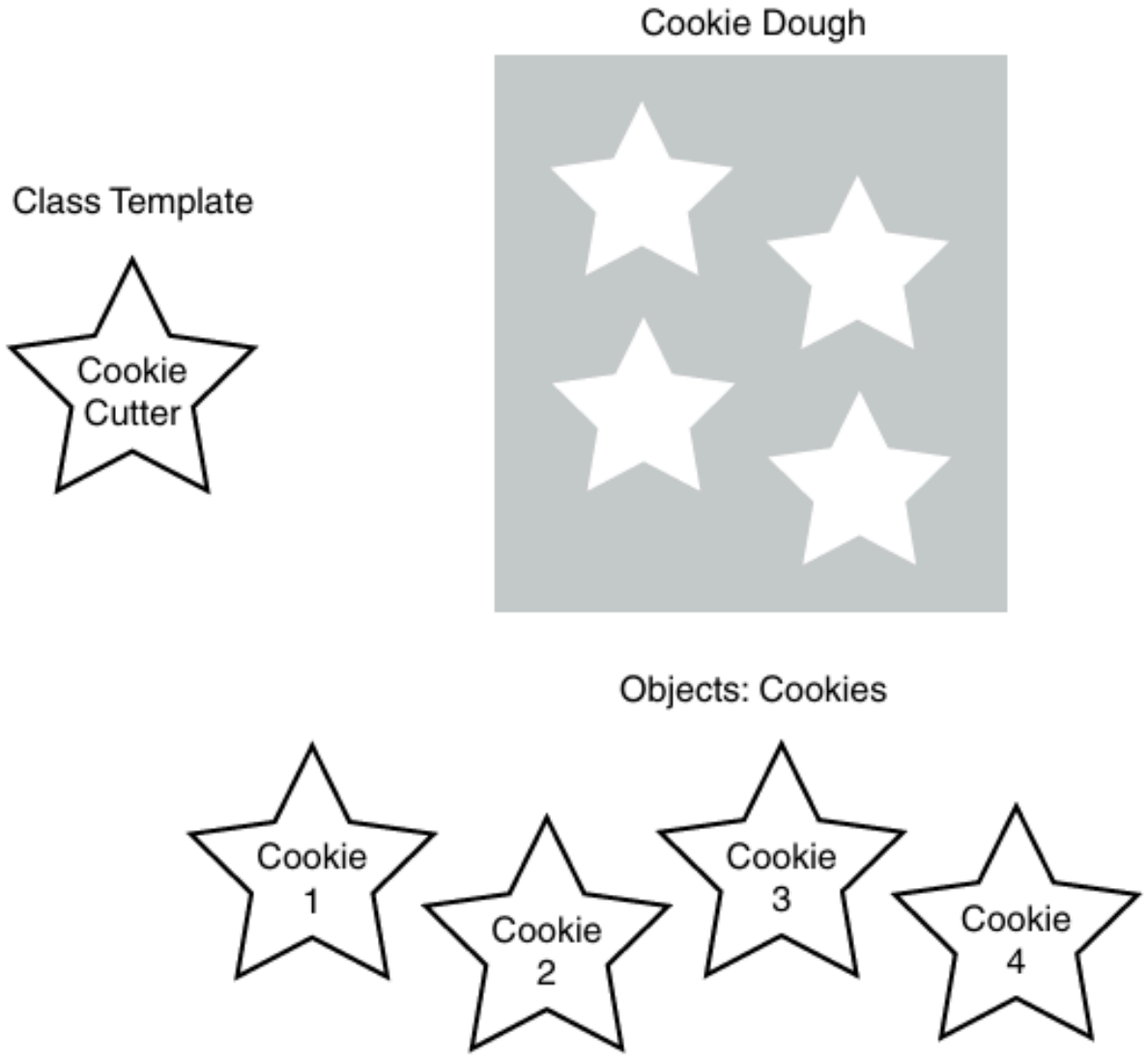


Figure 1.10 Class template.

يعني لو عندي Class اسمها Employee (موظف)، دي كده الوصفة العامة للموظف: بتقول إيه هي بياناته (Attributes)؟ وإيه هي أفعاله (Methods)؟

وبعدين أقدر أستخدم الكلاس دي عشان أخلق كائنات موظفين بجد زي أحمد وسارة.

- تماماً.
- أحمد هيبقى Instance من Employee Class، وسارة Instance تانية.
  كل واحد فيهم كائن مستقل ببياناته الخاصة.
  يعني أحمد له اسمه ورقمه، وسارة لها اسمها ورقمها المختلفين.
  لكن الاتنين ماشيين على نفس القالب بتاع الكلاس.

والكتاب بيقول إننا بنستخدم رسومات معينة اسمها **UML Class Diagrams** عشان نوضّح شكل الكلاس دي.

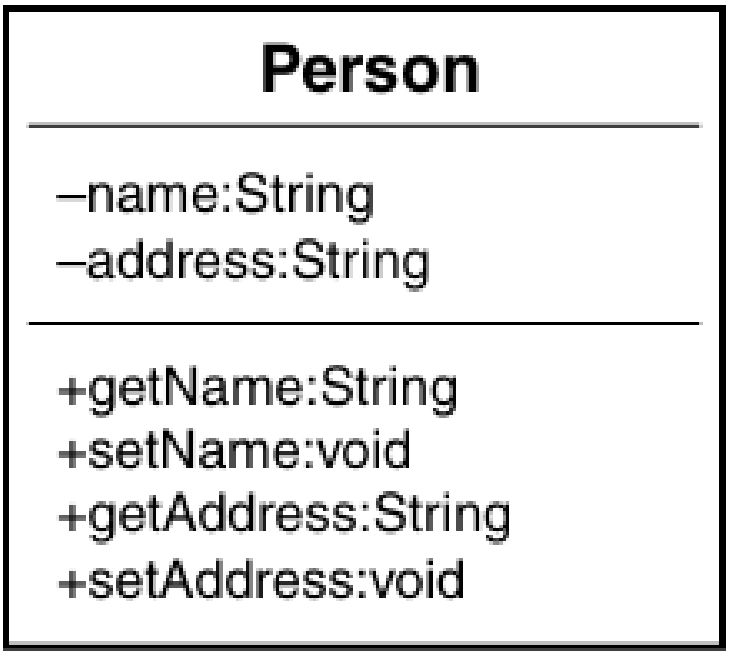


Figure 1.11 The Person class diagram.

- أيوة.
  الكلاس ده يا سيدي أداة مهمة قوي في التصميم.
  بيبقى عبارة عن مربع كده:
  فيه اسم الكلاس، وتحته الـ Attributes، وتحتهم الـ Methods.

وعلامات الزائد (+) والناقص (–) دي:

- الزائد يعني عام (Public) أي حد يقدر يستخدمه.
- الناقص يعني خاص (Private) ده بيكون متخبي جوه الكائن وما حدش يقدر يوصله من برّه.

الرسومات دي بتساعد قوي في فهم تصميم النظام ككل.

- تمام.
- نرجع تاني للتغليف **Encapsulation** وإخفاء البيانات **Data Hiding**. قلنا: الكائن بيجمع حاجته وبيتحكم فيها.

ليه الحتة دي محورية أوي كده؟ ليه الإخفاء ده مهم؟

الإخفاء ده مهم عشان حاجتين: **الأمان والمرونة**.

- الأمان: قلناه، بنحمي البيانات.
- لكن المرونة دي **هي القوة الحقيقية**.
- إزاي يعني؟
- لما الكائن بيخبي تفاصيله الداخلية، يعني إزاي بيشتغل من جوه بالظبط (Implementation Details)، ويقدم للعالم الخارجي مجرد واجهة (Interface) واضحة ومحددة، ده بيعمل ميزة كبيرة قوي: إن اللي بيستخدمه مش لازم يعرف التفاصيل.
- بالظبط.
- زي ما بتستخدم التكييف: إنت بتتعامل مع الريموت، الواجهة بتاعته مش محتاج تعرف الماتور شغال إزاي جوّه.
- تمام.

الأهم بقى إن الشركة المصنّعة ممكن تغيّر التصميم الداخلي للتكييف ده خالص،
تستخدم تكنولوجيا أحدث.
طالما الريموت (الواجهة Interface) هو هو، إنت كمستخدم مش هتحس بحاجة، ومش محتاج تغيّر أي حاجة.

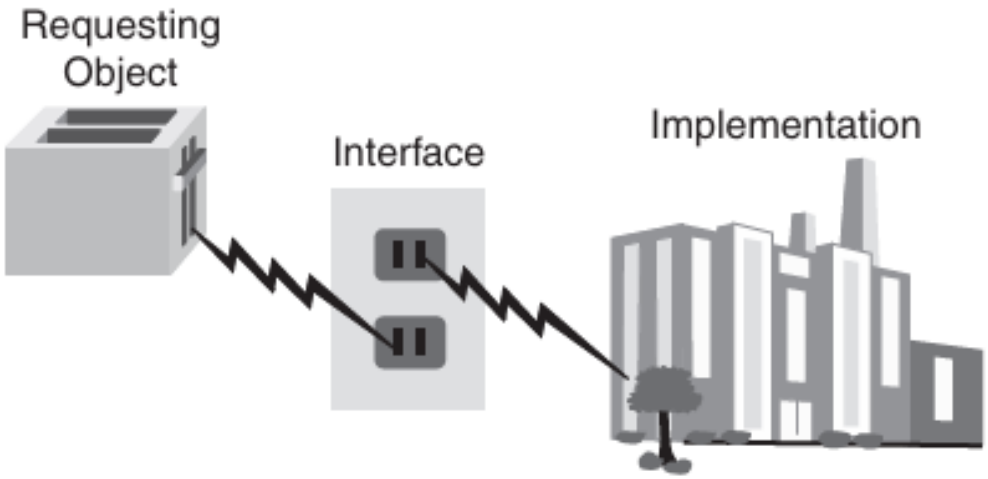


Figure 1.12 Power plant example.

- آه، يعني أقدر أغيّر أنا الكود اللي جوّه الكائن بتاعي، أصلّحه أو أُحسّنه، وما حدش من اللي بيستخدموه يتأثر، طالما أنا محافظ على الواجهة اللي هم متعودين عليها.
  دي حاجة قوية جداً في الصيانة.

هي دي الفكرة بالظبط: الفصل بين الواجهة (Interface) والتنفيذ (Implementation).
ده سر البرامج المرنة اللي نقدر نطوّرها بسهولة.

الكتاب بيضرب مثال بكود بيحسب الجذر التربيعي.

- آه، مثال الآلة الحاسبة.
- أيوة.

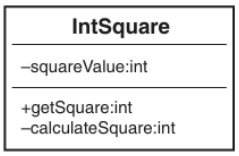


Figure 1.13 The `Square` class.

This class diagram corresponds to the following code:

```
public class IntSquare {

    // private attribute
    private int squareValue;

    // public interface
    public in getSquare (int value) {

    squareValue = calculateSquare(value);

        return squareValue;

    }


    // private implementation
    private in calculateSquare (int value) {

        return value*value;

    }
}
```

المستخدم عايز يعرف إزاي يطلب الجذر (دي الواجهة)، لكن مش مهم عنده الحسبة بتتم إزاي جوّه (ده التنفيذ).
لو المبرمج لقى طريقة حساب أسرع، يقدر يغيّر التنفيذ الداخلي، وما حدش يحس غير إن النتيجة بقت أسرع.
مثال في الكتاب بيوضح ده برضه: التنفيذ المخفي.

```
calculateSquare.

// private implementation
private intcalculateSquare (int value) {

    return = Math.pow(value,2);

}
```

- فهمت.

يبقى التغليف **Encapsulation** بيعمل وحدات مستقلة ومحمية بواجهات واضحة.

طب لو عندنا بقى كائنات متشابها في حاجات كتير، يعني زي ما قلت في الأول: لو بنعمل سيستم لعيادة وعندنا قطط وكلاب.
أكيد فيهم حاجات مشتركة. هنفضل نكرر نفس الخصائص دي في Class القطة وClass الكلب؟

- لأ طبعاً.
- هنا بييجي دور مفهوم أساسي تاني وهو **الوراثة – Inheritance**.

الوراثة دي بتسمح لنا نعمل Class جديدة بنسميها الابن (Child Class) أو Subclass، تاخد أو ترث من Class تانية موجودة أصلاً بنسميها الأب (Parent Class) أو Superclass.

- ليه بنعمل كده؟
- الهدف الأساسي هو تجميع الخصائص والسلوكيات المشتركة في مكان واحد (الـ Superclass) عشان نتجنّب التكرار وننظم الكود.

زي مثال الحيوانات.

- بالظبط. بدل ما نحط خاصية زي "لون العين" أو "الأكل" في Cat ونرجع نحطهم تاني في Dog، لأ، ممكن نعمل Class أكبر اسمها Mammal (ثدييات)، ونحط فيها الحاجات المشتركة دي.
- وبعدين؟
- وبعدين نخلي Cat وDogg يرثوا من Mammal.
  بكده أي قطة أو كلب هنخلقه هيبقى عنده تلقائي لون عين، وبيقدر ياكل، بالإضافة طبعاً لأي خصائص أو سلوكيات خاصة به هو، بنحطها في Class بتاعته.

Chapter 1 Introduction to Object-Oriented Concepts

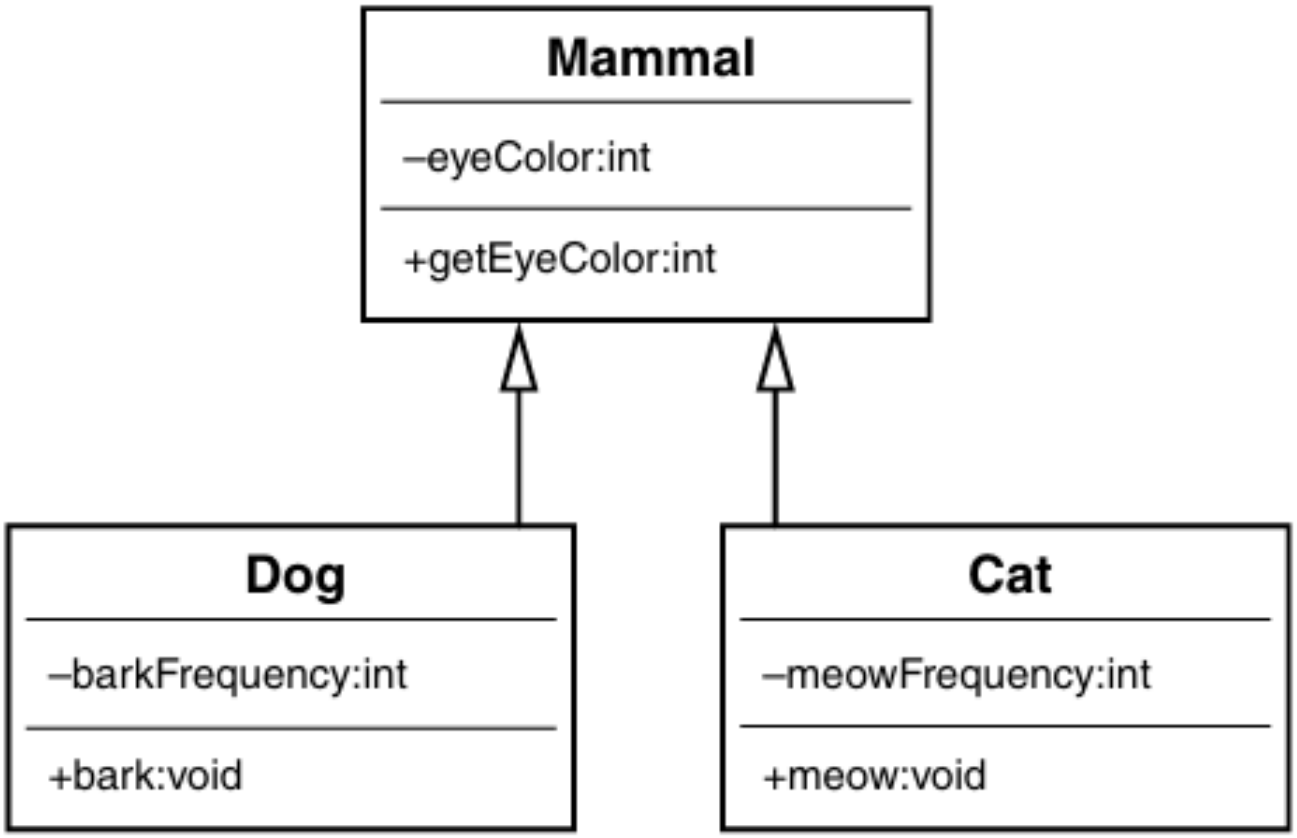


Figure 1.14 Mammal hierarchy.

يعني كأننا بنقول:
القطة نوع من الثدييات.
والكلب برضه نوع من الثدييات.

دي علاقة **"is a"**.
هي المفتاح هنا.

- تماماً.
  اختبار الـ "is a" ده مهم قوي.
  لو العلاقة دي منطقية، يبقى الوراثة ممكن تكون اختيار كويس.
  بتساعد نبني هرم Classes بيعكس العلاقات دي، وبيخلي الكود منظم وقابل لإعادة الاستخدام.
- تمام.
- الوراثة مفيدة عشان ما نكررش الكود.

طيب، لو عندنا بقى كائنات ورثت من بعض...
زي مثلاً أشكال هندسية مختلفة:
دائرة، مستطيل، مثلث.

كلهم وارثين من Class اسمها Shape.
وكلهم محتاجين Method اسمها draw() عشان يرسموا نفسهم، أو calculateArea() عشان يحسبوا مساحتهم.

بس أكيد طريقة رسم الدائرة غير المستطيل.
وحساب مساحة دي غير دي.

- هنتصرّف إزاي هنا؟
- سؤال ممتاز. ده بقى ياخدنا للمفهوم الرابع المهم: **تعدد الأشكال – Polymorphism**.

Polymorphism كلمة كبيرة دي.
معناها الحرفي "أشكال متعددة".

الفكرة ببساطة: إنك تقدر تبعت نفس الرسالة (Message)، يعني تنادي على نفس Method بنفس الاسم، لكائنات من أنواع مختلفة، بس ليهم علاقة ببعض بالوراثة غالباً. وكل كائن منهم يرد أو يتصرّف بطريقته الخاصة المناسبة لنوعه.

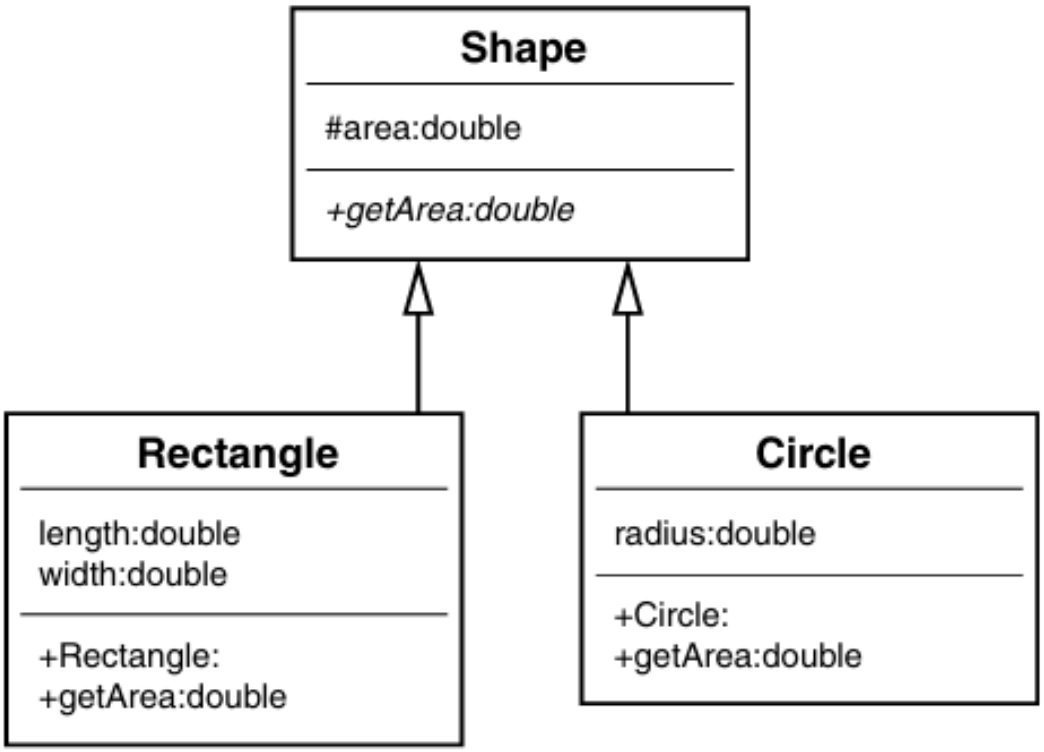


Figure 1.18 Shape UML diagram.

زي مثال الأشكال الهندسية اللي قلناه بالظبط.
لو عندك List فيها أشكال مختلفة (كائنات من Subclasses مختلفة)، وعايز تحسب مساحتهم كلهم.

ممكن تلف عليهم واحد واحد، وجوّه اللفة تنادي على نفس الـ Method اللي هي calculateArea().
نفس الاسم لكل الأشكال.

- أيوة.
- لما تنادي calculateArea() على الدائرة، هتشتغل طريقة حساب مساحة الدائرة.
  ولما تناديها على المستطيل، هتشتغل طريقة حساب مساحة المستطيل.

نفس الاسم، نفس الطلب، لكن التنفيذ بيختلف حسب مين اللي بيتنفّذ عليه الطلب ده.

ده كده بيخلي الكود اللي بيستخدم الأشكال دي مرن قوي.
مش محتاج أعرف نوع الشكل عشان أطلب منه يحسب مساحته.

هي دي قوة الـ Polymorphism: بتفصل الكود اللي بيطلب عن الكود اللي بينفّذ.

وممكن بسهولة تضيف أشكال جديدة بعد كده من غير ما تعدّل في الكود القديم اللي بيستخدم الأشكال دي.

الكتاب كمان بيلمح إن Class الأب زي Shape دي ممكن تكون **Abstract Class**.

- يعني إيه مجردة؟
- أيوة، يعني مش كاملة. ممكن يكون فيها Methods متعرّفة (Declared) بس مش مكتوب الكود بتاعها (Implementation).

يعني زي مثلًا ()calculateArea ، الهدف منها إنها تكون زي عقد بتجبر كل اللي هيرث منها إنهم لازم يوفروا التنفيذ الخاص بهم ده. وما ينفعش نعمل كائن مباشر **Instance** من **Abstract Class**.
حلو قوي.

كده شفنا **Encapsulation Inheritance** و **Abstraction** و **Polymorphism**.

فاضل مفهوم أساسي أخير بيذكره المؤلف: لو الكائنات بقى مش بترث من بعض، بس هي عبارة عن **أجزاء من بعض**، زي العربية جواها موتور وعجل. ده بالظبط المفهوم الخامس: التجميع **Composition**. الفكرة منطقية جدًا: الحاجات الكبيرة بتكون من حاجات أصغر.

الكمبيوتر فيه شاشة وكيبورد.
العربية فيها محرك وشاسيه وعجلات.

يعني هنا العلاقة مش "هو نوع من" (is-a) لأ، العلاقة "يحتوي على" **(has-a)**.

بالظبط كده، بنقول: العربية تحتوي على محرك (Car has an Engine)، مش المحرك هو نوع من العربيات.

ال **Composition** ده طريقة تانية قوية جدًا لبناء كائنات معقدة، عن طريق إننا نجمع جواها كائنات تانية أبسط وأكثر تخصصًا. الكائن الكبير بينسّق الشغل بين الأجزاء اللي جواه.

وده برضه مفيد لإعادة الاستخدام؟ أكيد.
لو صممت كلاس كويس للمحرك (Engine)، ممكن أستخدمها وأعمل منها كائنات محركات في أكتر من موديل عربية أو حتى في حاجة تانية خالص زي مركب أو مولد كهرباء.

ال **Composition** بيدي مرونة عالية جدًا، وناس كتير بتفضّله على ال **Inheritance** في مواقف كتير، لأنه بيخلي الكلاسات معتمدة على بعض بشكل أقل. ممتاز.

كده غطّينا المفاهيم الأساسية اللي في الفصل الأول:

- الكائنات **object** = بيانات وسلوك **methods** , **Attributes**.
- الفئات **classes**= قوالب.
- التغليف **Encapsulation** = تجميع وإخفاء.
- الوراثة **Inheritance**.
- تعدد الأشكال **Polymorphism** = نفس الرسالة، استجابات مختلفة.
- والتجميع (has-a) **Composition**.

نرجع بقى للسؤال الكبير: ليه اللفة دي؟ إيه اللي يخلي طريقة التفكير "الشيئية OO" دي مهمة ومفضّلة عن الطريقة الإجرائية Procedural القديمة في مشاريع كتير؟

زي ما قلنا في الأول: الموضوع مش حرب بين القديم والجديد.
لو في نظام قديم إجرائي شغال كويس وبيأدي الغرض، ممكن يفضل زي ما هو، ما حدش هييجي جنبه.

لكن لما نبدأ مشاريع جديدة، خصوصًا لو كبيرة أو معقدة أو بتحاكي تفاعلات في عالم حقيقي، أو بنتكلم بقى عن تطبيقات الويب والموبايل الحديثة، هنا الـ OOP بيبين قوته.

وإيه هي القوة دي؟ المزايا يعني.

المزايا هي خلاصة اللي شرحنا:

1. **التنظيم وإدارة التعقيد**:
   - التغليف بيقسم السيستم الكبير لوحدات صغيرة (كائنات) نقدر نتعامل معاها.
   - بتبقى أسهل في الفهم والصيانة.
   بالظبط.
2. **إعادة الاستخدام**:
   - الوراثة والتجميع بيخلونا نستخدم كود كتبناه قبل كده بدل ما نعيد اختراع العجلة.
   - ده بيوفّر وقت ويقلل أخطاء.
   صح.

3. **الصيانة والمرونة**.
حتة فصل الواجهة عن التنفيذ دي بتخلي التعديل أسهل وآمن.
تقدر تغيّر جوة الكائن براحتك طالما الواجهة ثابتة، وتعدد الأشكال بيضيف مرونة لإضافة وظائف جديدة.

4. **نقطة محاكاة الواقع**.
أيوة، طريقة التفكير بالكائنات وخصائصها وتفاعلاتها ساعات بتكون أقرب لطريقة تفكيرنا الطبيعية، وده ممكن يسهّل التصميم في بعض الحالات.

بس هنا لازم نحط في الاعتبار حاجة مهمة جدا
إيه هو؟ التعليق بتاع روبرت مارتن اللي الكتاب ذكره: إن فكرة محاكاة الواقع دي ممكن تكون تسويقية أكتر، وإن الهدف الأهم للـ OOP هو **إدارة الاعتماديات Dependencies** بين الأجزاء.

يعني مش لازم الكود يبقى نسخة طبق الأصل من الواقع، مش بالضرورة.
الأمثلة البسيطة زي القطة والعربية بتساعد في الشرح، لكن في الأنظمة الكبيرة والمعقدة، محاولة المحاكاة الحرفية ممكن تعقّد الدنيا زيادة.

الأهم – زي ما وجهة النظر دي بتقول – إن الـ OOP بيساعدنا نفك التشابك بين أجزاء الكود، ونخلي كل جزء معتمد على الأجزاء التانية بأقل قدر ممكن. وده اللي بيخلي النظام سهل صيانته وتطويره على المدى الطويل.

يعني الأهم هو **تنظيم الكود نفسه وتقليل الاعتماديات**.
بالظبط، سواء كان التصميم ده بيحاكي الواقع بشكل مباشر أو لأ.

وعجبتني برضه فكرة إننا مش لازم نرمي القديم.
الكتاب قال ممكن نستخدم **Wrapper** عشان نغلف كود قديم.

أيوة، الـ Wrapper ده زي كائن جديد بيعمل واجهة لكود قديم إجرائي، بيخبيه جواه ويقدمه للعالم الخارجي بشكل شيئي.
ده بيسمح بدمج الأنظمة القديمة في أنظمة أحدث، أو الانتقال التدريجي من غير ما نضطر نعيد كتابة كل حاجة من الصفر.

الفكرة دايمًا هي **إدارة التعقيد والمرونة**.

يبقى خلاصة رحلتنا في الفصل الأول:

- البرمجة الشيئية دي طريقة تفكير بالكائنات.

- كل كائن جواه بياناته وأفعاله، ومتغلف عشان يحمي نفسه ويقدم واجهة.
- الفئات = المصانع.
- الوراثة = لإعادة الاستخدام.
- تعدد الأشكال = نفس الطلب بس استجابات مختلفة.
- والتجميع = لبناء المركّب من البسيط.

وهذا هو الأساس. وزي ما عنوان الكتاب بيقول: هي عملية تفكير **Thought Process.**

الأهم من حفظ الكلام ده هو **فهم الفلسفة وراه**، وإزاي المبادئ دي بتخلينا نفكر بشكل مختلف عشان نعمل برامج أحسن وأسهل في صيانتها.

ونقطة أخيرة نسيبها للتفكير:
وهي حتة "محاكاة الواقع" دي ومدى أهميتها.
الفصل بيقدّم قواعد زي is-a و has-a المستوحاة من الواقع كدليل.
بس السؤال اللي يفضل قائم:

لحد فين المفروض نلتزم بالمحاكاة دي؟
وهل لو تمسكنا بيها قوي في الأنظمة الكبيرة ممكن تعقّد الأمور بدل ما تبسّطها؟

سؤال فعلًا يستحق نفكر فيه لحد ما نكمل باقي الرحلة مع الكتاب سلام.

الفصل الثاني How to Think in Terms of Objects :

النهاردة هنكمل كلامنا عن كتاب **عملية التفكير الموجَّه للكائنات**، وهنركز على **الفصل التاني**.

الفصل الأول، لو فاكرين، كان بيحط الأساسيات والمفاهيم.
أيوة، كان زي تعريف أدوات.
بالظبط.

الفصل ده بقى عنوانه: **إزاي تفكر بمنطق الكائنات؟**
يعني بننتقل من سؤال: إيه هي البرمجة الكائنية؟
لسؤال: إزاي أستخدمها صح كطريقة تفكير؟
تمام.

النقلة دي مهمة جدًا.
مش مجرد معرفة.
لأ، دي تغيير في العقلية نفسها.
خصوصًا للي جاي من خلفية Procedural Programming يعني.

الكتاب بيركّز أوي على النقطة دي.
فعلاً، والهدف الأساسي النهاردة هو إننا نفهم الفرق الجوهري بين الـ **Interface (الواجهة)** واللي هو **Implementation** (**التنفيذ الفعلي**).

أظن ناس كتير بتقابل مشكلة إن الكود بتاعها بيتأثّر بسرعة جدًا بأي تغيير بسيط.
يمكن هنا الحل؟
هو ده فعلًا حجر الزاوية: **فصل Interface عن Implementation**.

طيب، خلينا نبدأ من هنا.
الفصل ده بيبني إزاي طريقة التفكير دي؟ الكتاب بيتكلم كتير عن فصل ماذا يفعل الكائن عن كيف يفعله.
Interface مقابل **Implementation**.

هل دي مجرد مسألة تنظيم كود وخلاص؟ ولا الموضوع أعمق؟
لأ، طبعًا أعمق بكتير.
ده مش مجرد تنظيم.
ده أساس بناء أنظمة قوية ومرنة.
مرنة وسهلة الصيانة كمان.

دي استراتيجية أساسية للتحكم في التعقيد، خصوصًا في البرامج الكبيرة.
مم، استراتيجية للتحكم في التعقيد.
أيوة.

الكتاب استخدم تشبيه حلو قوي بتعفِيشة الكهربا ومحطة التوليد.

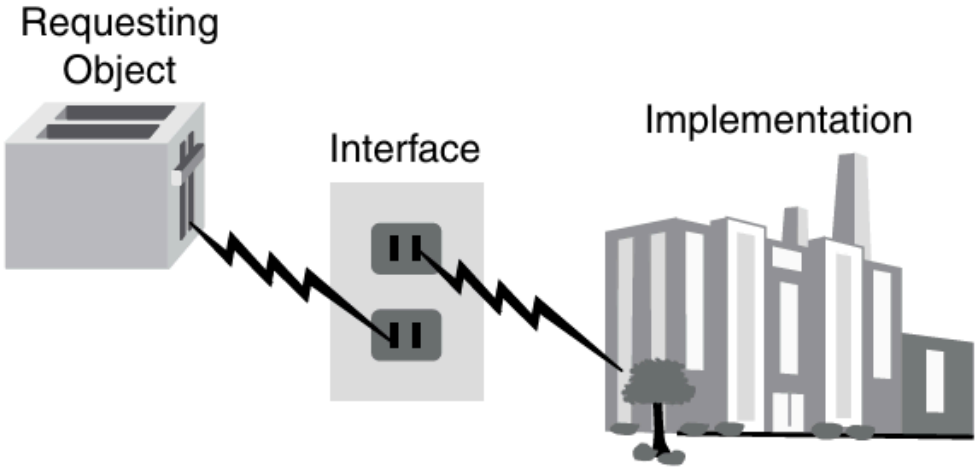


Figure 2.1 Power plant revisited.

إنت لما بتستخدم الفيشة دي (الواجهة)، كل اللي يهمك إنها بتوصل لما تحط فيها الجهاز.
مش فارق معاك خالص محطة التوليد اللي وراها دي — اللي هو التنفيذ Implementation — شغالة إزاي: بالفحم؟ بالغاز؟ بالطاقة الشمسية؟ كل دي تفاصيل داخلية.

يعني أنا كمستخدم للفيشة أقدر أوصل أي جهاز طالما الفيشة بتاعته مناسبة، ومش هشيل هم المحطة نفسها اتغيّرت إزاي؟
تمامًا.

وممكن ناخد مثال تاني أقرب لينا: **زي العربية**.
تمام؟ الواجهة بتاعتك كسائق هي إيه؟
الدركسيون، الدواسات، الفتيس، الزراير اللي قدامك.
دي الأدوات اللي بتقولي بيها للعربية إيه المطلوب: تمشي، تقف، تلف.
فاهم.

أما بقى إزاي العربية بتنفّذ ده؟ فده شغل الماتور والفرامل وكل الميكانيزمات المعقدة اللي تحت الكبوت.
ده هو التنفيذ.
يعني مش لازم أكون ميكانيكي عشان أسوق العربية.

دي بس لفت نظري حاجة في كلامك: لو غيروا الموتور ده بموتور أضعف بكتير، هنا تجربتي في السواقة هتتأثر حتى لو شكل الدركسيون هو هو، صح؟

نقطة ممتازة .
الواجهة Interface مش بس شكل وأسماء الوظائف، لأ.
الواجهة Interface دي كمان بتشمل — زي ما تقول — **عقد سلوكي**.

يعني Interface بتوعد بأداء معين.
لو اتغير التنفيذ، زي الموتور الأضعف ده، **خالف العقد ده**.

يعني العربية بقت بطيئة جدًا مثلًا لدرجة غير مقبولة، يبقى كأن Interface نفسه اتغيرت بشكل مش مباشر، لأنه ما بقاش بيوفي بوعده الأصلي.

آه، فهمت قصدك: العقد السلوكي.
بالظبط

الهدف بقى من فصل Interface عن Implementation إننا نقدر نغيّر Implementation الداخلي ده براحتنا: نحسّن فيه، نصلّح أخطاء، نبدّل مكون بمكون تاني خالص، كل ده من غير ما نكسر الدنيا عند اللي بيستخدموا Interface دي، طالما Interface نفسها والعقد بتاعها فضلوا زي ما هم.

ده اللي بيسموه: **controlled ignorance**.
الله ينور عليك .

هو ده، بنخلي الأجزاء التانية من النظام تستخدم الكلاس بتاعنا من غير ما تحتاج تعرف تفاصيله المعقدة.

دي مش رفاهية خالص.
دي ضرورة لبناء أي نظام معقد وقابل للتطوير.

مفهوم controlled ignorance ده قوي فعلًا.

طيب، عشان نوضّح أكتر: مين هو **المستخدم** اللي بنتكلم عنه هنا؟
هل هو الشخص العادي اللي بيستخدم البرنامج النهائي؟ ولا حد تاني؟
وهل الواجهة هنا هي نفسها واجهة المستخدم الرسومية GUI؟

لأ، مهم قوي نفرّق.
المستخدم في سياق الفصل ده غالبًا بيكون **كود تاني**.
يعني ممكن يكون مبرمج زميلك اللي هيستخدم الكلاس اللي إنت كتبته.
آه، مطوّر تاني.
أيوة، أو جزء تاني من نفس البرنامج اللي إنت شغّال عليه.

أما **الواجهة**، فدي مش الواجهة الرسومية خالص.
الواجهة هنا هي مجموعة **الوظائف العامة Public methods** اللي الكلاس بيقدمها للعالم الخارجي.

زي قايمة أوامر كده: الكلاس ده بيعرف ينفّذ إيه، بنحدد فيها أسماء الوظائف functions، إيه البيانات اللي محتاجاها parameters، وإيه النتايج اللي بترجعها return values.

تمام.
وفكرة **إخفاء البيانات Data hideing** جزء أساسي من الموضوع ده.
يعني بنخلي المتغيرات الداخلية تكون **Private**، بنخبي التفاصيل دي عشان اللي برّه الكلاس ما يشوفش غير اللي يخصه بس.

بالظبط كده.
تمام أوي.

يبقى:

- Interface هي: الوظائف والمتغيرات الضرورية.
- Implementation هو: كل الكود الداخلي والـ **Private Attributes** والـ **Methods**.

الكتاب دخل في مثال عملي عشان يوضّح ده: **قارئ قاعدة البيانات DatabaseReader**.

Figure 2.2 shows a class diagram representing a possible interface to the `DataBaseReader` class.

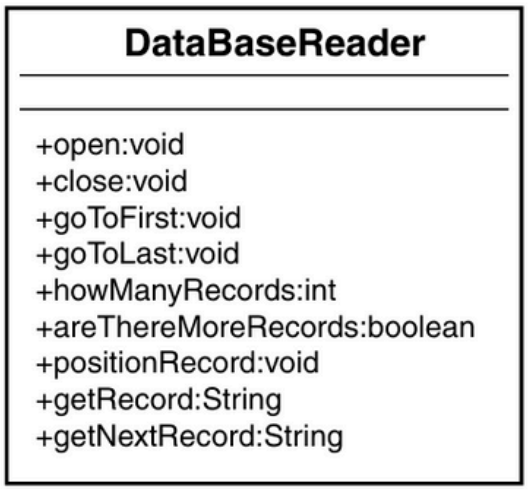


Figure 2.2 A Unified Modeling Language class diagram for the `DataBaseReader` class.

ممكن تشرح لنا المثال ده بيوضح إزاي عملية الفصل دي؟
أكيد.

افترض إننا محتاجين نعمل كلاس يقرا بيانات من Database.
الكلاس ده **DatabaseReader** مطلوب منه شوية وظائف أساسية: يفتح الاتصال (**Open**)، يقفله، يتنقّل بين السجلات (**GetRecord** مثلاً)، يمكن يعرف عدد السجلات، وهكذا.

دي الوظائف اللي الكود التاني هيحتاجها عشان يتعامل معاه.
تمامًا.

فأول خطوة في التصميم هي تحديد الوظائف دي اللي هتكون متاحة للاستخدام من برّه.
ودي اللي بتشكّل **الواجهة بتاعته Interface**.
زي الأزرار اللي على الريموت كنترول كده.

ولو حبّينا نرسم شكل يوضّح الكلاس ده، الكتاب استخدم **Class Diagram** في شكل *2.2*.
الكلاس دايجرام زي خريطة للكلاس.

بالظبط.
هو أداة بتورينا:

- اسم الكلاس.
- متغيراته (**Attributes**).
- وظائفه (**Methods**).

وبيوضّح مين فيهم **Public** (متاح للعالم) جنبه علامة **+**، ومين **Private** (مخفي) جنبه علامة **-**.

بيساعدنا نشوف Interface والأجزاء بشكل واضح، منفصل عن Implementation.
حلو أوي .

يبقى الكود اللي هيستخدم **DatabaseReader**، كل اللي يهمه يعرفه هو أسماء الـ **Public Methods** دي وطريقة استخدامها.
يعني مثلًا: يستدعي **Open**، وبعدين **GetRecord**.

ومش فارق معاه خالص بقى: الكلاس ده من جوّه بيتعامل مع إيه؟
Oracle ولا **SQL Server**، ولا حتى ملف **Excel**.

صح كده؟
مية مية.

هنا بقى بتظهر القوة الحقيقية للمبدأ ده.

الكتاب شرح في شكل *2.3* إزاي ممكن يكون عندنا تنفيذ للكلاس ده بيشتغل مع **Oracle**، وبعد فترة قررنا نغير ونخليه يشتغل مع تانية خالص زي **SQL Anywhere** أو **MySQL**.

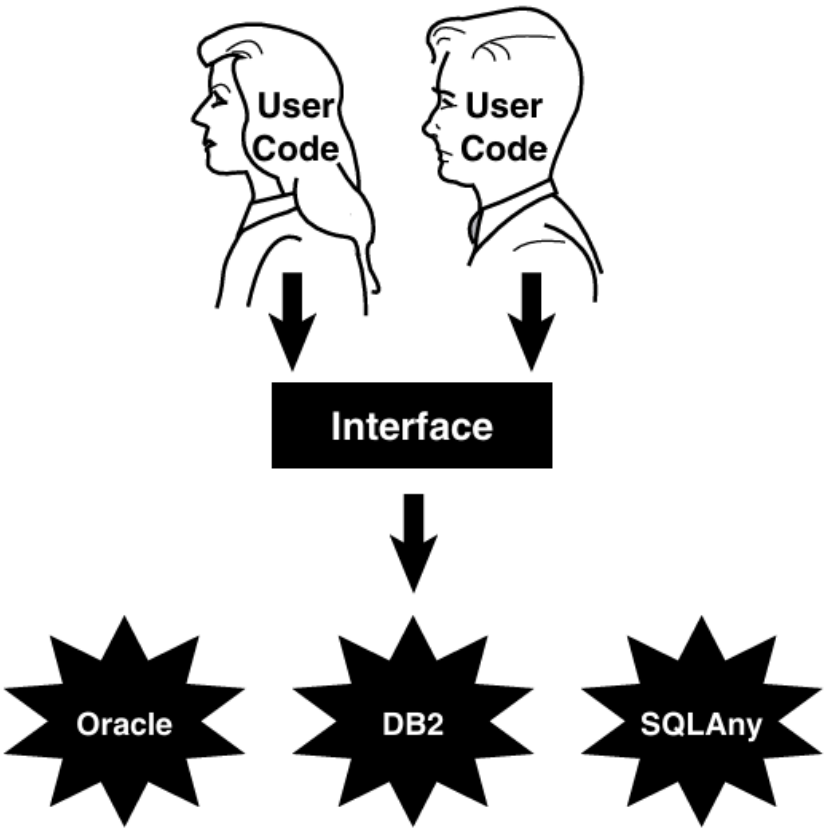


Figure 2.3 The interface.

آه.
طالما حافظنا على نفس Interface، بنفس الأسماء ونفس طريقة الاستخدام، يعني حافظنا على ال Interface، الكود اللي كان بيستخدم النسخة القديمة مش هيحتاج يتغير فيه ولا سطر واحد عشان يشتغل مع النسخة الجديدة.

دي ميزة جبّارة فعلًا، بتخلي التحديث والصيانة أسهل بكتير جدًا.
يعني لو عندي سيستم كبير وفيه مية حتة بتستخدم الكلاس ده؟ لو ما فيش الفصل ده، التغيير هيبقى كابوس حقيقي.

فعلاً كابوس.
إنك تفصل ال Interface عن ال Implementation ده بينقذنا من الكوابيس دي.

التعديل بيبقى محصور جوه كلاس **DatabaseReader** بس.
ممكن الفريق اللي شغّال على باقي السيستم ما يعرفش أصلًا إن الـ Database اتغيّرت.

الكتاب كمان لمح لنقطة تقنية مرتبطة بده، وهي موضوع **إعادة بناء الكود**.
لو الكود اللي بيستخدم **DatabaseReader** بيتحمّل ديناميكيًا وقت التشغيل (زي في **Java** أو **C#**)، التغيير في تنفيذ **Reader** ممكن ما يحتاجش حتى إعادة بناء للأجزاء التانية.
التغيير معزول تمامًا.

بالظبط.
ولو الربط ثابت (Static Linking) زي بعض اللغات زي C++، ممكن نحتاج نعمل Linking تاني، لكن برضه الكود المصدري للأجزاء التانية مش بيتغير.

الفكرة هي: **احتواء التغيير**.
"احتواء التغيير" مصطلح مهم.

طيب، ده يودّينا لنقطة مهمة تانية في الفصل:
بما إن الواجهة Interface دي بالغة الأهمية، إزاي نصمّمها صح؟

الكتاب بيركّز على أهمية **التفكير التجريدي Abstract thinking** وإحنا بنصمّم الواجهات.
هل ده معناه نخلي الواجهة Interface عامة شوية؟

فاكر مثال التاكسي؟

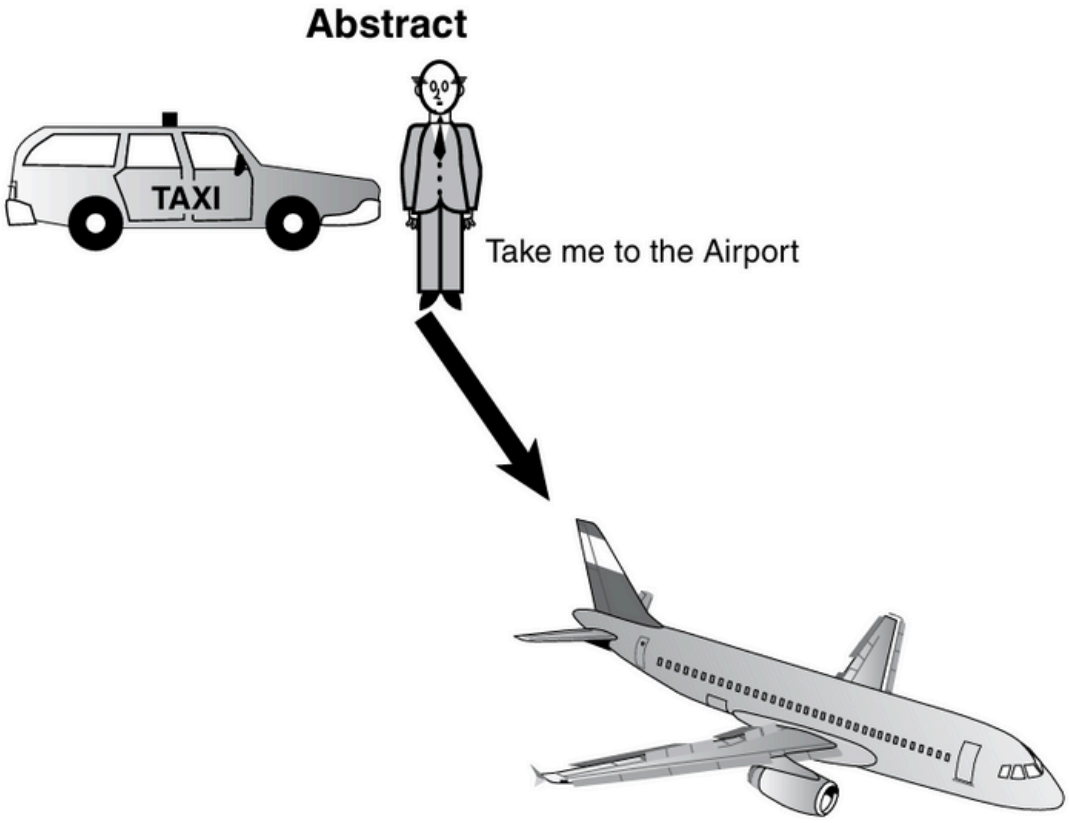


Figure 2.4 An abstract interface.

في شكل *2.4* و *2.5*: المقارنة بين إني أقول للسواق: "وصلني المطار"، وده طلب تجريدي، وبين إني أقوله تفاصيل الطريق: "خُش يمين، بعدين شمال". وده طلب ملموس قوي.

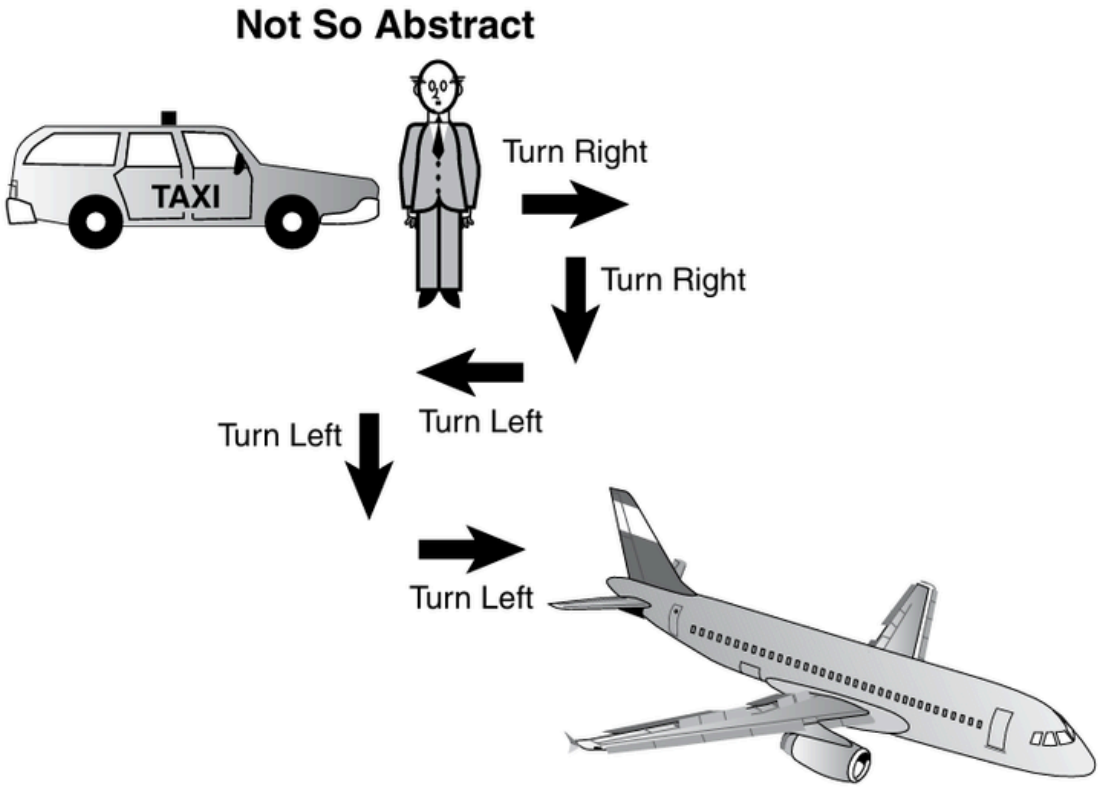


Figure 2.5 A not-so-abstract interface.

مظبوط جدًا .
التفكير التجريدي هنا يعني نفكّر في **الغرض أو الهدف** من الوظيفة، مش في الخطوات التفصيلية للتنفيذ.

جملة "وصلني المطار" بتعبّر عن **الغرض**، وبتسيب للسواق (الكائن object) حرية يحدد أحسن طريق.
ده التنفيذ بتاعه Implementation.

الطلب ده **تجريدي**، مش معتمد على مكانك فين دلوقتي، وممكن تستخدمه في أي مدينة.
آه، قابل لإعادة الاستخدام.
بالظبط .

أما بقى التعليمات التفصيلية بتاعة "يمين في شمال" دي واجهة **ملموسة (Concrete Interface)** مرتبطة بسياق معيّن جدًا، ومش هتنفع غير مرة واحدة.
يعني ال Abstraction هي اللي بتدينا **قابلية إعادة استخدام** أكبر بكتير.

أكيد، Abstraction بتفصل المستخدم عن تفاصيل التنفيذ الدقيقة Implementation.
وده بيخليها مرنة أكتر، وقابلة للتكيّف مع أي تغييرات في المستقبل.

الهدف هو نصمّم واجهات زي: "وصلني المطار" — تكون قوية ومفيدة في سيناريوهات كتير، بدل واجهات ضعيفة ومحدودة زي تعليمات الطريق يمين في أول شمال ....

ده بيخلي الكود بتاعنا مش بس سهل الصيانة، لأ، كمان **قابل لإعادة الاستخدام** في مشاريع تانية.

وده يمكن يوصلنا لفكرة إن ال Interface تكون **أصغر ما يمكن (Minimal Interface)**.
يعني ندي للمستخدم بتاع الكلاس أقل عدد ممكن من الوظائف اللي هو فعلًا محتاجها عشان يحقق هدفه، من غير ما نغرّقه في تفاصيل زيادة.

تمامًا ، البساطة قوة.

الواجهة Interface المثالية لازم تكون حاجتين:

1. **كاملة (Complete)**: فيها كل اللي المستخدم محتاجه عشان يتعامل مع الكائن.
2. **صغيرة (Minimal)**: ما فيهاش أي حاجة زيادة عن اللي المستخدم محتاجه فعلًا.

أي وظيفة زيادة بتعقّد الدنيا وبتزوّد احتمالات الغلط.

طيب، إزاي نوصل للواجهة البسيطة والمجرّدة دي عمليًا؟
الكتاب اقترح خطوات عملية:

أولًا: نحدد مين المستخدمين.
مين بالظبط اللي هيتعامل مع الكائن ده؟ كلاسات تانية؟ أنظمة خارجية؟

في مثال التاكسي (شكل *2.6*)، المستخدم الأساسي هو الراكب (**Customer**) اللي بيتفاعل مع كائن سواق التاكسي.
تمام.

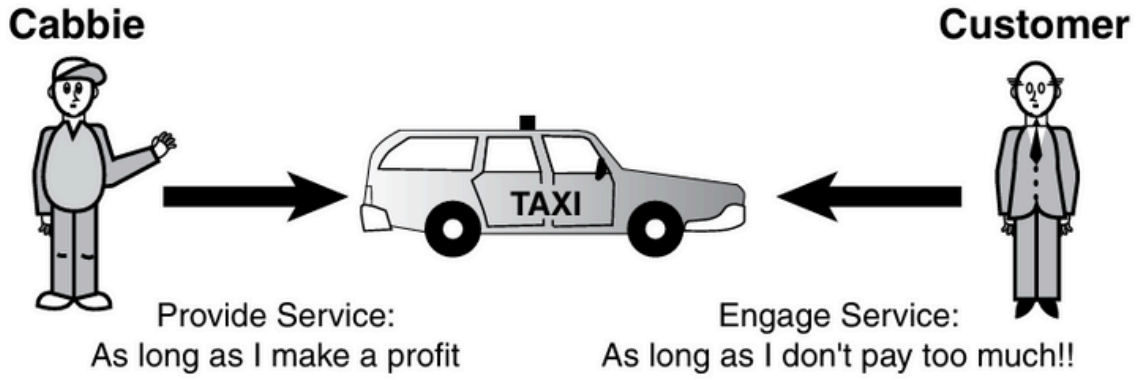


Figure 2.6 Providing services.

تانيًا: نحدد إيه السلوكيات المطلوبة.
بناءً على المستخدمين دول، نشوف إيه الوظائف الأساسية للكائن ده اللي لازم يقدّمها.

زي في مثال التاكسي:

- الراكب محتاج يقدر يوقفه **(Hail Taxi)**.
- يقوله يروح فين **(specifyDestination)**.
- يحاسبه **(payCabbie)**.
- يمكن يدي له بقشيش **(tipCabbie)**.
- وينزل **(Exit Taxi)**.

الكتاب حط قايمة بيهم في شكل *2.7*.

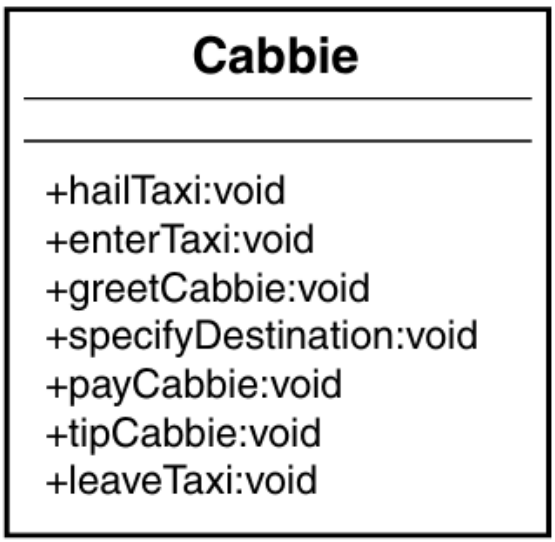


Figure 2.7 The methods in a `Cabbie` class.

منطقي جدًا.

تالتًا: التركيز وفصل الاهتمامات.
كل وظيفة في الواجهة، كل **Public Method**، تركز على مهمة واحدة بس، وما تختلطش المسئوليات.

يعني ما ينفعش وظيفة "ركوب التاكسي" (**EnterTaxi**) يكون جواها كمان جزء من منطق الدفع مثلًا.
عشان تبقى الواجهة Interface واضحة وسهلة.

بالظبط .

ورابعًا وأخيرًا: إخفاء الباقي.

أي تفاصيل داخلية عن إزاي الكائن بيشتغل، أي **Methods** داخلية، أي وظائف مساعدة **Private** بنستخدمها جوه، كل ده لازم يكون مخفي تمامًا.
ده جزء التنفيذ Implementation اللي لازم نحميه ورا الواجهة Interface.

زي مثلًا لو في **تشفير** لكلمة المرور بيحصل جوه، ده لازم يبقى **Private**.
الخطوات دي واضحة أوي وعملية.

بس بيتهيأ لي حتة تحديد إيه هو الـ **Minimal** أو الحد الأدنى ده ممكن تكون صعبة شوية ومحتاجة خبرة؟
يعني إزاي أعرف إن الوظيفة دي فعلًا أساسية ومش مجرد إضافة nice to have؟

سؤال مهم جدًا فعلًا. تحديد (Minimal Interface) بيجي مع الخبرة وفهم السياق، لكن القاعدة العامة:
ابدأ بأقل القليل.
ضيف وظيفة للواجهة بس لما يكون في احتياج حقيقي ومبرر من مستخدم محدد.
وقاوم بشدة رغبة إضافة وظائف "يمكن نحتاجها في المستقبل".

غالبًا لأ.
غالبًا لأ فعلًا.

ولو احتجتها فعلًا تقدر تضيفها بحذر طبعًا عشان ما تكسرش العقود القديمة.
الأسهل دايمًا تضيف للواجهة عن إنك تشيل منها حاجة الناس بقت بتعتمد عليها.

نصيحة ممتازة وعملية جدًا.

طيب لو حبينا نربط كل الخيوط دي ببعض ونلخص أهمية الفصل ده لشخص لسه بيتعلم OOP:
ليه كل التركيز ده على فصل Interface عن Implementation و Abstract thinking ؟ إيه الخلاصة اللي ياخدها معاه؟

الخلاصة إن المبدأ ده – فصل الواجهة والتنفيذ، وتصميم واجهات مجردة وبسيطة – مش مجرد **Technique**، ده استراتيجية أساسية لإدارة وحش التعقيد في البرمجيات.

لما بنطبقه صح بنكسب مميزات كتير أوي:

- بنقدر نقسم الشغل على فرق مختلفة تشتغل مع بعض.
- فريق يبني الواجهة، وفريق يبني التنفيذ.

تقسيم العمل.
بالظبط.

- سهولة الاختبار لكل مكون لوحده.
- سهولة الصيانة والتطوير زي ما شفنا في المثال ال Database.

ده بيخلي الكود مش جامد rigid ومش صعب تغييره.
وبيقلل fragility الهشاشة.

تمامًا.

لما التغيير في حتة ما بيأثرش على حتت تانية مالهاش علاقة، الكود بيبقى أقل هشاشة.
مش كل تغيير صغير يعمل **ريبل إيفِكت** ويكسر الدنيا.

وبيزود **قابلية إعادة الاستخدام** طبعًا.
أكيد الواجهات المجردة الكويسة قابلة لإعادة الاستخدام.

المفاهيم دي — زي تجنّب الجمود والهشاشة — هي أساس مبادئ تصميم أكبر زي **سوليد**.
عشان كده الكتاب بيسميها عملية تفكير ونقلة نوعية.

أنت مش بس بتكتب كود أنت بتفكر في هيكل الكود كله ومستقبله.
فهمت؟ يبقى الرسالة الأساسية من الفصل ده:
ركّز طاقتك وإنت بتصمّم على الواجهة Interface.
فكّر في المستخدم بتاعها — الكود التاني.
خَلِّيها بسيطة ومجرّدة وتعبّر عن الغرض.
أمّا التنفيذ فاعتبره تفصيلة داخلية ممكن تتغيّر، وطالما الواجهة مستقرة... ده هو سر بناء أنظمة قادرة على النمو والتكيّف.

بالضبط كده.
فصل الاهتمامات Separation of concerns هو مفتاح السيطرة على التعقيد.

بكده نكون
غطّينا الفرق الحيوي بين **الواجهة** و**التنفيذ**، وأهمية التفكير التجريدي، وخطوات عملية لتصميم واجهات فعّالة وقابلة للصيانة والتطوير.

الفصل الثالث More Object-Oriented Concepts :

بعد ما خدنا فكرة في الفصل الأول والتاني عن الأساسيات: يعني إيه ، **Object**، **Class**، **Encapsulation** **inheritance**، الحاجات دي كلها.

الفصل ده بقى بياخدنا خطوة لقدام.
مش مجرد بنعرف الكائن **Object**،
لأ، بنشوف إزاي بقى؟ إزاي بيتخلق فعلا ويتجهز للشغل؟ إزاي بيصحى؟ يعني...
بالظبط.

وكمان إزاي بيتعامل لو قابلته مشاكل؟ وإزاي يعرف حدوده؟
يعني مساحته فين بالظبط؟ تمام.

فالنهاردة الكلام هيكون عن مفاهيم زي البنائين **Constructors**، ودول مهمين قوي في البداية.
جدا.

وكمان التعامل مع الأخطاء، الـ **Error Handling**،
عشان البرامج بتاعتنا تبقى متماسكة كده.

ومفهوم تاني مهم: النطاق أو الـ **Scope**،
اللي هو كل جزء يشوف إيه وما يشوفش إيه.

وشوية حاجات متقدمة زي الـ **Operator Overloading**.
جاهز يا سيد؟
بسم الله يا أبو السيد توكل على الله.

والمفاهيم دي مش مجرد تفاصيل زيادة وخلاص،
لأ، دي بتبني على اللي فات.
تمام.

بتخلينا نفهم إزاي الـ Systems الـ OOP بتشتغل بجد بقى،
بشكل عملي وموثوق.

بنتنقل من مرحلة الرسم لفهم التشغيل الفعلي.
حلو قوي.

طيب نبدأ من الأول خالص،
لحظة الولادة بتاعة الـ **Object** ، ال Constructor البنائين.
نقدر نقول إنها زي أول خطوة تجهيز لل **Object** ده ليه مهم قوي كده؟

ال Constructor ده أول كود بيشتغل لما بتعمل new لـ Object،
يعني أول حاجة بتتنفذ.
أول حاجة خالص.

أيوة.
ودوره أساسي جدا إنه يضمن إن الـ Object يبدأ وهو في حالة مظبوطة،
حالة مستقرة وآمنة , بنسميها stable, safe state.

بنشبهها زي ما بنجهز موظف جديد كده، ندي له مكتبه والحاجات بتاعته.
بالظبط كده.
مش بنسيبه تايه.

ال Constructor هو اللي بيعمل التهيئة الأولية دي للـ Object،
يظبط القيم المبدئية للمتغيرات بتاعته.

يعني لو عندنا counter object مثلا،
ال Constructor هو اللي يتأكد إن القيمة تبدأ بصفر.
تماما.

دي وظيفته الأساسية:
تهيئة **attribute** الخصائص بتاعة الـ Object،
عشان لو ما عملناش كده ممكن تاخد قيم غريبة أو قيم افتراضية مش عاوزينها وده ممكن يعمل مشاكل بعدين، يعني تبقى صعبة في تتبعها.

طب الكتاب قال فيه حاجة اسمها **Default Constructor**. ده معناه إيه؟
يعني لو أنا مكتبتش Constructor خالص؟

بص، هو غالبا الدنيا مش هتقف.
لو إنت مكتبتش Constructor بنفسك،
معظم لغات البرمجة بتوفر لك واحد ببلاش كده، افتراضي.

بس وظيفته بسيطة أوي.
كل اللي بيعمله إنه بينادي الـ Default بتاع الكلاس الأب،
الـ Super Class.

آه، بس كده؟
أيوة.

وهنا بقى الخطورة، لو الكلاس بتاعك فيه متغيرات مهمة محتاجة قيم أولية معينة عشان يشتغل صح،
Default Constructor ده مش هيعملها.
يعني ممكن يسيبها فاضية أو بقيم مش مظبوطة؟

عشان كده دايما بنقول: الأفضل تكتب على الأقل واحد بنفسك،
حتى لو فاضي، بس تبقى متحكم في اللي بيحصل.

Providing a Constructor

The general rule is that you should *always* provide a constructor, even if you do not plan to do anything inside it. You can provide a constructor with nothing in it and then add to it later. Although there is technically nothing wrong with using the default constructor provided by the compiler, for documentation and maintenance purposes, it is always nice to know exactly what your code looks like.

فهمت؟

طب نقطة تانية:
هل ممكن الكلاس الواحد يكون ليه أكتر من Constructor؟
الكتاب سماها **Using Multiple Constructors** ولازمتها إيه دي؟

دي بقى حاجة مفيدة جدا، بتدي مرونة عالية قوي،
واسمها Overloading أو التحميل الزائد.

Overloading؟
أيوة.

Signature

```
public String getRecord(int key)
```

Signature = getRecord + (int key)
method name + parameter list

Figure 3.1 The components of a signature.

Thus, the following methods *all* have different signatures:

```
public void getCab();


// different parameter list
public void getCab (String cabbieName);

// different parameter list
public void getCab (int numberOfPassengers);
```

فكر فيها كأنك ممكن تطلب بيتزا.
ممكن تطلبها مارجاريتا، أو بالخضار، أو بالبيبروني.
كلها في الآخر بيتزا، بس طرق طلبها مختلفة.
تمام.

نفس الفكرة هنا: نقدر نوفر كذا Constructor عشان نخلق أو ننشئ **ال object** حسب المعلومات اللي معانا وقت الإنشاء.
ممكن واحد مش بياخد أي Parameters خالص،
واحد تاني ياخد معلومة واحدة،
تالت ياخد معلومتين.

آه، زي المثال بتاع الـ DatabaseReader اللي في الكتاب.

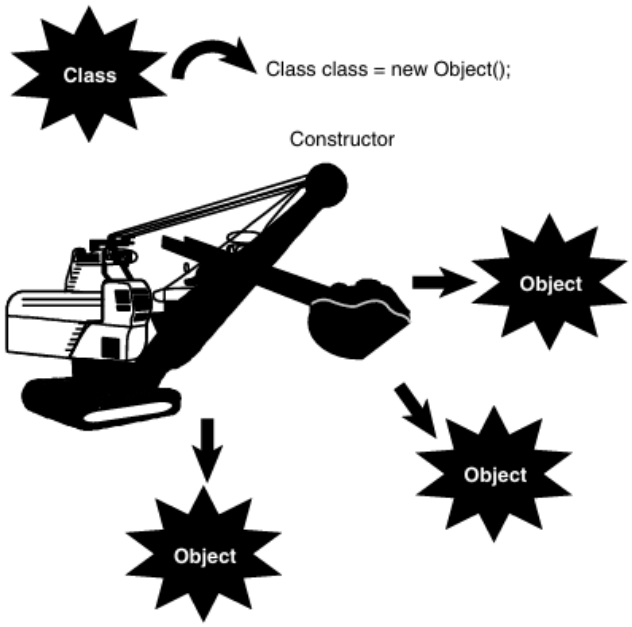


Figure 3.3 Creating a new object.

```
public class DataBaseReader {

    String dbName;
    int startPosition;

    // initialize just the name
    public DataBaseReader (String name){
        dbName = name;
        startPosition = 0;
    };

    // initialize the name and the position
    public DataBaseReader (String name, int pos){
        dbName = name;
        startPosition = pos;
    };

    .. // rest of class

}
```

مرة ندي له اسم الـ Database بس، ومرة ندي له الاسم ومعاه منين هيبدأ يقرا.
بالظبط كده.

ده بيخلي استخدام الكلاس بتاعك أسهل وأوضح لناس تانية.
كل واحد يستخدم الطريقة اللي تناسبه.

بس إزاي الـ Compiler بيفرق بينهم، وكلهم نفس الاسم تقريبا؟
بيفرق بحاجة اسمها التوقيع أو الـ **Signature**.

التوقيع ده عبارة عن اسم الكلاس مع قايمة أنواع الـ Parameters اللي بياخدها.
آه، يعني DatabaseReader بدون Parameters غير DatabaseReader(String) غير DatabaseReader(String, int).
بالظبط.

كل واحد توقيع مختلف،
فالـ Compiler بيعرف يميز بينهم وينادي الصح حسب إنت بعت له إيه.
جميل جدا.

طب لو عايزين نصمم Constructors كويسة بقى؟
إيه أهم النصايح؟

الكتاب كان بيركز قوي على تهيئة كل الـ Attributes.
دي فعلا أهم نصيحة.

القاعدة: ما تسيبش أي متغير حالة جوة الكلاس من غير قيمة أولية واضحة ومقصودة، إنت حاططها بنفسك في الـ Constructor.

يعني ما نعتمدش على القيمة الافتراضية اللي اللغة ممكن تحطها.
أبدا.

لأن القيم دي ممكن تختلف من لغة للتانية،
أو حتى من إصدار للتاني.

فالأهم إنك تدي كل حاجة قيمة بنفسك.
تمام.

والنقطة التانية المهمة إنك تفهم الترتيب.
لما بتعمل new object ال Constructor بتاع الـ Super Class، بتاعه بيتنفذ الأول.
آه، بتاع الأب الأول.
أيوة.

وبعدين الكود اللي جوة Constructor بتاعك إنت.
ده بيربط ال Constructor ب ال inheritance اللي اتكلمنا عنه قبل كده،
وبيضمن إن البناء بيتم طبقة فوق طبقة بشكل سليم.
بالظبط.

لو نسيت تهيئة متغير مهم، ممكن تلاقي الـ Object بيتصرف بشكل غريب بعدين.

كده الـ Object اتولد واتهيأ تمام.
لكن مش دايما كل حاجة بتمشي زي ما إحنا عاوزين.

بتحصل أخطاء.
أكيد طبعا.

وهنا ييجي دور الـ Error Handling أو التعامل مع الأخطاء.
ليه الموضوع ده مهم قوي في تصميم الكلاسات؟

بص، ببساطة: ما فيش برنامج مضمون إنه مش هيغلط أو مش هتحصل له مشاكل غير متوقعة.
يعني الأخطاء دي جزء طبيعي؟
جزء طبيعي جدا من البرمجة.

الشبكة ممكن تقطع.
المستخدم ممكن يدخل حاجة غلط.
ممكن يقسم على صفر بالغلط.
الملف اللي بندور عليه ممكن ما نلاقيهوش.
حاجات كتير ممكن تحصل.
بالظبط.

التصميم الكويس بقى هو اللي بيحسب حساب الحاجات دي،
مش بيتجاهلها.

الـ Error Handling ده هو خط الدفاع بتاعنا.
بيعمل إيه يعني؟
بيمنع الأخطاء دي إنها توقف البرنامج كله فجأة،
أو تخليه يطلع نتايج غلط خالص،
أو يدخل في حالة بايظة كده ومش عارفين نرجعه.

بيدي للبرنامج مناعة وقدرة إنه يتعافى.

طب الكتاب عرض كذا طريقة نتعامل بيها مع الأخطاء دي.
هل ممكن نتجاهلها؟
وده طبعا أسوأ اختيار ممكن،
زي اللي بيدفن راسه في الرمل.

طيب هل ممكن نوقف البرنامج خالص ؟
ده ممكن يكون مقبول في حاجات بسيطة قوي أو أدوات بنستخدمها إحنا بس،
لكن في System كبير بيخدم ناس كتير،
صعب توقفه كله عشان غلطة صغيرة.
صح.

وقال كمان: ممكن نحاول نصلح المشكلة.
دي ممكنة أحيانا.
يعني لو المستخدم دخل رقم غلط، ممكن نطلب منه يدخله تاني.
زي المثال بتاع القسمة على صفر: ممكن نكتشف إن المقام بصفر، فنقوم مغيرينه لواحد مثلا عشان

البرنامج يكمل.
ممكن تعمل كده فعلا.

بس خلي بالك: البرنامج هيكمل آه،
بس النتيجة اللي هتطلع **هتبقى غلط منطقيا.**
آه صح.

وده ممكن يسبب مشاكل أكبر في حتة تانية في الـ System وإنت مش واخد بالك.
فالأفضل هنا غالبا إنك تبلغ إن في مشكلة حصلت.

وهنا ييجي دور الـ **Exceptions**، الطريقة الرابعة اللي الكتاب قال عليها.
بالظبط.

الـ Exceptions تعتبر هي الطريقة الأقوى والأكتر استخداما في معظم لغات الـ OOP الحديثة للتعامل مع الأخطاء.

طب وإزاي بتشتغل دي؟
الكتاب جاب سيرة **Try و Catch**.

Here is the structure for a Java `try/catch` block:

```
try {

    // possible nasty code

} catch(Exception e) {

    // code to handle the exception
}
```

فكرتها ذكية جدا.
هي بتسمح لنا نفصل الكود اللي ممكن يعمل مشكلة عن الكود اللي بيعالج المشكلة لو حصلت.

نفصلهم عن بعض؟
أيوة. بنحط الكود اللي فيه احتمال خطر جوة **Block بنسميه Try.**
تمام.

لو الكود ده ماشي تمام وما حصلش مشاكل، خلاص بيكمل عادي بعد الـ Try.
لكن لو حصل خطأ، أو زي ما بنسميه Exception، جوة الـ Try،
إيه اللي بيحصل؟

التنفيذ العادي بيقف فورا عند نقطة الخطأ دي،
والبرنامج بيبدأ يدور على Block تاني اسمه Catch،
يكون متخصص في التعامل مع النوع ده بالذات من الـ Exceptions.

يعني كل نوع خطأ له Catch بتاعه.
ممكن تعمل كده.

وده الأفضل.
لو لقى Catch مناسب، الكود اللي جوة الـ Catch ده بيتنفذ.

الكود ده بقى هو اللي بيحتوي الموقف.
زي إيه؟ ممكن يسجل الخطأ في ملف Log،
أو يعرض رسالة للمستخدم يقول له المشكلة،
أو يحاول يعمل خطوة بديلة عشان البرنامج يكمل بشكل سليم قدر الإمكان.
فهمت.

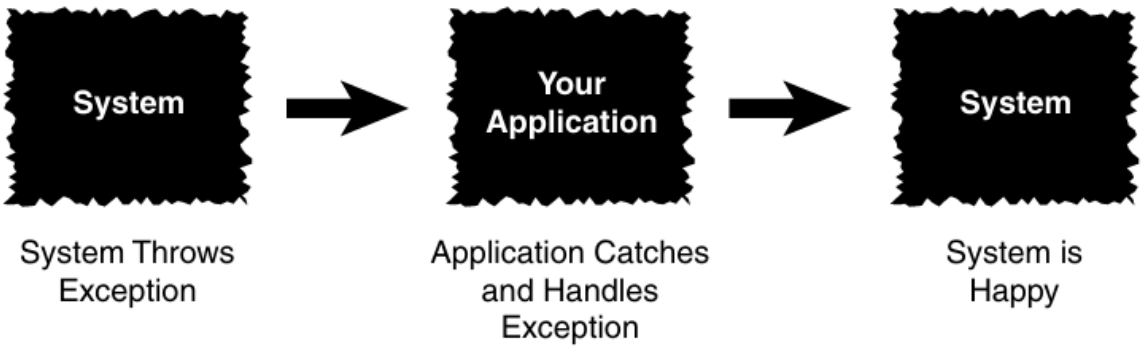


Figure 3.5 Catching an exception.

فكده بنعالج الخطأ من غير ما نلخبط الكود الأساسي بتاع البرنامج.
بالظبط.

دي إيه الميزة الكبيرة؟
الكود الأساسي بيفضل نظيف وواضح،
والتعامل مع الحالات الاستثنائية بيتحط في مكان منفصل ومنظم.

مش بنقعد نكتب Checks عشان نشيك على كل خطوة.

المثال بتاع **ArithmeticException** لما نقسم على صفر بيوضح ده:
الـ Catch بيصطاد الخطأ ده ويتعامل معاه.

```
try {

    // possible nasty code

} catch(Exception e) {

    // code to handle the exception
}
```

The `Exception` parameter in the `catch` block is used to catch any exception that might be generated within the scope of a `try` block.

> **Bulletproof Code**
>
> It's a good idea to use a combination of the methods described here to make your program as bulletproof to your user as possible.

تماما.
الـ Exception ده زي إنذار،
والـ Catch هو خطة الطوارئ اللي بتشتغل لما الإنذار يضرب.
حلو أوي التشبيه ده.

طيب، يبقى اتولد صح،
وعنده مناعة ضد الأخطاء.
فضل إنه يعرف حدوده بقى.

يعرف إيه اللي يقدر يشوفه ويتعامل معاه،
وإيه اللي لأ؟
آه.

وهنا ييجي دور مفهوم مهم تاني:
الـ Scope أو النطاق.

الـ Scope يعني إيه بالظبط؟ وبيأثر إزاي على تصميم الكلاسات؟
الـ Scope ببساطة بيحدد منطقة الرؤية بتاعة المتغيرات والـ Methods جوة البرنامج.

يعني مين مسموح له يشوف مين ويستخدمه؟

ده له علاقة Encapsulation اللي اتكلمنا عنه قبل كده؟
له علاقة قوية جدا.

الـ Encapsulation هدفه يخبي التفاصيل الداخلية،
والـ Scope هو الآلية اللي بنستخدمها عشان ننفذ الإخفاء ده.

بنحدد مين يشوف إيه.
تمام.

الكتاب ميز بين تلت أنواع أو مستويات لل **attributes** حسب الـ Scope بتاعها.
مظبوط.

Methods represent the behaviors of an object; the state of the object is represented by attributes. There are three types of attributes:

- Local attributes
- Object attributes
- Class attributes

أول نوع هو **Local attributes** أو المتغيرات المحلية.
دي اللي جوة الـ Methods.
بالظبط.

دي بتتعرف جوة Method معينة،
وحياتها كلها بتبدأ وتنتهي جوة الأقواس curly braces {} بتاعة الـ Method دي.
ما حدش براها يقدر يشوفها أو يستخدمها.

Local Attributes

Local attributes are owned by a specific method. Consider the following code:

```
public class Number {

    public method1() {
        int count;

    }

    public method2() {

    }

}
```

The method `method1` contains a local variable called `count`. This integer is accessible only inside `method1`. The method `method2` has no idea that the integer `count` even exists.

زي أدوات بنستخدمها في مهمة معينة، وبنسيبها مكانها لما نخلص.

تاني نوع هو **Object attributes** أو متغيرات الكائن.
دي بقى اللي على مستوى الكلاس كله.
هي متعرفة على مستوى الكلاس.
آه.

Object Attributes

In many design situations, an attribute must be shared by several methods within the same object. In Figure 3.6, for example, three objects have been constructed from a single class. Consider the following code:

```
public class Number {

    int count;    // available to both method1 and method2

    public method1() {
        count = 1;
    }

    public method2() {
        count = 2;
    }

}
```

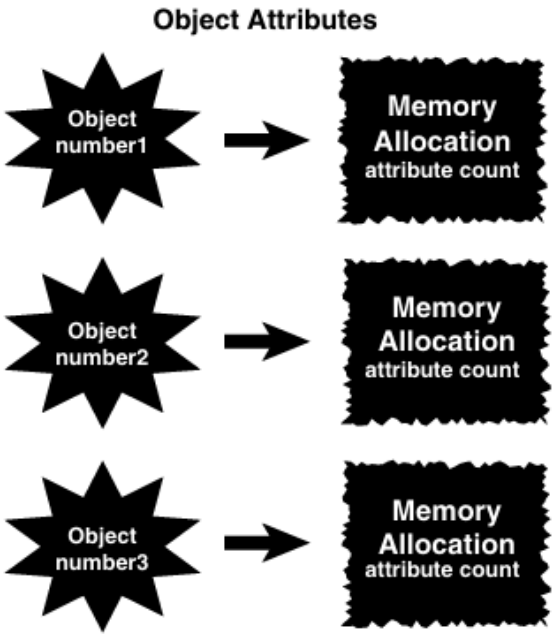


Figure 3.6 Object attributes.

لكن الأهم إن كل كائن بيتعمل من الكلاس ده بيكون ليه نسخة خاصة بيه من المتغيّرات.
يعني لو عملت **objects 3** من كلاس فيه متغيّر **counter**، كل كائن هيكون له العدّاد الخاص بيه.
وبالتالي كل كائن يقدر يشوف ويستخدم النسخة الخاصة بيه زي ممتلكاتك جوه شقتك، هي بتاعتك لوحدك، وكل أوضة في الشقة تقدر توصل لها بحرّيتك.

To illustrate, let's create three copies of the `Number` class:

```
Number number1 = new Number();
Number number2 = new Number();
Number number3 = new Number();
```

Each of these objects—`number1`, `number2`, and `number3`—is constructed separately and is allocated its own resources. There are three separate instances of the integer `count`. When `number1` changes its attribute `count`, this in no way affects the copy of `count` in object `number2` or object `number3`. In this case, integer `count` is an *object attribute*.

نيجي بقى للنوع التالت: **متغيّرات الكلاس** أو الـ **class attributes**، واللي بنعرّفها بكلمة **static**.
ودي مختلفة جدًا، لأنها ما بتعملش نسخ متعددة لكل كائن، **لكن بتكون نسخة واحدة مشتركة بين كل الكائنات Objects اللي اتعملت من نفس الكلاس.**
بمعنى: لو كائن واحد غيّر قيمتها، كل الكائنات التانية هتشوف القيمة الجديدة فورًا... زي لوحة إعلانات محطوطة في مدخل العمارة، أي حد يعلّق ورقة عليها كل السكان يشوفوها.

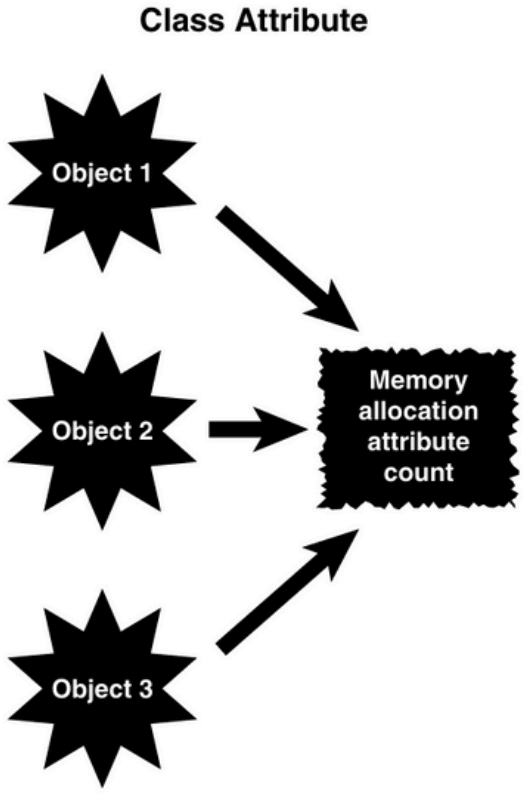


Figure 3.7 Class attributes.

المشكلة إن المتغيّرات الـ static قريبة جدًا من فكرة المتغيّرات العامة (global variables) اللي دايمًا بنحذّر منها.
هي صحيح محصورة جوه الكلاس مش متاحة للبرنامج كله، لكن تفضل خطيرة جدًا ولازم نستخدمها بحذر شديد.

ليه؟ **عشان لو أكتر من Object حاولوا يغيّروا فيها في نفس اللحظة**، ممكن تحصل مشاكل تزامن **concurance** وتبوظ الدنيا.

زي المثال بتاع الـ **account** المشترك بين كائن (count1) وكائن (count2).
بالضبط، ممكن يحصل إن كائن يغيّر القيمة المشتركة، والكائن التاني يكون معتمد على القيمة القديمة في حساباته، فيطلع بنتائج غلط.
وده بيعمل **ترابط قوي tight coupling** بين الكائنات Objects بشكل يخليها معتمدة على بعض.

طب إمتى نستخدم المتغيّرات الـ static أصلًا؟
غالبًا بتكون مفيدة في **الثوابت** اللي الكلاس كله محتاجها بنفس القيمة، أو في عدّادات مشتركة بين الكائنات في أنماط تصميم زي **Factory** أو **Singleton**.
لكن القاعدة العامة: لو مش متأكد 100% إنك محتاجها... يبقى غالبًا إنت **مش محتاجها**.
لأن الأمان في استخدام الـ static صعب، وأي خطأ ممكن يسبب مشاكل كبيرة.

ننتقل دلوقتي لمفهوم تاني ممكن يبان معقّد أو يسبب لخبطة:
تحميل المعاملات الزائد (Operator Overloading).

إيه معناه؟ ببساطة، هو إعادة تعريف وظيفة بعض العلامات أو المعاملات المعروفة زي:
+ , - , == , / , * ... إلخ.
اللغات زي C++ (بشكل كبير) و C# (بدرجة أقل) بتسمح لك تعيد تعريف سلوك المعاملات دي.

مثال مشهور جدًا:
علامة الجمع +. أصلها رياضي للجمع، لكن في لغات زي Java و C# ممكن نستخدمها كـ **concatenation** علشان نربط نصّين ببعض.

However, at times a plus sign could represent something else. For example, in the following code:

```
String firstName = "Joe", lastName = "Smith";

String Name = firstName + " " + lastName;
```

You would expect that `Name` would contain `Joe  Smith`. The plus sign here has been overloaded to perform string concatenation.

يعني بدل ما + بتجمع أرقام، اتعمل لها Overloading عشان تدمج Strings.

مثال تاني: لو عندك كلاس بيمثل **Vector** رياضي، ممكن تخلي + تجمع اتنين Vectors مع بعض.
أو عندك كلاس بيمثل **List**، فتخلي + يضيف عنصر جديد أو يدمج قائمتين.
الهدف من ده إن الكود يبقى شكله طبيعي وأقرب لطريقة التفكير البشري.

لكن... هنا الخطورة!
القوة دي بتيجي معاها مسئولية كبيرة.

لو استخدمت الـ Overloading بشكل غير متوقع أو غير منطقي، الكود ممكن يتحول لطلاسم، ويبقى صعب القراءة والصيانة لأي مبرمج بعدك.

النصيحة:

- استخدم الـ Overloading بحذر شديد.
- اعملها بس لو المعنى الجديد واضح جدًا ومنطقي جدًا في سياق الكلاس.
- لو في أي شك أو لبس، الأفضل تكتب **method عادية** باسم واضح وصريح للوظيفة دي.
- والأهم: **التوثيق** الجيد لكل حاجة بتعملها.

بعد كده الفصل مر بسرعة على مفهومين إضافيين: **الوراثة المتعددة (Multiple Inheritance)** و **عمليات الكائن (object operations)**.

- بالنسبة للوراثة المتعددة: الاتجاه الحديث في معظم اللغات هو الاكتفاء بالوراثة من **كلاس أب واحد**.
  ليه؟ علشان نتجنب مشاكل التعقيد والتضارب اللي بتحصل لو ورثنا من أكتر من كلاس أب.
  المشكلة المشهورة هنا اسمها **Diamond Problem**.

- طب والحل العملي لو محتاج سلوك من أكتر من مصدر؟
  الحل هو استخدام **Interfaces** (زي في Java أو C#).
  الـ Interface بيسمح للكلاس يوعد إنه ينفّذ مجموعة وظائف من غير ما يورث التطبيق نفسه.
  وبالتالي ناخد **المرونة** اللي كنا محتاجينها من الوراثة المتعددة، لكن بشكل **آمن ومنظم أكتر**.

وده بيكون مجرد مقدمة لمواضيع هتتشرح بتوسع أكتر في الفصول الجاية، خصوصًا الفصل الثامن.

تمام.
النقطة التانية كانت **عمليات الكائن (Object Operations)**.
دي كانت إشارة سريعة غالبًا لنقط زي: إزاي نعمل نسخة من كائن Object ، أو إزاي نقارن بين اتنين ونعرف هل هما نفس الحاجة ولا لأ.

الحاجات دي بتبان سهلة، بس تصميمها محتاج تفكير:

- لما بنعمل نسخة: هل بننقل القيم اللي جوه بس (Shallow Copy)؟ ولا لو في كائنات تانية جوه الأصلي، نعمل منها نسخ جديدة كمان (Deep Copy)؟
- ولما بنقارن: هل بنشوف إذا كانوا نفس الكائن في الذاكرة؟ ولا المهم يكون عندهم نفس القيم وخلاص؟

كل دي تفاصيل مهمة جدًا، محتاجة **تصميم دقيق**. وغالبًا ليها كلام أكتر في فصول جاية، زي ما الكتاب قال (وأشار للفصل الخامس مثلًا).

ممتاز.
يبقى كده لو حبينا نلخص رحلتنا في الفصل التالت:

- بدأنا من لحظة ميلاد الكائن بالـ **Constructors**، وعرفنا أهميتها في التهيئة، وإزاي نعمل أكتر من واحد للمرونة.
- بعدين انتقلنا لإزاي نخلي object صامد وقادر يتعامل مع المشاكل الطارئة عن طريق **Error Handling** والـ **Exceptions**، عشان يبقى قوي ومستقر.
- وبعدها فهمنا أهمية تحديد مساحة الرؤية لكل جزء في الكود عن طريق الـ **Scope** بأنواعه المختلفة Local attributes, Object attributes, Class attributes، عشان نحافظ على النظام و (Encapsulation).

يعني انتقلنا من مجرد معرفة إن في حاجة اسمها كائن (Object) لفهم إزاي الكائن ده بيشتغل ويتفاعل بشكل صح وآمن.
وده فهم أساسي جدًا لبناء تطبيقات قوية نقدر نعتمد عليها.

هو ده جوهر الـ **OOP** العملي.

وهنا ممكن نسيب سؤال للمستمعين يفكروا فيه:
لو بصينا على آخر كود كتبناه، حتى لو مشروع بسيط...
هل كان في حتة ممكن نستخدم فيها **Constructor** أوضح؟
أو نعالج خطأ بشكل أحسن؟
أو نحدد **Scope** لمتغير بشكل أذكى؟

بالضبط! هل حاجة زي دي كانت ممكن توفر علينا وقت بعدين؟ أو تخلي الكود أسهل لو رجعنا له، أو حد تاني اشتغل عليه؟

شكرًا لوقتكم مرة تانية، ونتقابل قريب إن شاء الله.

الفصل الرابع The Anatomy of a Class تشريح الكلاس :

الفصل ده اسمه **تشريح الكلاس The Anatomy of a Class**.
يعني كأننا بالظبط بناخد جولة جوة مصنع صغير.
بنشوف كل حتة فيه، كل مكنة بتعمل إيه، عشان في الآخر المنتج يطلع سليم.

المنتج هنا هو ال Object اللي بنتكلم عنه.
وده بيكمل كلامنا اللي بدأناه قبل كده.

لو نفتكر كنا اتكلمنا عن الأساسيات: يعني إيه Object اصلا
وإزاي نفكر في خصائصه وسلوكياته، واتكلمنا عن **Interface ,Implementation** ... فاكر؟ كمان عن **Constructor** و**الـ Scope** بتاع المتغيرات. كل ده؟ بالظبط.

الفصل ده بقى كأنه بيلم كل الخيوط دي،
وبيورينا إزاي بتتربط جوة الكلاس نفسه بقى.

تمام قوي.
وعشان يوضح ده، الفصل بيستخدم مثال واضح ومستمر معانا:
اسمه **Class Cabbie**.

اللي هو سواق تاكسي؟ بالظبط، سواق تاكسي.
وده مش مجرد مثال نظري وخلاص،

لأ، ده بيساعدنا نشوف كل جزء في التشريح ده وهو بيتطبق عملي:
البيانات اللي بيخزنها، الوظايف اللي بيعملها، كده يعني.

```
public class Cabbie {

}
```

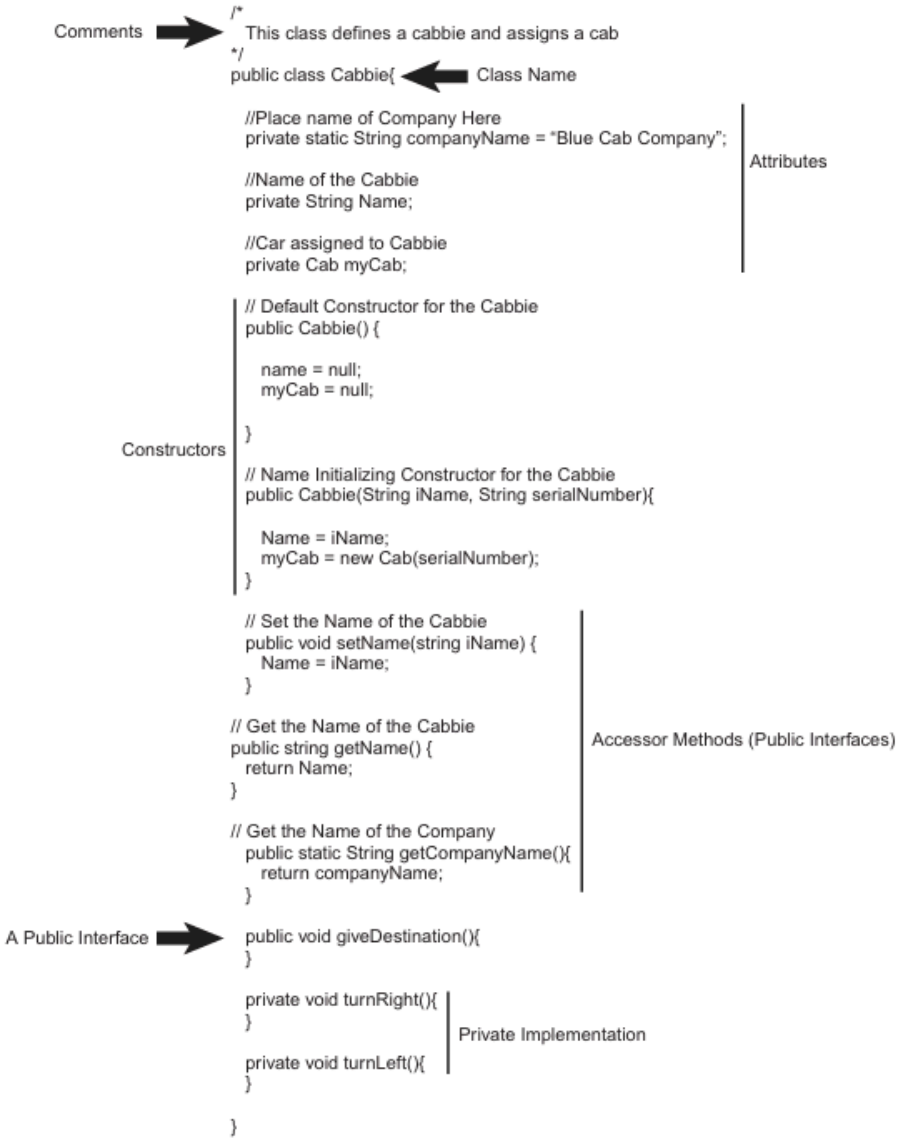


Figure 4.1 Our sample class.

ممتاز.
طيب نبدأ بأبسط حاجة: يمكن الاسم.
الكلاس بتاعنا اسمه **Cabbie**.

يعني اختيار الاسم ده ممكن يبان بديهي شوية.
هو يبان بديهي، بس الكتاب بيلمح إنه أكتر من مجرد كده وخلاص.
لازم يكون له معنى، بيعبر عن هوية الكلاس وبيعمل إيه.

يعني الاسم ده هو الواجهة الأولى،
يُعتبر أول معلومة بناخدها، لازم يبقى واضح ومحدد عشان نفهم دوره في الصورة الكبيرة إيه.

مش بس كده، يعني في لغات زي **Java** مثلًا، في قواعد بتربط الاسم باسم الملف نفسه.
آه صح، لازم يبقوا زي بعض.
بالظبط.

فالموضوع مش مجرد اختيار شخصي،
دي خطوة أساسية عشان الكود يبقى منظم ومفهوم.

اسم زي **Cabbie** هنا واضح جدًا إنه سواق تاكسي،
مافيش لخبطة.
تمام.

طيب، بعد الاسم بنلاقي التعليقات **Comments**.
دي ساعات كتير بنعديها بسرعة كده وإحنا بنقرا الكود.
بس الفصل بخصص ليها جزء و بيتكلم عنها و عن اهميتها.

آه، التعليقات.
ليه بقى؟ يعني إيه أهمية شوية كلام أصلًا مش هيتنفذ؟
لأ، سؤال مهم فعلًا.

التعليقات دي ممكن نعتبرها زي الكتالوج أو دليل المستخدم بتاع الكلاس.
هي صحيح مش جزء من الكود اللي الكمبيوتر بيشغله، أكيد،
لكنها جزء أساسي من الكود اللي إحنا كبشر بنقراه.

الكتاب بيوضح نوعين مشهورين:

- التعليقات اللي هي كذا سطر، بتبدأ وتنتهي بـ /* ... */.
  دي بنشوفها كتير في أول الكلاس مثلًا.
  بالظبط.

زي في مثال **Cabbie**،
التعليق اللي في الأول بيشرح إن الكلاس ده بيمثل السواق بعربيته.
ده شرح عام.

Cabbie class:

```
/*

  This class defines a cabbie and assigns a cab

*/
```

- والنوع التاني اللي هو السطر الواحد، بيبدأ بـ //.
ده عملي أكتر، ممكن جنب سطر كود معين يوضح حاجة بسرعة.
تمام.

comment used with the Cabbie class:

```
// Name of the cabbie
```

بس الأهم بقى من الشكل هو: إيه اللي بنكتبه في التعليق؟ إيه اللي يخليه مفيد بجد؟ دي النقطة.

الكتاب بيأكد على حتة مهمة قوي:
التعليق الكويس مش دوره يقول الكود بيعمل إيه.
المفروض الكود نفسه، لو مكتوب كويس، يكون واضح كفاية ويقول هو بيعمل إيه.

أمال التعليق بيقول إيه؟
التعليق الكويس بيقول: **ليه الكود مكتوب بالشكل ده؟**
إيه الفكرة وراه؟ إيه القرارات التصميمية اللي خدناها؟
إيه الافتراضات اللي بنينا عليها؟

دي المعلومات اللي بتساعد أي حد هييجي بعدك،
أو حتى أنت نفسك بعد فترة، تفهم السياق صح.

يعني التعليقات بتكشف طريقة التفكير،
مش مجرد بترجم الكود لكلام.

آه، دي نقطة حلوة فعلًا.

طيب ندخل على المكون اللي بعده،
وده بقى يمكن صُلب الموضوع: **البيانات (Attributes)**.

آه، دي اللي بتمثل **object state** الحالة بتاعت ال object بالظبط.
وفي **class cabbie** في كذا مكان، أول حاجة لفتت نظري كلمة **Private** اللي بتتكرر دي.

```
private static String companyName = "Blue Cab Company";
```

كلمة **Private** دي تعتبر حجر زاوية في مبدأ مهم قوي في الـ OOP،
اسمه **إخفاء البيانات Data Hiding**.

وده جزء من مفهوم أكبر هو اللي كنا اتكلمنا عنه: **(Encapsulation)**، صح؟
بالظبط.

معناه ببساطة إن الكلاس بيعتبر بياناته دي حاجة خاصة بيه،
زي أسراره.
ماحدش من برة يقدر يوصل لها أو يغيرها بشكل مباشر كده.

كأن الكلاس بيقول: دي بياناتي وأنا بس المسئول عنها وعن سلامتها.

خلينا نبص على أنواع الـ Attributes في المثال.
في واحد شكله مختلف شوية.

```
private static String companyName = "Blue Cab Company";
```

كلمة **Static** دي بتعمل إيه؟
Static دي كلمة لها وزنها.
لما بنشوفها قبل Attribute، ده معناه إن ال Attribute ده مش ملك لكل وحدة (Object)،
لأ، ده ملك للكلاس **Cabbie** نفسه ككل.

يعني إيه ملك للكلاس نفسه؟
يعني تخيل مثلًا إن كل سواقين التاكسي اللي هنعملهم في نفس الشركة،
فاسم الشركة ده هيكون واحد ليهم كلهم.
لو غيرنا اسم الشركة في مكان واحد، هيتغير عند كل السواقين فورًا.

بنسميه **Class Attribute**.

(see Figure 4.2).

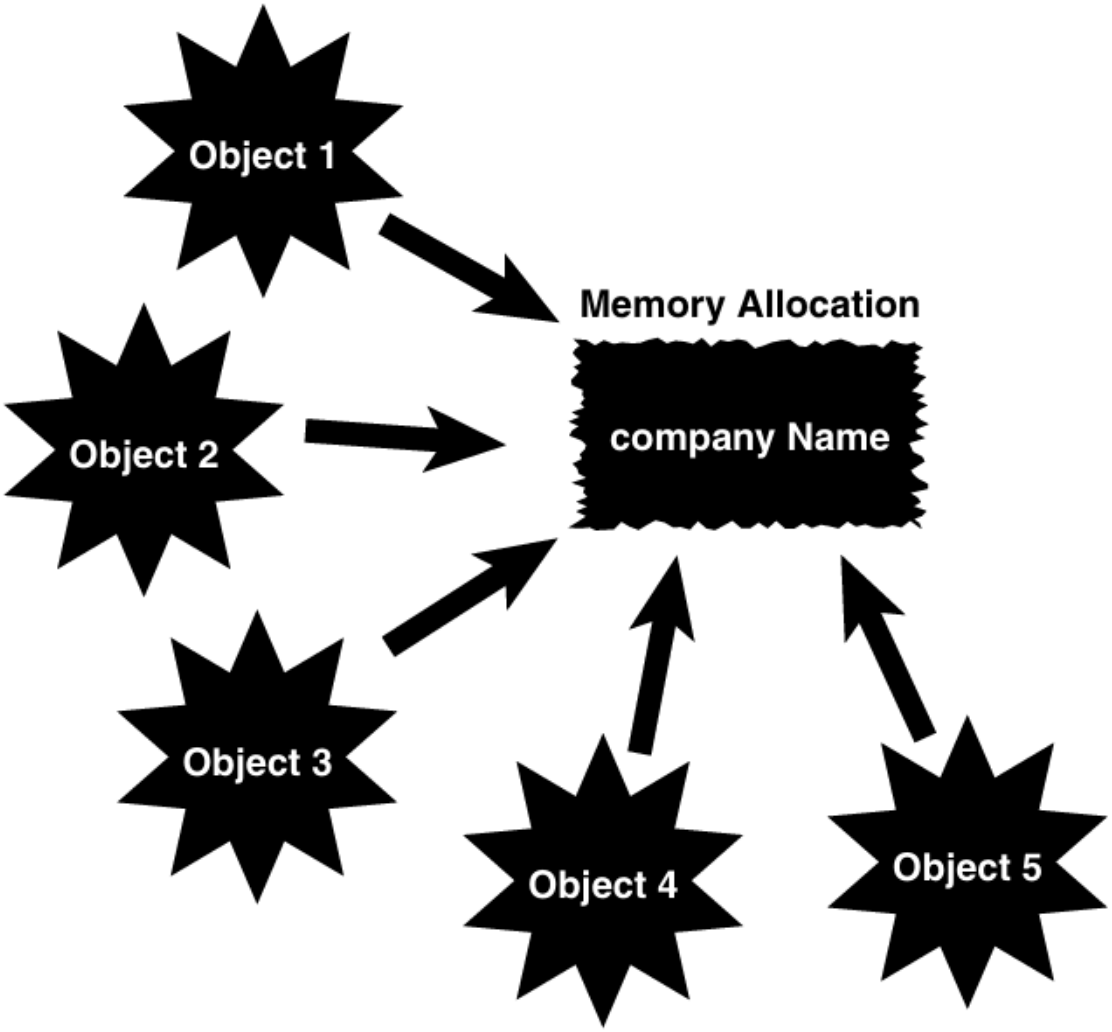


Figure 4.2 Object memory allocation.

فهمتك.
زي مثلًا لو عندنا كلاس **بيت**،
وكل البيوت في شارع واحد، فاسم الشارع ده هيبقى **Static**.
لكن رقم كل بيت ده خاص بالبيت نفسه.

بالظبط كده، تشبيه ممتاز.

ده معناه إن التانية بقى اللي مش Static،

```
private String name
private Cab myCab
```

زي الـ name اللي هو **string** وزي الـ myCab اللي هو من نوع **Cab** تاني خالص،

دول بقى بنسميهم **Object Attributes**،
ودول خاصين بكل Object لوحده.

تمامًا.
كل **Cabbie** Object جديد بنعمله،
بيكون له نسخة مستقلة وخاصة بيه من الـ name والـ myCab.

سواق اسمه أحمد وسواق تاني اسمه أبو السيد،
كل واحد له اسمه وعربيته.

والنقطة بتاعة الـ myCab دي، اللي هي من نوع **Cab**،
بتوضح فكرة تانية كمان، مش كده؟
آه، دي بتوضح فكرة **Composition** أو التجميع.

إن Object الـ Cabbie بيحتوي جواه أو بيستخدم Object اللي هو الـ Cab.
ده بيورينا إزاي الـ Objects ممكن تتركب من بعضها زي قطع الليجو كده.

جميل جدًا.

يبقى كده عندنا نوعين بيانات:

- بيانات على مستوى الكلاس كله (**Static**)
- وبيانات خاصة بكل Object (**Instance**)

وكلهم Private.

نرجع بقى: لو هما Private العالم الخارجي بيتعامل معاهم إزاي؟
هنا بقى بييجي دور الـ Methods.

بس قبل ما نتكلم عن كل أنواع الـ Methods،
خلينا نبص على نوع خاص جدًا منهم:
اللي هو **Constructors** (المنشآت).

آه، دي اللي بنستخدمها مع كلمة **new**.
بالظبط، دي المسئولة عن عملية خلق أو بناء Object جديد.

Constructors

This `Cabbie` class contains two constructors. We know they are constructors because they have the same name as the class: `Cabbie`. The first constructor is the default constructor:

```
public Cabbie() {

    name = null;
    myCab = null;

}
```

أول ما نكتب new Cabbie()، في المثال بتاعنا واضح إن في اتنين Constructors، صح؟
وجود أكتر من Constructor بنفس الاسم بس بقيمة Parameters مختلفة،
ده تكنيك مشهور اسمه **Overloading**.

تمام.

الأولاني **( ) Public Cabbie** ده ما فيهوش أي Parameters.
ده اللي بيسموه ساعات **Default Constructor**.
أو يمكن الأدق إنه **No-argument Constructor**، وغالبًا أنسب.

دوره إيه ده بقى؟
هو بيوفر طريقة نعمل بيها Object حتى لو ما عندناش أي معلومات نديها له.

المهم بقى نشوف بيعمل إيه جواه:
بيحط قيم مبدئية زي null أو صفر للأرقام.

ال null دي معناها ببساطة: ما فيش قيمة، أو فاضي.

فكرة إننا نبدأ بقيم واضحة زي null أو 0 للـ numbers
دي عادة كويسة جدًا.

ليه؟
عشان بتخلي الحالة معروفة من أول لحظة في الـ Object.

تخيل لو ما عرفناش قيمة الـ myCab دي هتبدأ بإيه؟
ممكن الكود بعد كده يحاول يستخدم عربية وهي أصلًا مش موجودة،
فيحصل مشكلة.

فدي خطوة احترازية مهمة جدًا.
تمام.

التاني: public Cabbie(String iName, String serialNumber)،

The second constructor provides a way for the user of the class to initialize the Name and myCab attributes:

```
public Cabbie(String iName, String serialNumber) {

    name = iName;
    myCab = new Cab(serialNumber);

}
```

In this case, the user would provide two strings in the parameter list of the constructor to properly initialize attributes. Notice that the myCab object is instantiated in this constructor:

```
myCab = new Cab(serialNumber);
```

ده بقى بياخد بيانات معاه وهو بينشأال Object.
أيوة، ده بيدينا طريقة تانية للبناء بمعلومات جاهزة من الأول.

وده بيسمح لنا نعمل Cabbie جديد وندي له اسمه ورقم سيريال عربيته في خطوة واحدة.
بالظبط.

وهنا في نقطة مهمة جوة الكود بتاعه:
السطر الأول ده بسيط،

```
name = iName
```

بيحط الاسم.

لكن السطر التاني، الخاص بالـ serialNumber،
ده بيورينا حاجة تانية.

```
myCab = new Cab(serialNumber)
```

بيعمل new لـ Object تاني.

أيوة.
مش بس بيحط قيم،
ده ممكن كمان ينشأ Object تانية جواه.

هنا هو بيعمل Cab جديد وبيخلي myCab يشاور عليه.

ده بيأكد تاني فكرة إن الـ Objects بتعتمد على بعضها وبتتعاون عشان تعمل شغلها.

ولاحظت كمان إن الاتنين Constructors واخدين **Public**؟
ده الطبيعي والغالب.

Constructors هي تعتبر البوابة اللي بندخل منها عشان نخلق Object من الكلاس ده.
لو عملناها **Private**، ماحدش هيقدر يعمل new Cabbie() إلا من جوة الكلاس نفسه.

ودي لها استخدامات خاصة ومتقدمة شوية،
زي في **Singleton Pattern**.
لكن العادي إنها تكون Public.

أوكي.

يبقى كده لخصنا:

- اسم الكلاس
- التعليقات للشرح
- بيانات داخلية Private بأنواعها
- (Constructors) **Public** بتبني الـ Object وتدي له حالته الأولية

نرجع بقى للسؤال اللي كان معلَّق من شوية:
إزاي نتعامل مع البيانات الـ Private دي من برة الكلاس؟

هنا بقى بنوصل للنوع التاني من الـ Methods،
اللي هي **Accessor Methods**.
واللي أغلبنا يعرفها باسم **Getters** و **Setters**.

آه، بالظبط.

دي تعتبر الواجهة المتحكم فيها للوصول للبيانات الداخلية.
بدل ما أي كود خارجي يقدر ييجي يغير قيمة الـ name كده بمزاجه،
طبعًا لأ، لازم يطلب ده بشكل رسمي من الكلاس نفسه عن طريق الـ Methods دي.

كأن في موظف استقبال:
الـ Getter أو الـ Setter.

هو بس اللي مسموح له يدخل المكتب الداخلي (اللي هو الـ Private Attribute)،
ويجيب معلومة أو يوصل ورقة.

تاني، في مثالنا:

```
// Set the Name of the Cabbie
public void setName(String iName) {
    name = iName;
}

// Get the Name of the Cabbie
public String getName() {
    return name;
}
```

- في Getter بترجع قيمة الاسم.
- وفي Setter بتسمح بتغيير الاسم.

دي تبقى الصورة بالظبط.

وده بيحقق لنا مبدأ **Encapsulation** بشكل عملي جدًا.
الكلاس بيغلف بياناته وسلوكياته وبيوفر طرق محددة جدًا للتعامل مع البيانات دي من برة.

طب وده مهم ليه قوي كده؟
عشان حاجة اسمها **سلامة البيانات (Data Integrity)**.

يعني الـ Setter دي ممكن جواها تتأكد إن الاسم الجديد ده مش فاضي مثلًا،
أو إنه بيتبع شكل معين،
أو تعمل أي **validation** مطلوب قبل ما تغير القيمة فعلًا.

لو كانت الـ name دي Public، ماكانش هيبقى فيه أي سيطرة أو حماية على القيمة اللي بتتحط فيها.

وده يرجعنا لنقطة **إخفاء البيانات Data Hiding** تاني.
إحنا مش بس بنخبي البيانات عشان ننظم الكود،
إحنا بنخبيها عشان نحميها ونضمن إن التعامل معاها بيتم صح من خلال البوابات دي، اللي هي الـ Getters والـ Setters.
تمامًا

وفكر فيها كمان من ناحية **تصحيح الأخطاء**.

لو حصل خطأ في قيمة الـ name وهو Public،
هتحتاج تدور في المشروع كله عشان تعرفي مين السبب.

لكن لو التغيير بيحصل بس من خلال الـ Setter،
المكان اللي هتدور فيه بقى معروف ومحدود جدًا.

وبيسهل الصيانة جدًا كده.
بالظبط.

هي كمان ممكن تكون **Static** زي الـ Attribute بتاعها.
ده له نفس المعنى.

أيوه، نفس المنطق.
زي ما في **Static Attribute**، اللي هي class attribute
في كمان **Static Method**. اللي هي class method

ال method دي مش مرتبطة بـ Object معين،
لأ، دي مرتبطة بالكلاس **Cabbie** ككل.

وعشان كده نقدر نناديها باسم الكلاس على طول،
زي: Cabbie.getCompanyName()

Notice that the `getCompanyName` method is declared as `static`, as a class method; class methods are described in more detail in Chapter 3. Remember that the attribute `companyName` is also declared as `static`. A method, like an attribute, can be declared `static` to indicate that there is only one copy of the method for the entire class.

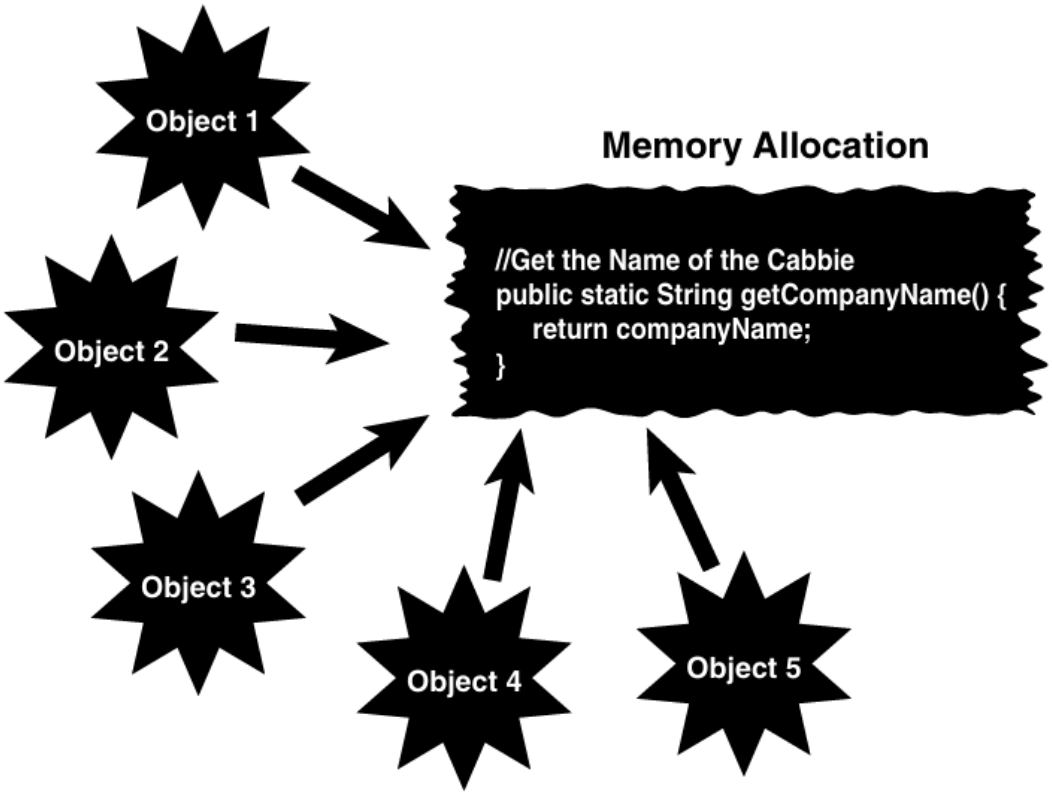


Figure 4.5 Method memory allocation.

بالظبط كده،
من غير ما نحتاج نعمل new Cabbie الأول.

منطقي إن الميثود اللي بترجع معلومة مشتركة بين كل Objects تكون هي كمان Static.
واضحة جدًا الفكرة دي.

يبقى ال**Accessor Methods اللي هما الـ Getter أو الـ Setter**
دول جزء مهم من الواجهة (**Interface**) اللي الكلاس بيقدمها للعالم الخارجي.

بس أكيد الكلاس مش مجرد **مخزن بيانات** بنسأل منه وبنديله حاجات وخلاص.
إيه بقى الوظايف الفعلية اللي بيعملها؟ **السلوكيات بتاعته**.

هنا بقى بيجي دور النوع التالت من الـ Methods،
اللي هي **Public Interface Methods**.

دي اللي بتمثل الخدمات الأساسية اللي الكلاس ده بيقدمها.

الكتاب بيأكد إن الواجهة دي المفروض تكون مجردة شوية:
تعبر عن "إيه اللي الـ Object ده يقدر يعمله" (**What**)
ومش بالضرورة "إزاي بيعمله" (**How**) بالتفصيل.

زي إيه؟
في مثالنا بتاع cabbie عندنا public void giveDestination(String dest).

Public Interface Methods

Both the constructors and the accessor methods are declared as public and are part of the public interface. They are singled out because of their specific importance to the construction of the class. However, much of the *real* work is provided in other methods. As mentioned in Chapter 2, the public interface methods tend to be very abstract, and the implementation tends to be more concrete. For this class, we provide a method called `giveDestination` that is the public interface for the user to describe where she wants to go:

```
public void giveDestination(){

}
```

What is inside this method is not important at this time. The main point is that this is a public method, and it is part of the public interface to the class.

الميثود دي متاحة لأي حد معه Object من نوع Cabbie،
يقدر يناديها ويقول له: "أنا عايز أروح الحتة الفلانية".

بغض النظر بقى هو هيروح إزاي من جوة.
تمام.

ده جزء من العقد (**Contract**) اللي الكلاس بيقدمه.
أي Cabbie لازم يقدر يستقبل وجهة.

إزاي بقى هو بينفذ الرحلة دي؟
دي قصة تانية ممكن تكون معقدة جدًا جوة.

بس **المستخدم اللي برة مش محتاج يعرف التفاصيل** دي كلها.
هو بس بينادي giveDestination()

طيب، بما إننا قلنا إن التفاصيل الداخلية المفروض تكون مخفية،
ده معناه إن مش كل Methods لازم تكون Public برضو.

يبقى فيه Methods بيستخدمها الكلاس لنفسه بس؟
أكيد.

Private Implementation Methods

Although all the methods discussed so far in this chapter are defined as `public`, not all the methods in a class are part of the public interface. It is common for methods in a class to be hidden from other classes. These methods are declared as `private`:

```
private void turnRight(){
}

private void turnLeft() {
}
```

These private methods are meant to be part of the implementation and not the public interface. You might ask who invokes these methods, if no other class can. The answer is simple: You might have already surmised that these methods are called internally from the class itself. For example, these methods could be called from within the method `giveDestination`:

```
public void giveDestination(){

.. some code

    turnRight();
    turnLeft();

.. some more code

}
```

وده النوع الرابع والأخير من المكونات اللي الفصل بيشرحها.
زي ما بنعمل متغيرات، ممكن كمان نعمل بعض الـ Methods برايفت.
دي بتبقى عبارة عن وظائف مساعدة، الكلاس يستخدمها داخليًا عشان ينفذ شغله ويعمل مهامه.

يعني مش جزء من الواجهة العامة؟
لأ خالص، دي تفاصيل داخلية.

في مثال الكابي في **private void turnRight** و **private void turnLeft**.
دي أكيد الكابي بيستخدمها وهو بيحاول يوصل لل destination، بس أنا كمستخدم للكلاس ما أقدرش أقول له يلف يمين بنفسي كده.

بالظبط، دي تفاصيل تنفيذية بحتة.
الكلاس هو اللي بيقرر إمتى وفين محتاج يلف يمين أو شمال عشان يوصل، ما حدش برة المفروض يتدخل في الحاجات دي.

ممكن نفكر في مثال تاني:
كلاس بيتعامل مع كلمات سر مثلًا، ممكن يكون جواه method private بتعمل تشفير معقد.
الـ **setPassword** هي اللي بتنادي الـ private method دي، لكن ما حدش من برة يقدر ينادي التشفير مباشرة.

يعني الـ private methods دي بتساعدنا نفصل بين الـ interface (اللي هو الواجهة) وبين التنفيذ (اللي هو الـ implementation) والتفاصيل الداخلية بشكل أقوى كمان.

بالظبط، المستخدم بيتعامل مع الواجهة وبس.
والكلاس حر يغير في تفاصيل وتنفيذه الداخلي زي ما هو عايز طالما الواجهة دي ثابتة وما بتتغيرش.

وده من أهم مكاسب الـ encapsulation مش كده؟
طبعًا، ده بيخلي الكود أكتر مرونة.
يعني كل جزء بيبقى معزول وقائم بذاته.

لو بكرة اكتشفنا طريقة أحسن وأسرع عشان method تشتغل، نقدر نغيرها جوة الكلاس، وكل الأكواد التانية اللي بتستخدم الكلاس مش هتحس بأي فرق ومش هتحتاج تتغير هي كمان.

الصيانة والتطوير بيبقوا أسهل بكتير على المدى الطويل.

كده الصورة اكتملت بشكل كبير قوي.
الكلاس فعلًا زي ما شبهناه في الأول عامل زي تصميم هندسي دقيق، أو زي ما قلت مصنع صغير منظم له اسم واضح ومعبر، وتعليقات بتشرح فكرته وليه اتعمل كده، وبيانات بتمثل حالته سواء خاصة بكل object أو مشتركة للكلاس كله، ومنشآت بتبنيه وتجهزه بالحالة الأولية السليمة، ومجموعة من السلوكيات أو الوظائف متقسمة في واجهة عامة للعالم الخارجي (public) وفي تفاصيل تنفيذية مخفية (private).

وكل خطوة في التشريح ده بتتضمن قرارات تصميمية مهمة لازم المبرمج ياخد باله منها وهو بيكتب الكلاس:
إيه اللي نخبيه private؟ وإيه اللي نظهره public؟
هل محتاجين بيانات ولا لأ؟
إزاي نضمن إن الـ object بيبدأ صح ؟
إيه الوظائف الأساسية للكلاس ده اللي لازم يقدمها؟
وإيه التفاصيل اللي مفروض تبقى داخلية وما حدش يشوفها؟

فهم التشريح ده هو اللي يخلينا نعرف نجاوب على الأسئلة دي.
بالظبط.

وده بيربط كل اللي قلناه بمبادئ الـ OOP الكبيرة اللي كنا بنتكلم عنها في الفصول اللي فاتت.
الموضوع مش مجرد إننا بنطبق قواعد وخلاص،
لأ، ده له هدف.

فعلاً لما فصصنا الكابي كلاس ده حتة حتة، إحنا شفنا بنفسنا تجسيد عملي لمفاهيم أساسية زي **التغليف**:
إزاي البيانات والسلوكيات متجمعين مع بعض في وحدة واحدة هي الـ object.

تانيًا: **إخفاء البيانات**، أهمية الـ private عشان نحمي الحالة الداخلية للـ object، ودي مهمة جدًا.

تالتًا: **فصل الواجهة عن التنفيذ**، الفرق الواضح بين العقد أو الواجهة وبين التفاصيل الداخلية (طريقة تحقيق العقد ده).

الفصل ده بيورينا إن المبادئ دي مش كلام نظري، دي أدوات بنستخدمها فعلاً وإحنا بنبني الكلاسات بتاعتنا.

يعني فهم التشريح الداخلي للكلاس ده هو الخطوة الأولى عشان نقدر نبني objects مش بس شغالة وخلاص، لكن كمان تكون قوية، سهلة الفهم، والأهم قابلة للصيانة والتطوير في المستقبل.

بالظبط كده، جودة تصميم الكلاس من جوة بتنعكس بشكل مباشر على جودة الـ objects اللي بيطلع منها، وعلى قدرة الكلاس ده إنه يشتغل كويس ويتفاعل مع الكلاسات التانية في نظام أكبر وأعقد.
بناء كلاس صح من جوة هو أساس بناء برامج صح كلها.

ودي نقطة ممتازة نقف عندها ونفكر فيها شوية:
بما إننا عملنا الجولة دي جوة الكلاس وشفنا التركيبة الداخلية الدقيقة دي بكل تفاصيلها وبكل القرارات التصميمية اللي وراها،
تفتكر إزاي التركيبة الداخلية دي بتأثر بعد كده على شكل العلاقات والتفاعلات اللي بتحصل بين الكلاس ده والكلاسات التانية في السيستم ككل؟

يعني إزاي اللي جوة بيحدد شكل اللي برة؟
سؤال بيفتح الباب على حاجات كتير جاية بعد كده.

فهم التركيب الداخلي ده، التشريح ده، هو الأساس اللي هنبني عليه فهمنا لحاجات أكبر،
زي مثلًا: إيه القواعد أو الإرشادات اللي المفروض نمشي عليها عشان نصمم كلاسات كويسة بشكل عام؟

وده موضوع الفصل الخامس Class Design Guidelines سلام.

الفصل الخامس Class Design Guidelines:

في الفصول اللي فاتت كنا اتكلمنا عن أساسيات البرمجة الشيئية، يعني إيه **class** , **Object** ، والمفاهيم الأساسية زي

Encapsulation، وكمان أهمية إننا نفصل بين الواجهة **Interface** والتنفيذ **Implementation** بالظبط.
ودي كانت زي ما تقول كده الأساسيات اللي بنبني عليها فهم أعمق للموضوع.

النهارده بقى المصدر اللي معانا هو الفصل الخامس من الكتاب.
الفصل ده عنوانه أو يعني "إرشادات تصميم الكلاسات" Class Design Guidelines.
فإحنا دلوقتي بنتنقل من مرحلة التعريفات اللي هو: إيه هي المكونات ال oop، لمرحلة التطبيق.

يعني: إزاي أستخدم المكونات دي عشان أبني كلاسات كويسة بجد؟ وده سؤال مهم قوي لأي حد عايز يكتب كود مش بس بيشتغل وخلاص، لأ، كود يكون سهل يتفهم، وكمان يتطور بعد كده.

تمامًا. والكتاب زي ما بيقدم مجموعة إرشادات بصراحة قيمة جدًا، يعني هي جاية من خبرة عملية وتجارب ناس كتير في المجال.
المهم بس نفهم إن دي مش قوانين رياضية يعني: واحد زائد واحد يساوي اتنين. لأ، هي أشبه بمبادئ بتوجّهنا للطريق الصح. الهدف منها في الآخر إننا نوصل لكود يكون قوي، مرن، ومفهوم.

ممكن يعني نفكر فيها زي نصايح معماري خبير بيقول لك: إزاي تبني أساسات بيت سليم، أساسات تستحمل الزمن وتستحمل التغيير.

جميل قوي.
طيب خلينا نبدأ بأول مجموعة أفكار الفصل بيركز عليها، واللي ليها علاقة مباشرة بفكرة "العقد" اللي الكلاس بيقدمه للعالم الخارجي.

الكتاب كان اتكلم قبل كده كتير عن أهمية نمذجة العالم الحقيقي، بس هنا في الفصل ده بيأكد كمان على تقليل الواجهة العامة وإخفاء التفاصيل.
إزاي الأفكار دي بتمشي مع بعض؟ بتمثل جوهر الـ Encapsulation نفسه.

الفكرة مش إننا نعمل محاكاة فوتوغرافية للواقع بكل تفاصيله، لأ خالص.
إحنا بنبني نموذج مفيد وعملي للمشكلة اللي إحنا بنحاول نحلها.**ال Class** المفروض يمثل مفهوم منطقي في السيستم بتاعنا. زي المثال اللي الكتاب بيستخدمه بتاع سواق التاكسي أو الكابّي.

لكن الأهم من ده إن الكلاس لازم يحدد بوضوح شديد: إيه الخدمات اللي هو بيقدمها للعالم الخارجي؟ دي اللي بنسميها الواجهة العامة أو **Public Interface**.
وإيه التفاصيل اللي هو بيخبيها جواه؟ وده اللي بنسميه **Implementation**.

يعني الواجهة دي زي المنيو كده، قائمة الخدمات اللي بيقدمها.
بالظبط كده.

الواجهة دي هي العقد، الاتفاق بين الكلاس وبين أي كود تاني هيستخدمه. عشان كده لازم تكون محدودة وواضحة جدًا، وما تقدمش غير اللي المستخدم محتاجه فعلًا.

زي ما الكتاب بيقتبس مقولة لجيلبرت ماكارتي:

Identifying the Public Interfaces

It should be clear by now that perhaps the most important issue when designing a class is to keep the public interface to a minimum. The entire purpose of building a class is to provide something useful and concise. In their book *Object-Oriented Design in Java*, Gilbert and McCarty state that "the interface of a well-designed object describes the services that the client wants accomplished." If a class does not provide a useful service to a user, it should not have been built in the first place.

واجهة الكائن المصمم جيدًا تصف الخدمات التي يريد العميل إنجازها.

يعني بتركّز على "إيه المطلوب"، مش "إزاي بيتعمل". تمام.
طب ليه الإصرار ده كله على إخفاء التفاصيل؟ يعني مش ده ممكن يصعّب الدنيا شوية لو أنا كمبرمج احتجت أوصل لحاجة جوة الكلاس ده؟ هو ممكن يبان كده في الأول، لكن الفايدة على المدى الطويل بتبقى ضخمة جدًا.

القاعدة الذهبية هنا زي ما الكتاب بيقول: كل حقول البيانات (Fields) لازم تكون خاصة **Private**.
ليه بقى؟ عشان ده بيدينا حرية إننا نغيّر طريقة شغل الكلاس من جوة. يعني نصلح أخطاء، نحسّن الأداء، نعمل أي حاجة من غير ما نكسر الكود اللي بيستخدم الكلاس ده.

- آه يعني طالما الواجهة العامة ثابتة؟
- بالظبط. طالما الواجهة العامة **Contract** (العقد) ثابت، اللي برّه مش هيحس بأي تغيير بيحصل جوة.

وده بيخلي الصيانة أسهل بمراحل. تخيل لو في مشكلة في طريقة اسم السواق. بما إن الخاصية دي **Private** وما حدش بيوصل لها مباشرة من برّه، أنا كمبرمج ببقى عارف إن المشكلة وحلها محصورين جوة حدود الكلاس ده وبس.

ده بيقلل التعقيد، وبيمنع إن الأخطاء تنتشر في السيستم كله زي النار في الهشيم.

- فهمتك.
يبقى المجموعة الأولى دي من الإرشادات بتركّز قوي على بناء حدود واضحة للكلاس، يعني واجهة عامة تكون دقيقة ومحدودة، وتفاصيل داخلية تكون محمية ومخفية.

وده اللي بيخلي الكلاس في الآخر وحدة بناء مستقلة وموثوقة نقدر نعتمد عليها.

كده، وبالمناسبة فكرة الواجهات دي نفسها اتطورت كمان مع الوقت. دلوقتي كتير بنستخدم **Interfaces** بشكل صريح عشان نحدد السلوكيات المطلوبة. وممكن نلاقي أنواع تانية من الكلاسات البسيطة زي الـ DTOs أو **Data Transfer Objects** اللي مهمتها مجرد نقل بيانات، وده بيخلي فصل المسئوليات أوضح وأوضح كمان.

- تمام.

طيب بعد ما حددنا شكل الكلاس من برّه وحدوده، إيه النصايح اللي بتخلي الكلاس نفسه من جوة يبقى قوي ومستقر؟

الفصل بيتكلم عن حاجات زي **ال Constructors و ال Destructors** وكمان التعامل مع الأخطاء. دي مجموعة تانية مهمة جدًا من الإرشادات، بتركّز على صحة الكائن طول فترة حياته، من ساعة ما يتولد لحد ما ينتهي.

أول حاجة **ال Constructors**: اللي هي نقطة البداية، لحظة الميلاد بتاعة الـ Object. لازم نتأكد تمامًا إن أي Object بيتولد وهو في حالة سليمة، مستقرة، ومفهومة.

يعني إيه حالة سليمة ومستقرة؟ يعني لازم نعمل **Initialization**، يعني ندي قيم أولية لكل الخصائص **attribute** بتاعته بقيم منطقية ومناسبة جوه نفسه، وما نعتمدش أبدًا على القيم الافتراضية اللي اللغة ممكن تديها لوحدها، لأنها ممكن ما تكونش مناسبة.

لازم ال object يبدأ حياته وهو جاهز للشغل وما فيش حاجة ناقصة.

طب و **ال Destructors** دي بصراحة مش بنشوفها كتير في لغات زي Java أو C#.

- صحيح. وده لأن اللغات دي فيها حاجة اسمها **Garbage Collector**، هو اللي بيقوم بمهمة تنظيف الذاكرة بالنيابة عن المبرمج.

لكن في لغات تانية زي C++، دوره حيوي ومهم جدًا. هو المسؤول إنه يتأكد إن أي موارد الـ Object ده حجزها طول فترة حياته، زي ذاكرة مفتوحة، اتصالات بالشبكة، أي حاجة، يتم تحريرها بشكل سليم لما خلاص دوره ينتهي.

In languages that include destructors, it is of vital importance that the destructors include proper clean-up functions. In most cases, this clean-up pertains to releasing system memory that the object acquired at some point. Java and .NET reclaim memory automatically via a garbage collection mechanism. In languages such as C++, the developer must include code in the destructor to properly free up the memory that the object acquired during its existence. If this function is ignored, a memory leak will result.

- آه يعني لو ده ما حصلش؟
- لو ده ما حصلش، بيحصل حاجة اسمها تسرب للموارد، أشهرها تسرب الذاكرة أو **Memory Leaks**.

وده ممكن يسبب مشاكل كبيرة جدًا في الأداء واستقرار النظام، وممكن يوصل إنه ينهار خالص.
يعني البداية والنهاية لازم يكونوا محسوبين كويس قوي.

- طب وما بينهم بقى؟
- الدنيا مش دايمًا وردي. الأخطاء واردة جدًا وبتحصل. الكلاس مفروض يتعامل معاها إزاي؟ دي نقطة حرجة جدًا.

الكود الكويس لازم يتوقع إن الأخطاء هتحصل. ما ينفعش أبدًا نتجاهلها أو نفترض إن كل حاجة هتمشي تمام وزي الفل. البرنامج ما ينفعش ينهار فجأة كده في وش المستخدم. ده شيء غير مقبول.

- أكيد.
  الكلاس المصمم صح لازم يكون عنده خطة للتعامل مع الأخطاء دي. يا إمّا يقدر يتعامل مع الخطأ داخليًا ويصلحه ويكمل شغله عادي كأن ما فيش حاجة حصلت، أو لو الخطأ كبير وما يقدرش يكمل، على الأقل لازم يبلّغ عن المشكلة دي بشكل واضح، ممكن عن طريق إنه يرمي **Exception** مثلًا. ولازم يضمن إنه مش هيسبب فقدان بيانات أو يسيب السيستم في حالة غير متناسقة قبل ما يتوقف.

التجاهل التام للمشاكل ده مش اختيار مقبول أبدًا في تصميم كويس.

والكتاب كمان بيجيب سيرة سلوك النسخ والمقارنة (**Copy & Compare**).

- ليه دي بالذات محتاجة اهتمام خاص؟
- ممم، سؤال كويس.

لأن الـ Objects مش زي الأرقام والحروف مجرد بيانات بسيطة. لما بنقول "انسخ الـ Object ده" أو "هل ده بيساوي الـ Object التاني؟"، المعنى مش دايمًا بيبقى واضح وبديهي.

- إزاي يعني؟
  يعني هل لما أقول "انسخ"، أنا قصدي ننسخ بس العنوان في الذاكرة اللي هو الـ **shallow copy**، فالنسختين يبقوا بيشاوروا على نفس البيانات؟ ولا قصدي نعمل نسخة جديدة تمامًا بكل محتوياته الداخلية اللي هي **Deep Copy**؟

ونفس الكلام في المقارنة. هل نعتبر اتنين **Objects** متساويين لو بيشاوروا على نفس المكان بالظبط في الذاكرة؟ ولا لو القيم اللي جواهم هي اللي متساوية بغضّ النظر عن مكانهم؟

- آه فهمتك. اللغة بتدي سلوك افتراضي.

- بالظبط. اللغة بتدي سلوك افتراضي غالبًا بيكون **Shallow Copy** والمقارنة بالعنوان. لكن السلوك ده ممكن ما يكونش هو اللي إحنا عايزينه للكلاس بتاعنا. عشان كده لازم نفكر كويس ونحدد إيه المعنى المقصود للنسخ والمقارنة بالنسبة للكلاس ده بالذات، ونوفر الطريقة الصحيحة عشان نعملهم لو الطريقة الافتراضية مش مناسبة للمنطق بتاعنا.
- تمام أوي.

يبقى المجموعة التانية دي من الإرشادات بتركّز على ضمان إن الكلاس مش بس شكله الخارجي كويس ومنظم، لأ، كمان سلوكه الداخلي قوي وموثوق ويقدر يتعامل مع المشكلات المتوقعة، سواء في البداية أو النهاية أو أثناء التشغيل.

- تمامًا. ال Class لازم يكون وحدة بناء صلبة نقدر نعتمد عليها، مش مجرد شكل حلو من برّه وخلاص.

طيب، كده إحنا بنينا كلاسات قوية وموثوقة كوحدات منفصلة. إزاي بقى نخليها تشتغل كويس مع بعضها وتكون مفيدة على المدى الطويل؟

في مجموعة إرشادات تانية بتتكلم عن التعاون، وإعادة الاستخدام، والتوسع، والصيانة.

- حاسس إن دول كلهم مرتبطين ببعض قوي، مش كده؟
- جدًا. هما كلهم أوجه مختلفة لهدف واحد كبير: إننا نبني نظام يكون مرن وقابل للتطور والاستمرار مع الوقت.

أولًا: التصميم بهدف التعاون. القاعدة بتقول: **"ما فيش كلاس عايش لوحده في جزيرة منعزلة"**. لازم وإحنا بنصمم الكلاس نفكر: هو هيتفاعل إزاي مع الكلاسات التانية؟ إيه الخدمات اللي هو هيقدمها لهم؟ وإيه الخدمات اللي هو ممكن يحتاجها منهم؟ زي مثال سواق التاكسي والمشرف اللي قلناه. العلاقة بينهم تبادلية: ده بيحتاج حاجة من ده.

In the cabbie example, the cabbie and the supervisor are not standalone entities; they interact with each other at various levels (see Figure 5.4).

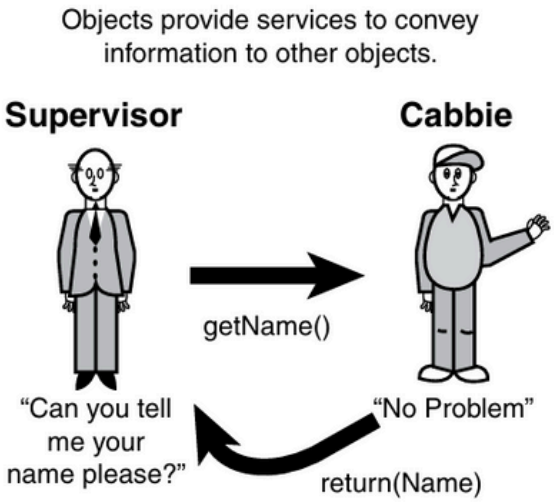


Figure 5.4 Objects should request information.

وده بيقدم حاجة لده. التفكير ده بيخلي تصميمنا واقعي أكتر وجزء من صورة أكبر. وده، بتهيألي، يودينا مباشرة لإعادة الاستخدام أو **Reuse**.

- بالظبط. لو صممنا الكلاس بشكل عام كفاية، يعني مش متخصص قوي في حتة معينة، ومنفصل عن سياق محدد جدًا، ممكن نقدر نعيد استخدامه في أجزاء تانية من نفس السيستم، أو يمكن حتى في مشاريع تانية خالص.
  وده طبعًا بيوفر وقت ومجهود كبير جدًا على المدى الطويل.

لكن مهم نقول إن تحقيق إعادة الاستخدام بشكل كويس دي شغلانة محتاجة مجهود واعي، وتفكير تجريدي **abstract thinking** من البداية. الموضوع مش بيحصل بالصدفة أو لوحده.

طيب، والتوسع أو **Extensibility**، إيه الفرق بينها وبين إعادة الاستخدام؟

- هم قريبين، بس التوسع **Extensibility** بيركّز أكتر على المستقبل بتاع الكلاس نفسه.

يعني: إزاي نخلي الكلاس بتاعنا ده سهل إننا نضيف عليه وظايف جديدة بعدين أو نخصص منه أنواع جديدة من غير ما نضطر نعيد كتابته من الأول.

هنا بقى بييجي دور إننا نستخدم **Inheritance** و ال **Interface** بشكل ذكي واستراتيجي. زي مثال ال**Person**:
نصمم كلاس أساسي **Base Class** زي **Person**، يكون فيه الحاجات العامة المشتركة بين كل الأشخاص.
وبعدين نورّث منه كلاسات تانية متخصصة زي **Employee** (موظف) أو **Customer** (عميل).

النقطة المهمة هنا إننا ما نتسرعش ونحط تفاصيل خاصة جدًا، زي مثلًا وظائف إشرافية، في الكلاس الأساسي **Person**.
لأ. نخلي التفاصيل دي في الكلاسات المتخصصة اللي محتاجاها فعلًا، زي **Manager** مثلًا، اللي هو نوع من **Employee**.

وكل ده بقى بيصب في فكرة الصيانة أو **Maintainability** في الآخر.

- أكيد.

كل المبادئ اللي اتكلمنا فيها دي: التقسيم لوحدات صغيرة ومفهومة اللي هي الكلاسات، الواجهات الواضحة، إخفاء التفاصيل، التصميم المرن القابل للتوسع... كل ده هدفه النهائي إن النظام ككل يكون سهل صيانته وتطويره في المستقبل.

التغيير في تفاصيل التنفيذ جوة كلاس واحد المفروض يكون تغيير سهل وآمن نسبيًا.
لكن التغيير اللي بيخوف وبيكون مكلف جدًا هو التغيير في الواجهات العامة **public interfaces** ، لأنه ممكن يكسر كل الكود اللي بيعتمد على الواجهة دي ويعمل تأثير الـ **Ripple Effect**.

يبقى واضح كده إن التفكير في المستقبل ده جزء أساسي ورئيسي من تصميم الكلاسات الكويس. مش مجرد حل مشكلة النهاردة.
وده بالظبط جوهر الهندسة البرمجية الجيدة: إننا نبني حلول مش بس لمشكلة النهاردة، لكن حلول تكون قادرة إنها تكبر وتتأقلم مع تحديات بكرة وبعده.

ممتاز. كده فاضل مجموعة أخيرة من الإرشادات اللي ممكن نعتبرها نصايح عملية أكتر أو تحذيرات بتكمل الصورة العامة.
زي مثلًا: التوثيق، واستخدام الكلمة المفتاحية **static**، واختيار الأسماء، وحاجات زي كده.

- أيوة. دي ممكن نعتبرها اللمسات النهائية اللي بتفرق كتير قوي في جودة الشغل اليومي، وبتخلي الحياة أسهل للمبرمجين اللي هيتعاملوا مع الكود ده بعدين.

أولًا: التوثيق وكتابة التعليقات بحكمة. التوثيق مهم طبعًا عشان نفهم الكود ونقدر نستخدمه ونصلحه بعدين، خصوصًا لو الكود ده هيستخدمه حد تاني غير اللي كتبه أو هيتم إعادة استخدامه.

بس كلمة "**بحكمة**" هنا مهمة.

> **Too Much Documentation**
>
> Be aware that over-commenting can be a problem as well. Too much documentation and/or commenting can become background noise and may actually defeat the purpose of the documentation. Just like in good class design, make the documentation and comments straightforward and to the point. Well-written code is, in itself, the best documentation.

التعليقات الكتير قوي زيادة عن اللزوم أو اللي بتقعد تشرح حاجات بديهية وواضحة من الكود نفسه دي ممكن تضر أكتر ما تفيد وتعمل زحمة ودوشة.
الكود الواضح اللي مكتوب كويس بأسماء معبرة هو يعتبر أحسن توثيق لنفسه.

والأسماء المعبرة دي لها إرشاد لوحدها مخصوص كمان، لأهميتها القصوى فعلًا. اختيار أسماء واضحة وصريحة للكلاسات بيخلي الكود يتكلم عن نفسه وبيقلل جدًا فرص اللبس وسوء الفهم.
مهم كمان إننا نتبع الأعراف المتعارف عليها في التسمية في اللغة **naming conventions** أو الفريق اللي بنشتغل فيه.

> **Good Naming**
>
> Make sure that a naming convention makes sense. Often, people go overboard and create conventions that might make sense to them but are totally incomprehensible to others. Take care when forcing others to conform to a convention. Make sure that the conventions are sensible and that everyone involved understands the intent behind them. Make variables descriptive of their use, not encoded based on their type.

Making names descriptive is a good development practice that transcends the various development paradigms.

طيب، والتحذير بتاع استخدام **static** بحذر، ليه ممكن تكون مشكلة؟ مش هي مفيدة أحيانًا؟
هي ممكن تكون مفيدة في حالات قليلة، لكن الإفراط في استخدامها هو المشكلة.

الأعضاء الـ **static** سواء كانت متغيرات أو دوال بتكون مشتركة على مستوى الكلاس كله، مش خاصة بأوبجيكت. ده بيعمل نوع من الترابط القوي أو الـ tight coupling بين أجزاء الكود المختلفة، وبيصعّب عملية الاختبار خصوصًا ال unit testing وعمليات الـ mocking، لأنك مش بتتعامل مع object مستقل.

مم... وكمان هي بتتعارض شوية مع فكرة الـ polymorphism المرتبطة بالـ object والـ overriding.
استخدامها بيكون مناسب ومنطقي أكتر في حالات محدودة جدًا، زي مثلًا الوظائف المساعدة **helper functions** البسيطة جدًا اللي ما لهاش أي آثار جانبية على حالة البرنامج، زي عمليات حسابية بحتة في كلاس زي Math مثلًا Math.add.

لكن استخدامها لعمليات لها علاقة بحالة البرنامج أو قواعد البيانات أو الـ I/O، ده غالبًا بيكون علامة على تصميم مش كويس.

وفيه كمان نقطة قريتها عن الحفاظ على نطاق صغير أو **Keep scope small**.
دي تقصد بها زي المتغيرات المحلية **local variables** . بالظبط كده. الفكرة العامة إن أي متغير أو دالة لازم يكون نطاق رؤيته **scope** أصغر ما يمكن، ومتاح بس في المكان اللي هو مطلوب فيه فعلًا.

يعني لو متغير معين أنا بحتاجه بس جوة دالة واحدة، يبقى المفروض متعرّف جوة الدالة دي، مش على مستوى الكلاس كله لو ما فيش أي دالة تانية محتاجاه. ده بيقلل فرص الأخطاء والآثار الجانبية غير المقصودة، وبيخلي الكود أوضح وأسهل في تتبعه.

> **Scope and Global Data**
>
> Minimizing the scope of global variables is a good programming style and is not specific to OO programming. Global variables are allowed in structured development, yet they can get dicey. In fact, there is no global data in OO development. Static attributes and methods are shared among objects of the same class; however, they are not available to objects not of the class. You could also share data via a file or database.

تمام. وفيه نقطتين كمان لفتوا نظري لهم علاقة بالتعامل مع البيئات المختلفة والاستمرارية.

أيوة. النقطة الأولى هي **الـ Abstract Non-Portable Code.**
دي ببساطة بتقول: لو عندك جزء من الكود بيعتمد بشكل مباشر على نظام تشغيل معين أو هاردوير مع كود native، حاول تعزل الكود ده جوة كلاس مخصوص بيعمل زي **wrapper** أو غلاف حواليه، بحيث باقي الكود بتاعك يتعامل مع الـ wrapper ده، مش مع الكود الـ native مباشرة.
ده بيخلي الكود الأساسي بتاعك (portable)، وتقدر تشغله على بيئات مختلفة بمجرد تغيير الـ wrapper.

والنقطة التانية عن الاستمرارية.
آه، الأخذ في الاعتبار استمرارية الكائنات.
يعني لو إحنا محتاجين حالة الأوبجيكت (البيانات اللي جواه) تفضل موجودة ومحتفظين بيها حتى بعد ما البرنامج يقفل ويتفتح تاني، لازم نفكر من الأول إزاي هنحقق ده. هل هنحفظها في ملف عادي عن طريق

عملية زي serialization مثلًا؟
ولا هنحطها في قاعدة بيانات **Relation DB** وساعتها هنحتاج نعمل mapping بين الأوبجيكت والجداول؟
ولا يمكن قاعدة بيانات **NoSQL** تكون أنسب؟
المهم إننا نخطط للحتة دي من بدري لو هي مطلوبة.

وأخيرًا، كان في نقطة عن اختبار الواجهات **Testing the Interface** باستخدام الـ **stubs**.

The minimal implementations of the interface are often called stubs

دي إيه فايدتها؟ دي مهمة جدًا عشان نقدر نبني ونختبر أجزاء السيستم المختلفة بشكل متوازي ومن غير ما نستنى كل حاجة تخلص.

الـ stub هو عبارة عن تطبيق بسيط جدًا أو وهمي لواجهة كلاس معين.
هو بيحقق العقد بتاع الواجهة (interface contract)، يعني في نفس التوقيع (signature)، لكن مش بيعمل الشغل الحقيقي اللي الكلاس الأصلي بيعمله. ممكن يرجع قيم ثابتة أو ما يعملش حاجة خالص.

ده بيسمح لك تختبر الكود اللي بيستخدم الكلاس ده حتى لو الكلاس الحقيقي نفسه لسه ما خلصش كتابته.

يعني بيساعد في التطوير والاختبار بدري؟ بالظبط.
بيسرع عملية التطوير وبيساعد في اكتشاف مشاكل التصميم أو التكامل بين الأجزاء المختلفة بدري.

والكتاب بينصح كمان إننا نحتفظ بالـ stubs دي عشان نستخدمها بعد كده في اختبارات regression testing، للتأكد إن التغييرات اللي بنعملها ما بوّظتش حاجة كانت شغالة قبل كده.

فكرة الاختبار لازم تكون جزء من عملية التصميم من البداية.

تمام أوي.

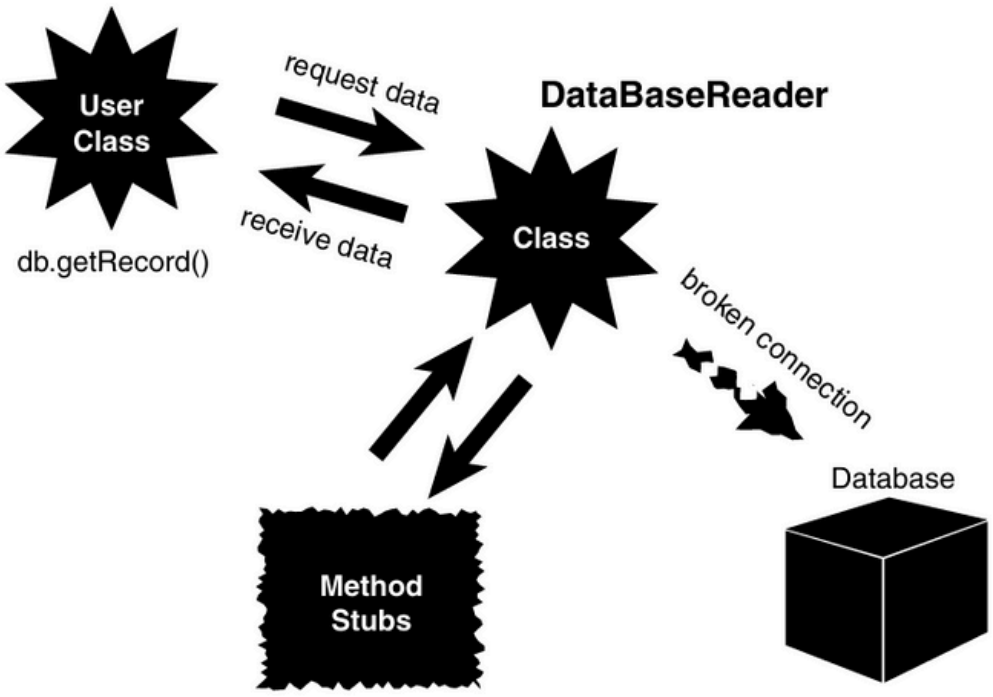


Figure 5.6 Using stubs.

كده أعتقد الصورة اكتملت إلى حد كبير.
الإرشادات كلها لما نحطها جنب بعض بترسم لنا خريطة طريق واضحة عشان ننتقل من مجرد كتابة كود بيستخدم مفاهيم الـ OOP إلى تصميم كلاسات وأنظمة قوية ومرنة ومستدامة فعلًا.

الهدف مش بس إن الكود يشتغل النهاردة، الهدف إنه يفضل شغال كويس ويكون سهل التعامل معه وتطويره بكرة وبعده.

بالظبط كده. هي في النهاية مجموعة أدوات ومبادئ بتساعدنا إننا نتعامل مع التعقيد اللي بيزيد بشكل طبيعي كلما حجم المشروع كبر وكلما متطلباته زادت.
لما بنلتزم بالإرشادات دي قدر الإمكان، بنقدر نبني أنظمة أكبر وأعقد وإحنا عندنا ثقة أكبر في إنها هتكون قابلة للصيانة والتطوير.

وده يرجعنا لهدف المستمع اللي بيتابع معنا وعايز يفهم إزاي ممكن يطبق مفاهيم الـ OOP دي بشكل عملي ومفيد.
الإرشادات اللي استعرضناها النهاردة دي هي الجسر الحقيقي بين النظريات اللي اتعلمناها والتطبيق الفعلي في بناء أنظمة حقيقية بتشتغل.

وهنا يمكن نضيف نقطة أخيرة كده للتفكير، يعني مش موجودة بشكل صريح كإرشاد، بس مهمة في الواقع العملي.
الإرشادات دي كلها بتمثل الحالة المثالية، لكن في شغلنا كمطورين دايمًا بيكون فيه موازنات وتنازلات.
بنبقى محكومين بوقت معين، بميزانية محدودة، ويمكن بنتعامل مع كود قديم مليان مشاكل.

التحدي الحقيقي بقى هو إزاي المطور يقدر يوازن بين السعي لتطبيق المبادئ دي بأفضل شكل ممكن وبين القيود العملية اللي بتقابله كل يوم في شغله.

دي نقطة مهمة جدًا فعلًا.
آه، هنا بقى بتيجي أهمية الخبرة والحكم السليم في اتخاذ القرارات الصعبة دي:
إمتى ألتزم بالمبدأ بحذافيره؟ وإمتى ممكن أعمل استثناء بسيط عشان أمشي الدنيا؟ وإمتى لازم أقف وأقول: لا، ما ينفعش نتنازل هنا؟

دي حرفة بتيجي مع الوقت والتجربة.

شكرًا لمتابعتكم، وإلى لقاء جديد قريبًا سلام.

الفصل السادس Designing with Objects :

المرة اللي فاتت ركزنا أكتر على تصميم الكلاس نفسها،
إزاي نعملها صح ومنظمة، وده كان كلامنا عن الفصل الخامس.

تمام.

النهاردة بقى بناخد الموضوع لمستوى تاني خالص.
هنبص على الصورة الأكبر: تصميم السيستم كله على بعضه.

بالظبط.

إزاي الكلاسات اللي عملناها دي بتتعامل مع بعضها عشان تعمل نظام شغال.
وهندخل كمان في حتة مهمة قوي اسمها **تغليف الكائنات** أو **Object Wrappers**.

آه، دي نقطة مهمة فعلا.

اللي فهمته إنها طريقة نستخدم بيها كود قديم أو مختلف جوة التصميم بتاعنا الجديد.

صح كده؟
صح جدا.

هو الفصل السادس في الكتاب بيركز قوي على النقلة دي:
إزاي نطلع من تفكير تصميم الكلاس الواحد لتصميم النظام ككل،
الصورة الكاملة.

مم.

ودي بصراحة محتاجة طريقة تفكير مختلفة، هنشوف إزاي بقى.

يبقى هدفنا النهاردة إننا نفهم الخطوات دي بتاعت تصميم الأنظمة، ونفهم أكتر حكاية **Object Wrappers**،
وإزاي بتساعدنا.

جاهزين ندخل في التفاصيل؟
يلا بينا.

طيب خلينا نبدأ بالسؤال البديهي يمكن:
ليه تصميم سيستم كامل ده حوار مختلف عن تصميم كلاس واحدة؟
وليه مهم قوي إننا ندي وقت ومجهود للتصميم ده في الأول؟

سؤال مهم جدا بصراحة. هو تخيل معايا كده بنبني بيت.
ينفع نبدأ نبني حيطة في أي حتة كده وخلاص، من غير ما يكون عندنا رسم هندسي واضح للبيت كله، فين الأوض؟ فين الشبابيك؟
لأ طبعا، مستحيل.

أكيد لأ.

نفس الفكرة بالظبط في السوفت وير.
أي قرار تصميم كبير، لو أخدناه متأخر، يعني بعد ما الكود اتكتب **أو الأخطر بقى بعد ما البرنامج نزل السوق والناس بتستخدمه**،
التكلفة عشان نغيره بتبقى رهيبة.

ولما بنقول **تكلفة**، **مش بس فلوس** صح؟
لأ طبعًا، مش بس فلوس. ده وقت بيروح، ومجهود بيضيع، والمشروع بيتأخر، وممكن كمان السمعة بتاعة المنتج أو الشركة تتأثر.

أكيد.

يعني الغلطة على الورق في مرحلة التصميم، تصليحها أسهل بكتير وأرخص بكتير من إننا نهدم حيطَة اتبنت خلاص.
الكتاب بيركز قوي وبيشبه جودة السوفتوير بجودة الكباري والمباني الضخمة: لازم تصميم دقيق واختبارات قوية.

منطقي.

وعشان كده هدفنا من التصميم الكويس بدري هو إننا نقلل **الاعتماديات المعقدة** بين أجزاء السيستم.
اللي هو **coupling** ده — كلما كانت الأجزاء أكثر تفكيكًا واستقلالية، كلما أصبح التغيير أسهل ومش بيخبط في باقي السيستم.

فهمتك: يعني الهدف نخلي المكونات مفككة شوية عشان تكون مرنة.

طيب، الكتاب كان بيتكلم عن طرق مختلفة لعملية التطوير زي ال **waterfall** مثلا . هل ده بيفرق في أهمية التصميم؟
هو بيوضح إن في مدرستين:

- المدرسة الأولى **waterfall** : كل مرحلة تخلص تمامًا قبل ما نبدأ اللي بعدها — تحليل، بعدين تصميم، بعدين برمجة وهكذا — زي نموذج الشلال (Waterfall). دي الطريقة التقليدية.

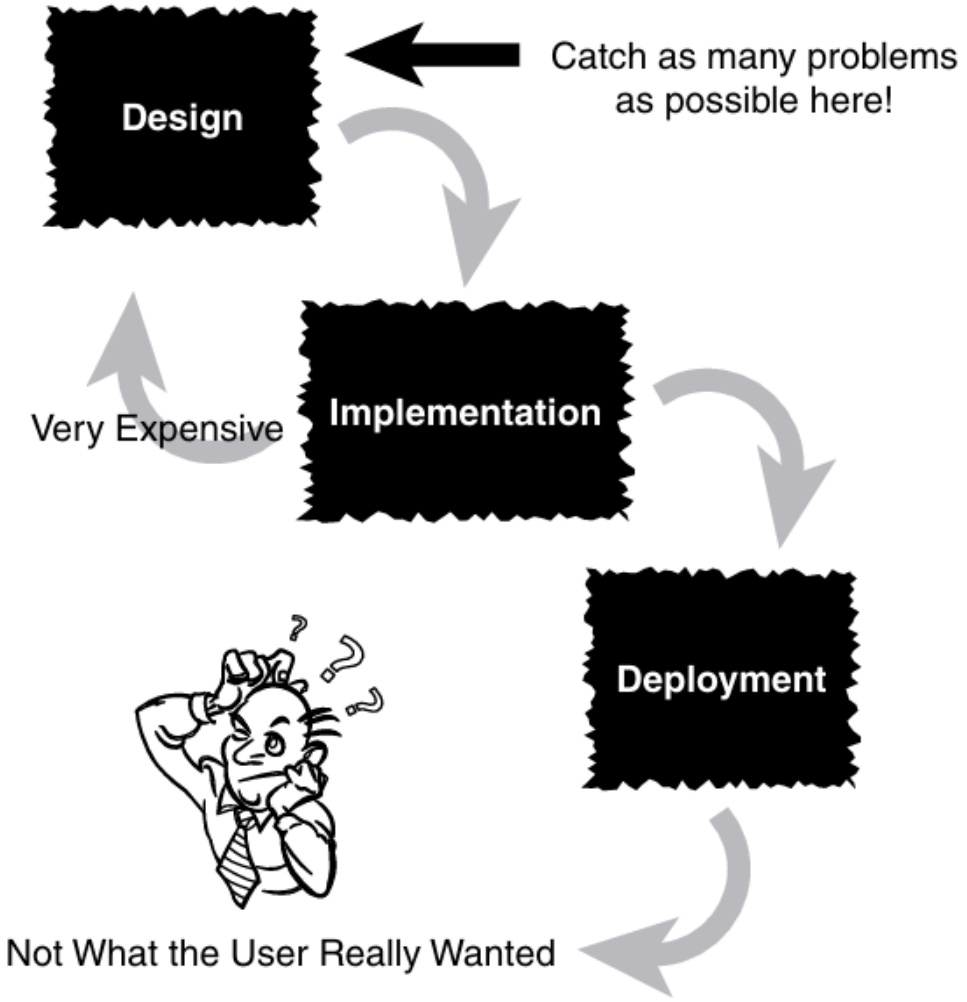
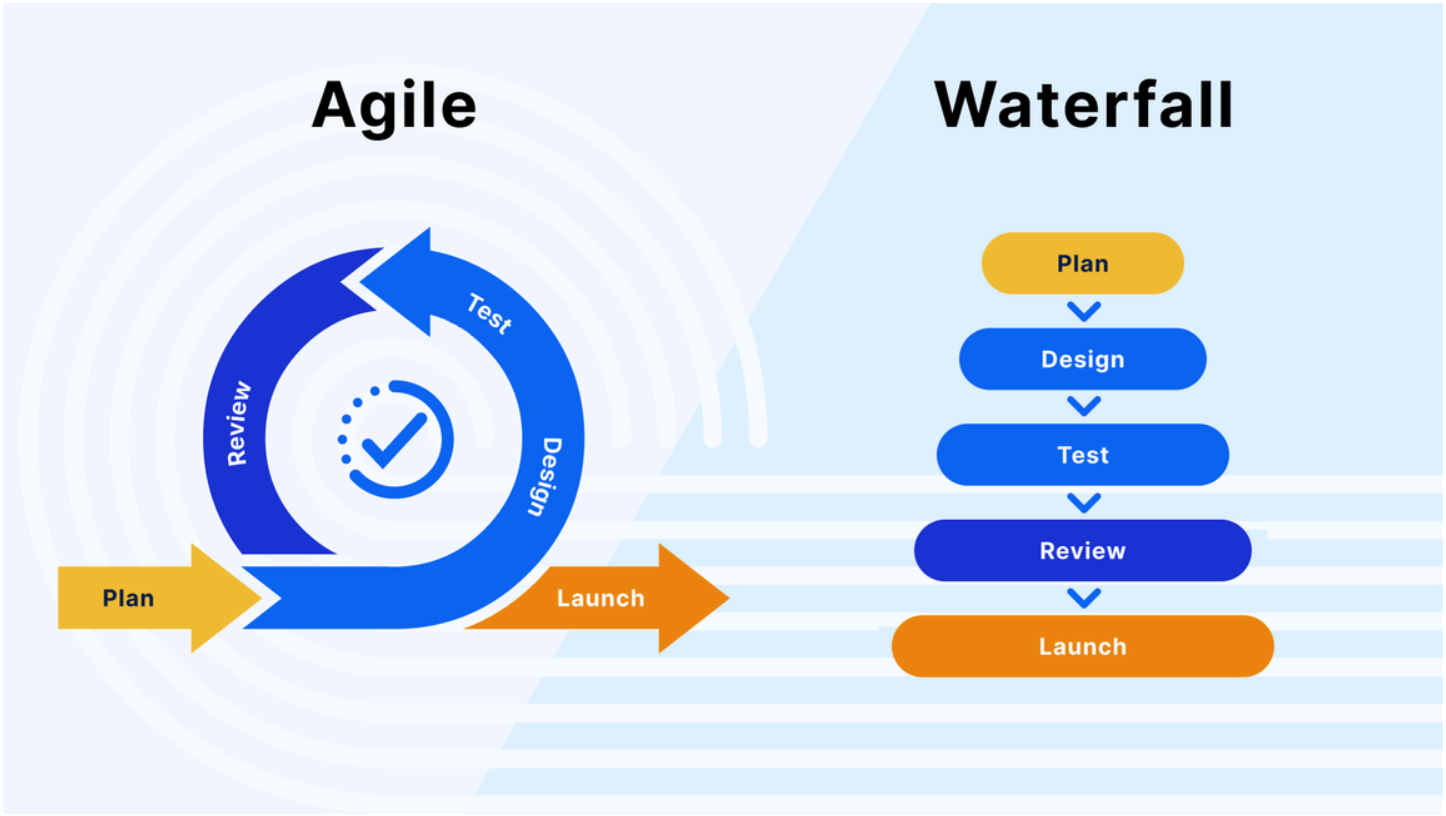


Figure 6.1 The waterfall method.

- والمدرسة التانية: المنتظمة أو التكرارية (Iterative) زي الأجايل (Agile) منتشرة منها دلوقتي. بنقسم الشغل لدورات قصيرة، وكل دورة فيها تحليل وتصميم وبرمجة واختبار لجزء صغير، وبناخد رأي الناس أولًا بأول.

مم.



لكن في الحالتين، فكرة التصميم والصورة الكلية تفضل مهمة جدًا. حتى في الأجايل مش بنبني أي حاجة بدون رؤية عامة؛ بيبقى في رؤية عامة للتصميم وبنعدلها ونحسنها مع كل دورة.
بس بنحاول، قدر الإمكان، نتجنب التغييرات الكبيرة في أساسات السيستم في مرحلة متأخرة.

منطقي جدًا.

طيب، بما إن التصميم ده مهم قوي كده، أكيد فيه خطوات عملية ومنظمة نمشي عليها.
الكتاب بيقترح مسار معين نمشي عليه خطوة بخطوة. العملية غالبًا بتكون تكرارية زي ما قلت — يعني ممكن نرجع خطوة لورا ونعدل لو اكتشفنا حاجة جديدة — بس بشكل عام الخطوات الأساسية هي كالتالي:

Generally, a solid OO design process includes the following steps:

1. Doing the proper analysis
2. Developing a statement of work that describes the system
3. Gathering the requirements from this statement of work
4. Developing a prototype for the user interface
5. Identifying the classes
6. Determining the responsibilities of each class
7. Determining how the various classes interact with each other
8. Creating a high-level model that describes the system to be built

أول حاجة خالص هي **التحليل**: دي مرحلة الفهم. لازم نفهم بالضبط السيستم ده المفروض يعمل إيه؟ إيه مشكلة البيزنس اللي بيحلها؟ مين هيستخدمه؟ إيه العمليات الأساسية؟ بنجمع معلومات، ونسأل، ونحلل أي ورق موجود. المهم نوصل لفهم كامل — يعني بنبني قاعدة معرفية قبل ما نفكر حتى في شكل الكود.

تمام. بعد الفهم ده بنحتاج **نوثقه**. دي مرحلة وثيقة العمل **statement of work that describes the system** — زي اتفاق مبدئي أو وصف عام للسيستم اللي هنعمله. بتحدد الإطار العام. ومن الوثيقة العامة دي بنبدأ ندخل في التفاصيل بقى.

بالظبط.

بنوصل للخطوة اللي ممكن فعلاً تكون أهم حتة في الموضوع كله **Requirements**.
المتطلبات هنا بنترجم الكلام العام إلى قائمة واضحة ومحددة وقابلة للقياس.
كل حاجة المستخدم محتاجها في السيستم لازم تتكتب بالتفصيل.

بس دايمًا بنسمع إن حتة المتطلبات دي بتبقى صعبة أوي.
إيه المشاكل اللي بتحصل عادة؟
دي حقيقة — مشاكلها كتير: ممكن تكون غامضة، أو فيها تناقض، أو مش كاملة أصلًا، أو ساعات المستخدم نفسه ما يكونش عارف هو عايز إيه بالضبط.
مم... بتحصل.

عشان كده الخطوة الرابعة مهمة جدًا: نعمل نموذج أولي **Prototype** خصوصًا لواجهة المستخدم **(UI)**.
مش لازم يبقى برنامج شغال كامل يعني — ممكن بس شوية رسومات أو شاشات تبيّن شكلها والتفاعل معاها، آه زي الـ **Mockups**.
أيوة — ده بيخلي المستخدم يشوف حاجة ملموسة ويتخيل المنتج النهائي، فيدينا رأيه بدري قوي.
وده بيقلل جدًا مشاكل التصميم — كأننا بنعمل بروفة للنظام قبل ما نبنيه فعليًا.

طيب، فهمنا الدنيا وعملنا النموذج، نبدأ بقى نفكر **Object Oriented**.
أيوة، هنا بندخل في صلب الموضوع.

خامس خطوة هي تحديد الفئات المبدئية **(Identifying the classes)**.
في طريقة مشهورة للبداية: بندور على الأسماء اللي ظهرت كتير في المتطلبات.
زي إيه مثلاً؟ زي "موظف"، "فاتورة"، "منتج"، "عميل"، "طلب".
الأسماء دي غالبًا بتشير إلى الكيانات الأساسية في السيستم.
القائمة دي بتبقى بداية كويسة — مش شرط تبقى نهائية من أول مرة.

هل الطريقة دي كافية؟ يعني ممكن نلاقي كلاسات مش مجرد أسماء؟
هي بداية كويسة جدًا، بس طبعًا مش كافية لوحدها — الخبرة وطبيعة السيستم بتفرق.

بعد ما بنعمل قائمة مبدئية بنروح للخطوة **السادسة**: تحديد مسؤوليات كل فئة
Determining the responsibilities of each class.
نسأل لكل فئة: المفروض تكون عارفة معلومات إيه وتقدر تعمل عمليات إيه؟ بنحدد دور كل واحد — يعني بنوزع الشغل مضبوط.

وبعد ما نعرف كل هيعمل إيه لوحده، لازم نشوف هيتعاملوا مع بعض إزاي — ودي الخطوة **السابعة**:
تحديد التعاون بين الفئات **Determining how the various classes interact with each other**.
مين بيكلم مين عشان ياخد معلومة أو يطلب خدمة؟ مين بيبعت رسالة لمين؟ هنا بنفكر في العلاقات بينهم وبنحاول نخليها منظمة ومش معقدة قوي.

نرجع تاني لفكرة تقليل الـ coupling — بنرسم خريطة الاتصالات جوه السيستم.

وهي دي بالضبط **الخطوة الأخيرة** في التصميم المبدئي: إنشاء نموذج للنظام
Creating a high-level model that describes the system to be built.
بنستخدم أدوات عشان نعرض التصميم ده بشكل مرئي. أشهر حاجة هي **UML**، خصوصًا الرسومات اللي فيها مربعات وأسهم.
أيوة، هي دي — بتوضّح الـ classes والـ attributes والـ methods والعلاقات اللي بينهم.

النموذج ده بيبقى هو الخريطة اللي الفريق كله بيشتغل عليها. طبعًا في أدوات تانية غير UML بس هي الأشهر.

واضح إن العملية دي مش خطّ مستقيم وخلاص — فيها رايح جاي كتير، والأهم إننا فاهمين كويس وبنتكلم مع المستخدم قبل ما نكتب أي كود.
ده بيعمل أساس قوي للشغل اللي بعد كده.

المسار العام للتصميم بقى واضح دلوقتي أكتر.
بس الفصل ده كمان بيركز على تكنيك معين: **التغليف Object Wrappers**، والحتة دي فعلًا شدتني.

إيه حكايتها بالظبط؟ وهل هي فعلًا زي ما تخيلت كده طريقة تخلينا نستعمل كود قديم أو نربط حاجات شكلها مش راكبة على بعض؟

توصيفك ده قريب جدًا.

ممكن نعتبره زي "محول"، أو الفكرة بسيطة قوي: بنعمل **Object** جديدة شغلتها كلها إنها "تغلف" أو تحتوي جواها حتة كود تانية.

والحتة دي ممكن تكون إيه؟ أي حاجة تقريبًا:
ممكن تكون كود قديم مكتوب بالـ structured code، أو كود بيتعامل مع الـ hardware أو نظام التشغيل نفسه (native)، أو حتى ممكن يكون **Object** تاني أصلًا موجود جاهز بس إحنا عايزين نغير طريقة التعامل معاه.

يعني الـ wrapper ده بيعمل زي طبقة في النص كده بتخبي التعقيد أو الاختلاف بتاع الكود الأصلي، وبتدينا واجهة Interface أسهل ومتوافقة مع باقي السيستم بتاعنا.
بالظبط كده. هو ده ملخص الموضوع.

و الـ wrapper دي لها استخدامات عملية مهمة جدًا.
الكتاب بيتكلم عن تلاتة أساسيين:

أول استخدام هو تغليف الكود **Wrapping Structured Code.**

Consider Figure 6.3. As you can see, an object contains methods, and these methods contain code, which can be anything from variable declarations to conditions to loops.

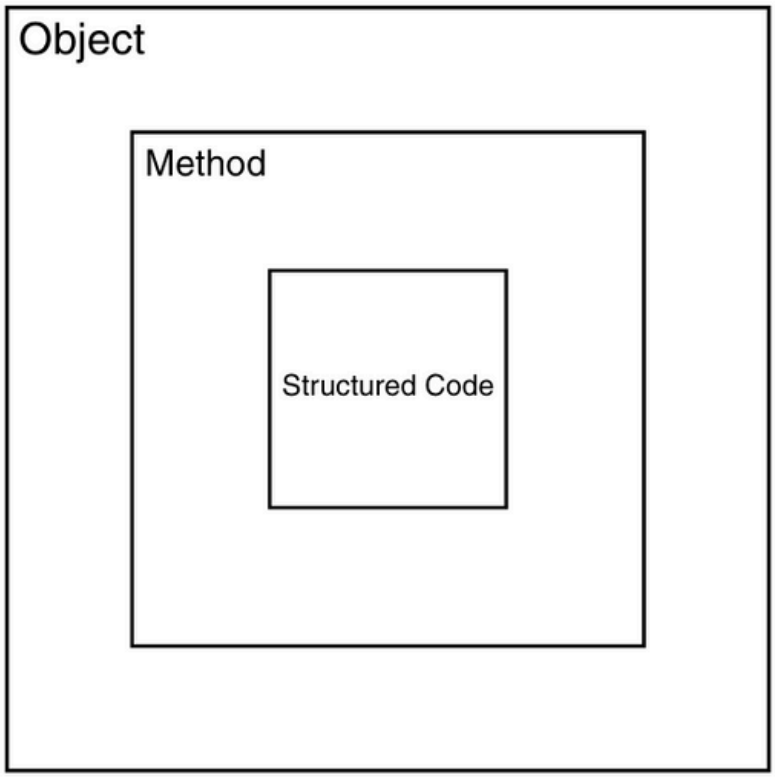


Figure 6.3 Wrapping structured code.

مهم قوي نفتكر إن الـ OOP مش ضد **Structured programming** يعني ال method اللي جوة الـ wrapper نفسها كتير بتستخدم loops , conditions, functions وده عادي.

زي المثال البسيط

```
class SomeMath {

    public int add(int a, int b) {

        return a + b;

    }

}
```

— هو يعتبر wrapper بسيط.
جواه method اسمها add، والكود بتاع الجمع ده Structured .
لو عندنا مكتبة أكواد قديمة شغالة كويس ومستقرة، بدل ما نعيد كتابتها من الأول بـ OOP، ممكن نعمل **Object Wrapper** يغلف استدعاء الدوال دي.
الـ wrapper يقدم لنا **Interface** بسيط وموحد في الكلاس الجديدة، وهو من جواه بينادي على الكود القديم.

وده بيحقق مبدأ إعادة الاستخدام (**reuse**).
بالظبط — وبنستفيد من شغل اتعمل قبل كده. مش لازم كل حاجة نبنيها من الصفر.
الذكاء في إننا ندمج الموجود صح.
ودي نقطة قوية جدًا بتوفر وقت ومجهود ومخاطر كمان.

طيب، إيه الاستخدام التاني؟
الاستخدام التاني هو **Wrapping Nonportable Code — native code**.

ساعات بنبقى محتاجين نتعامل مع حاجة خاصة بنظام تشغيل معين أو حتة hardware معينة.
الكود ده بيبقى native، مرتبط بالبيئة دي، ومش هيشتغل زي ما هو في حتة تانية.
ولو استخدمناه مباشرة جوة الكلاسات بتاعتنا، هنربطها بالبيئة دي ونفقد ميزة الـ portability.
صح كده.

الحل هنا إننا نعزل الكود الـ native ده جوة **Wrapper Class** مخصص له.

الكتاب بيشرح مثال بكلاس اسمه Sound، شغلته يطلع صوت beep.

```
class Sound {

    public void beep() {
        System.out.println("\007");
    }

}
```

Now, rather than having to know the code for making the sound, the programmer can use the class and call the beep method:

```
public class TestBeep {

    public static void main(String[] args) {
        Sound mySound = new Sound();
        mySound.beep();
    }

}
```

في Windows، ممكن الكود اللي بيعمل كده يبقى مجرد طباعة كاركتر معين زي \007.

كلاس Sound بيغلف الكود ده جوة method اسمها beep.
أي حد في السيستم عايز يعمل beep بيستخدم **Sound Object** وينادي ()mySound.beep
هو مش محتاج يعرف إيه اللي بيحصل جوة.

والميزة الكبيرة بقى: لو حبينا نشغل البرنامج ده على نظام تاني زي Linux مثلًا؟
بالظبط — لو رحنا Linux، الكاركتر \007 ده ممكن ما يعملش حاجة. نعمل إيه؟
نروح نعدل بس الكود اللي جوة كلاس Sound عشان يستخدم الطريقة الصح في Linux.

لكن الـ interface اللي هو method beep هيفضل زي ما هو.
باقي السيستم مش هيحس بأي تغيير.
ال **Wrapper** حمى السيستم كله، وزود قابليته للنقل.

بس هل الزيادة دي بتاعة الـ wrapper ممكن تبطأ الدنيا شوية وتعمل مشاكل أداء؟
سؤال كويس. أي طبقة زيادة ممكن تضيف **overhead** بسيط جدًا في الأداء، بس في معظم الحالات بيكون قليل قوي ومش محسوس،
بالذات لما نقارنه بالفايدة الكبيرة في الصيانة، والمرونة، والـ portability.

لكن طبعًا لو العملية دي حساسة جدًا للأداء وبتتكرر كتير جدًا، ممكن ساعتها نقيم الموقف ونشوف حل تاني لو لازم.
هي دايمًا موازنات.

تمام.

يبقى **تالت استخدام** بيركز عليه الكتاب هو **Wrapping Existing Classes** :

الاستخدام التالت هو تغليف فئات موجودة أصلًا. وده سيناريو بنقابله كتير جدًا.
ممكن نستخدم مكتبة جاهزة أو **Class** عامله فريق تاني في الشركة.
ال class ده شغال كويس، بس طريقة استخدامه — الـ interface بتاعه يعني — مش عاجبنا، أو مش ماشي مع الـ standards بتاعتنا، أو عايزين نضيف عليه شوية إضافات بسيطة ومش قادرين نعدل في الكود الأصلي.
يبقى ال **Wrapper** هو الحل تاني؟
بالظبط.

بنعمل**Wrapper Class جديدة** تلف حوالين الكلاس الأصلي ده.
بيبقى جواها object من الكلاس الأصلي.
ولما بننادي method في الـ wrapper، هو بقى بينادي الـ method الصح في الكلاس الأصلي.
ممكن كمان يعمل شوية شغل قبلها أو بعدها.
المهم إنه بيقدم للعالم الخارجي الـ interface الجديد أو الأحسن اللي إحنا عايزينه.

آه، زي المثال بتاع MailTool اللي في الكتاب؟
أيوة.

The following class provides a very simple example of a mail client for this example:

```
package MailTool;
public class MailTool {
   public MailTool () {
   }
   public int retrieveMail() {

    System.out.println ("You've Got Mail");

         return 0;
   }
}
```

When you invoke the `retrieveMail()` method, your mail is presented with the very original greeting "You've Got Mail." Now let's suppose you want to change the interface in all your company's clients from `retrieveMail()` to `getMail()`. You can create an interface to enforce this:

```
package MailTool;
interface MailInterface {
   int getMail();
}
```

لو الأداة الأصلية فيها method اسمها retrieveMail، وإحنا عايزين نخلي كله يستخدم getMail.
نعمل **Wrapper Class** فيها method getMail، وجواها بتنادي على retrieveMail الأصلية.
المبرمجين يتعاملوا مع الـ wrapper الموحد، وهو في الخلفية بيشغل القديمة.

فهمتك.
كأننا بنعمل واجهة تجميلية أو موحدة لحاجة موجودة.
زي ما أعمل غلاف جديد لكتاب قديم يمشي مع شكل المكتبة.

أو أعمل ريموت واحد يشغل كذا جهاز، بيترجم أوامره لكل جهاز بلغته.
فعلًا **Wrapper** دي بتبسط الدنيا جدًا.

طيب، بعد ما شفنا الاستخدامات المفيدة دي كلها، ممكن نربط ده كله ببعضه ونلخص:
ليه بقى ال **object Wrapper** دي بتعتبر أساس مهم جدًا في تصميم الأنظمة الكبيرة والقوية ؟
أكيد الـ wrappers دي بتخدم بشكل مباشر الأهداف الكبيرة اللي قلناها في الأول:

1. **أولًا: بتسهل الصيانة**.
   لما نغلف كود معقد أو بيتغير كتير جوة wrapper، أي تعديل أو تصليح بعد كده بيبقى في حتة واحدة بس جوة، مش بيأثر على باقي الأماكن اللي بتستخدمه.
   طالما الـ interface بتاعه ثابت، ده بيقلل تكلفة الصيانة جدًا، وبيقلل إن الغلطات "تسمّع" في حتت تانية.
   دي ميزة كبيرة جدًا.
2. **تانيًا: بتزود المرونة (flexibility).**
   الطبقة العازلة بتخلينا نقدر نبدل الحتة اللي جوة — الكود القديم، أو الـ native، أو الكلاس الجاهز — بحاجة أحدث أو مناسبة لبيئة تانية، من غير ما نغير أي حاجة برة.
   بتدينا حرية في التطوير.
3. **تالتًا: بتقلل التعقيد.**
   بتداري التفاصيل الرخمة اللي جوة دي ورا واجهة سهلة.
   اللي بيستخدم الـ wrapper بيركز على "هو بيعمل إيه؟" مش "بيعمله إزاي".
4. **رابعًا: إعادة الاستخدام (reuse).**
   بنقدر نستفيد من شغل جاهز وثبت إنه كويس حتى لو مش OOP أصلًا.

5. **وخامسًا بتقلل الاعتمادية**.
   لما الكلاسات تعتمد على الـ Interface بدل ما تعتمد على التفاصيل اللي جواه، بنقلل اللخبطة والاعتمادية الزيادة بين مكونات السيستم.
   ده بيخلي السيستم كله أسهل في الفهم والتعديل.

وكل النقط دي بتصب في فكرة أساسية في OOP نفسها، مش كده؟
بالظبط، ال **Wrapper** هي تطبيق عملي لفكرة الـ Encapsulation أو التغليف.
زي ما Class بتغلف الـ Data وبتخبي تفاصيلها، ال **Wrapper** بيعمل نفس الموضوع، بس على مستوى أكبر شوية.
ممكن يغلف Class بحاله،
أو كود Native جوه Class جديد بواجهة محددة.

ده معناه إن المطور الكويس مش بس اللي بيعرف يعمل Class من الصفر،
لأ، ده كمان اللي بيعرف إزاي يستخدم الحاجات الموجودة بذكاء
ويدمجها في السيستم بتاعه صح باستخدام أدوات زي دي.

فعلا هايل جدًا.
أعتقد كده قدرنا ناخد فكرة واضحة وشاملة عن محتوى الفصل السادس.

لو حبينا نلخص أهم الأفكار اللي اتكلمنا فيها النهاردة:

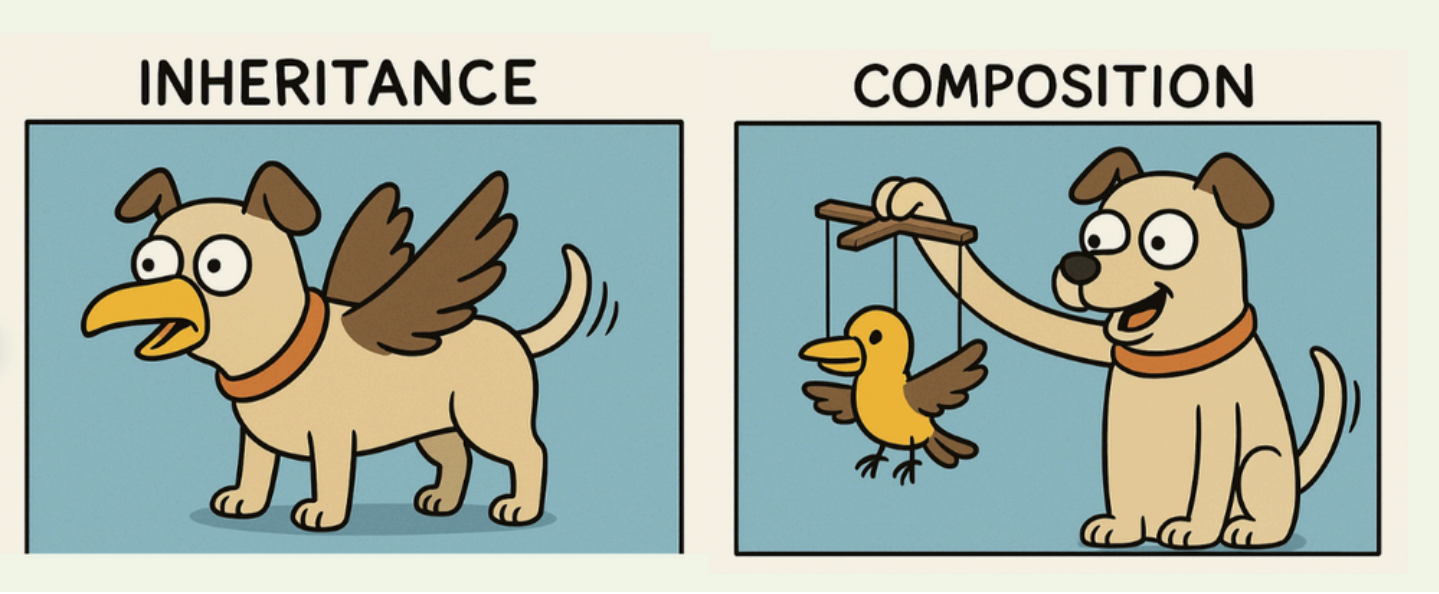
- أول حاجة: النقلة من تصميم الكلاس لتصميم السيستم محتاجة نبص على الصورة الكبيرة والعلاقات.
- تانيًا: إن يبقى في عملية تصميم واضحة ومنظمة حتى لو فيها تكرار، ده استثمار مهم جدًا.
- تالتًا: الـ Object Wrappers دي أداة فعّالة قوي عشان ندير التعقيد ونعيد استخدام الأكواد ونزود مرونة السيستم وصيانته عن طريق تغليف أنواع مختلفة من الكود أو الكلاسات.

فعلا دي أهم النقط.
تصميم الأنظمة ده في الآخر هو فن إدارة العلاقات والاعتماديات بين الأجزاء المختلفة دي.
والـ Wrappers بتدينا طريقة منظمة عشان ندير جزء كبير من العلاقات دي،
بالذات مع الحاجات اللي برة السيستم أو الحاجات القديمة أو اللي مش متوافقة.
هي بتطبق مبادئ زي الـ Encapsulation بشكل عملي ومفيد جدًا.

جميل.
وكالعادة، بنحب نختم بسؤال كده يخلينا نفكر أكتر.
أكيد.
بما إننا شفنا قد إيه فكرة التغليف أو الواجهة البديلة دي مفيدة في الـ Systems،
طب خلينا نفكر كده: فين تاني ممكن نشوف نفس الفكرة دي في هندسة البرمجيات؟
هل في أدوات تانية بتلعب على نفس الوتر ده؟
أو حتى لو طلعنا برة البرمجة خالص، في حياتنا فين ممكن نشوف أمثلة لحاجة بتشتغل كـ Wrapper أو عشان تخلي حاجتين مش ماشيين مع بعض يشتغلوا سوا،
أو عشان تبسط لنا التعامل مع حاجة معقدة؟

بس كده سلام.

الفصل السابع Mastering Inheritance and Composition:



دايمًا بنسمع عن الوراثة كأنها أول حاجة لازم نتعلمها عشان ننظّم الكود.
بس هل هي دايمًا الحل الأمثل؟ النهاردة في الفصل السابع من كتاب **Thought Process**.
هنركز قوي على مفهومين أساسيين:
Composition g Inheritance.

دول الطريقتين الكبار اللي بنستخدمهم عشان نبني علاقات بين الكائنات، والأهم عشان نعيد استخدام شغلنا.
وطبعًا ده بيبني على كلام قلناه قبل كده.

مهمتنا النهاردة نفهم الفرق بينهم كويس خصوصًا للمبتدئين، وهنعرف ليه دايمًا في نقاش وجدل: **يعني إمتى نستخدم دي وإمتى نستخدم دي؟**
يلا بينا نفصّص الحكاية.

نبدأ من الأول خالص:
ليه أصلًا محتاجين ننظّم الكود ونعيد استخدامه؟
الوراثة **Inheritance** والتكوين **Composition** هم دول الحلول ؟

نبدأ بالوراثة **Inheritance**
الوراثة أو **Inheritance** هي ببساطة علاقة **is-a**.
يعني بنقول: **"ده نوع من ده".**
زي المثال المشهور بتاع الكتاب: الكلب هو نوع من الثدييات A dog **(is a)** type of mammal..

هنا الكلب (subclass) بيورّث صفات وسلوكيات من الثدييات (superclass).
آه يعني Class الابن بياخد كل حاجة من Class الأب ويضيف عليها أو يغيّر فيها شوية.
بالظبط كده، بيورّث الخصائص attributes والسلوكيات methods، وده بيوفّر علينا كتابة كود متكرر كتير قوي.

تمام، منطقي. طب والتكوين **Composition** ؟ إيه فكرته؟
التكوين **Composition** بقى مختلف شوية.
ده بنفكّر فيه كعلاقة **has-a**.
يعني **"ده جواه ده" أو "ده عنده ده".**
المثال بتاع الكتاب برضه: العربية فيها موتور.
مش بنقول العربية نوع من الموتور، لأ.
آه، العربية بتحتوي على موتور كجزء منها The car **has a** motor.

بالظبط.
التكوين هو إننا نبني كائن معقد زي العربية عن طريق تجميع كائنات أصغر وأبسط زي الموتور، الكراسي، والإطارات.
الكائنات الصغيرة دي ممكن تكون مستقلة بنفسها.

الكتاب سمّى الكائنات الكبيرة دي ساعات "كائنات مركبة" **Aggregate Objects**.
أوكي تمام.
يبقى: الوراثة **is-a** والتكوين **has-a**.
سهلة كده.
بس الوراثة شكلها أقوى لإعادة الاستخدام.

الوراثة بتشتغل إزاي بالظبط في الكود؟
هي قائمة على فكرتين أساسيتين: **التعميم** و**التخصيص Generalization and specialization**.

يعني لما نلاقي كذا Class فيهم حاجات مشتركة بنعمل **تعميم**.
بنطلع الحاجات المشتركة دي في Class أب (superclass).

زي مثلًا لو عندي كلب وقطة والاتنين بياكلوا وليهم لون عين.
أعمل Class "حيوان أليف" فوقيهم؟
بالظبط.

Class حيوان أليف ده هيبقى فيه الصفات والسلوكيات المشتركة، وبعدين الكلب والقطة يورّثوا منه ويزودوا بقى صفاتهم الخاصة.
ده اسمه **تخصيص**.

الكتاب بيدي مثال أعمق شوية:
آه، ممكن نبدأ بـ Class كلب وبعدين نلاقي إن في أنواع كلاب بتتصرف بشكل مختلف.
يعني كلب بينبح كتير BarkingDog وكلب بيعمل صوت غريب كده YodelingDog.
ساعتها نعمل Class لوحده يورث من كلب؟
أيوة، ده تخصيص أكتر.

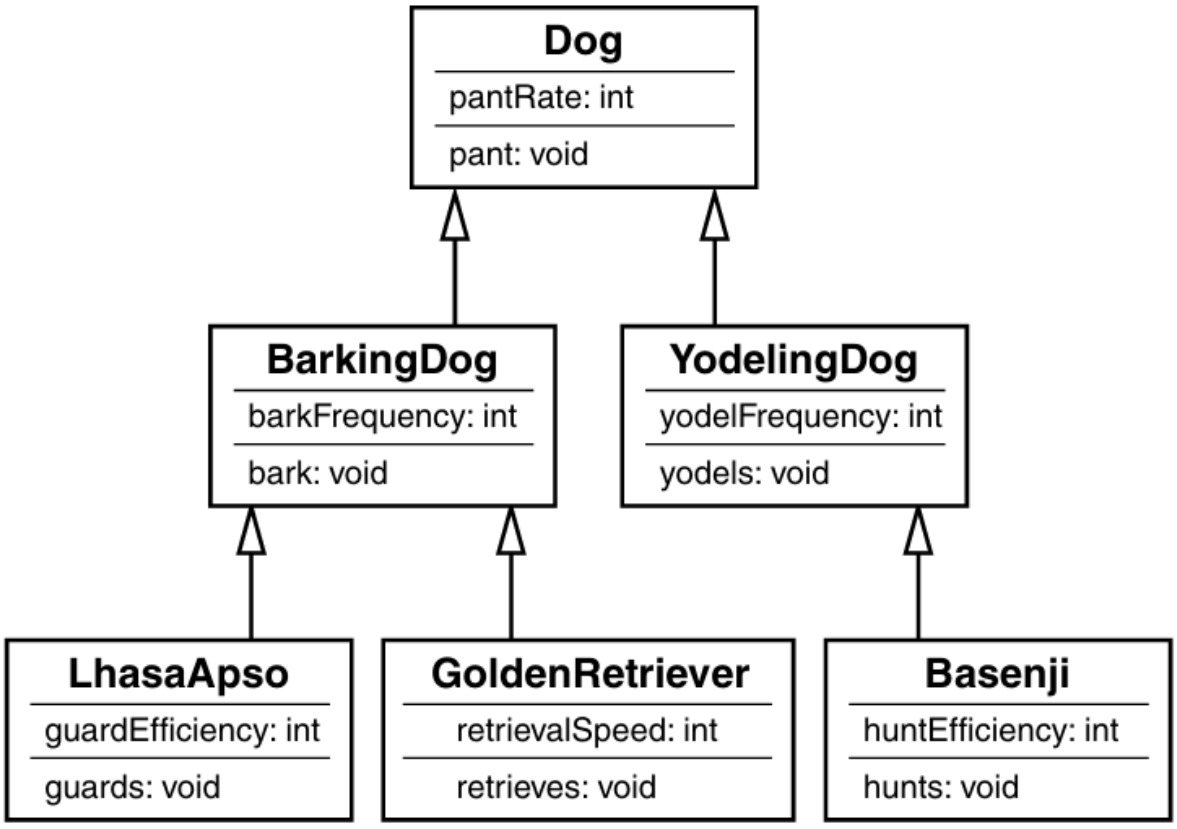


Figure 7.4 The Dog class hierarchy.

ولو حبينا نعمم كمان، ممكن نطلع صفات مشتركة بين كل دول وحيوانات تانية في Class أعلى خالص زي الثدييات (Mammal).

بس لحظة...
حتة التخصيص الزيادة دي الكتاب بيقول إنها ممكن تعمل مشكلة.
يعني لو فضلنا نعمل Class لكل تفصيلة، الدنيا هتعقد قوي.

عندك حق تمامًا.
هي دي بقى الموازنة الصعبة.
كل ما نحاول نخلي النموذج بتاعنا دقيق أوي زي الواقع، كل ما بنزود Classes وعلاقات وراثة، وده بيخلي النظام كله أعقد وأصعب في الفهم والصيانة.

لازم دايمًا نسأل نفسنا: هل التعقيد ده ضروري؟
يعني لحد فين نقف؟
هل منطقي أضيف الضباع والثعالب في نفس شجرة الوراثة بتاعة الكلاب عشان هم قرايب؟
بالظبط.
السؤال ده مهم جدًا.
الإجابة بتعتمد على إحنا بنعمل البرنامج ده ليه أصلًا؟
هل التفاصيل دي لها لازمة فعلًا ولا لأ؟

بس قبل ما نسيب الوراثة، فيه فايدة تانية مهمة جدًا لها وهي **تعدد الأشكال (Polymorphism)**.
آه، دي اتكلمنا عنها قبل كده.
اللي هو نفس الأمر يتنفذ بطرق مختلفة.
تمام.

الكتاب هنا بيستخدم مثال حلو قوي بتاع الأشكال الهندسية (Shape).
تخيّل عندك Class أب اسمه Shape وكلاسات ابن زي الدائرة (Circle) والمستطيل (Rectangle).

لو طلبت من أي شكل فيهم إنه يرسم نفسه (draw) أو يحسب مساحته (getArea)، كل واحد هيعملها بطريقته.
الدائرة هترسم دائرة، والمستطيل هيرسم مستطيل.

ده بيحصل إزاي؟
لازم كل Class يعرف يرسم نفسه ويحسِب مساحته.
لكن الـ Shape نفسه ما يعرفش يعمل كده، لأنه مش شكل محدد.
آه، هو بيحط القاعدة بس.
بالظبط.
بيعرف الطرق دي كـ abstract methods.

طرق مجردة (Abstract Methods) لها اسم بس من غير تنفيذ.
الكلاسات الابن الحقيقية بقى زي الدائرة (Circle) والمستطيل (Rectangle) هي اللي بتقدّم التنفيذ الفعلي للطرق دي.

فهمت، يبقى الوراثة بتدينا إعادة استخدام وكمان مرونة مع الـ Polymorphism والكلاسات الـ Abstract.

طب نخش بقى على التكوين **Composition**.
فكرة بناء الحاجات الكبيرة من أجزاء صغيرة، زي العربية كده.
تمام قوي.
مثال العربية ده ممتاز.

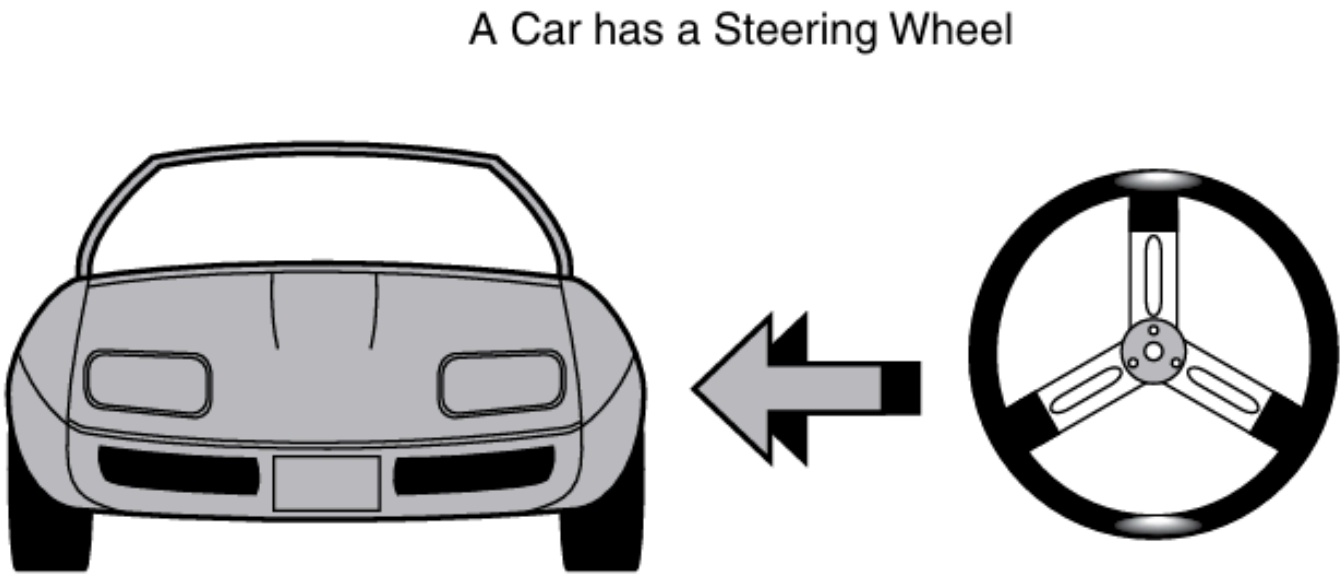


Figure 7.6 An example of composition.

العربية كلها على بعضها بتتكون من أجزاء: موتور، فتيس، أبواب، كراسي، كاسيت.
الأجزاء دي نفسها ممكن تكون مركبة جواها أجزاء أصغر.
يعني الموتور جواه بساتم، والكاسيت جواه راديو وسماعات.

دي علاقة **has a** واضحة:
العربية فيها موتور، والموتور فيه بساتم.
بالظبط.

الكتاب بيقول إن في الـ UML اللي هي لغة الرسم بتاعة التصميم، بنرسم خط بين الكائن الكل والجزء، ونحط شكل معين (اللي شبه المربع بس زعلان شوية فا جاي على جنب) كده عند ناحية الكل عشان نقول إن ده بيتكوّن من دول.

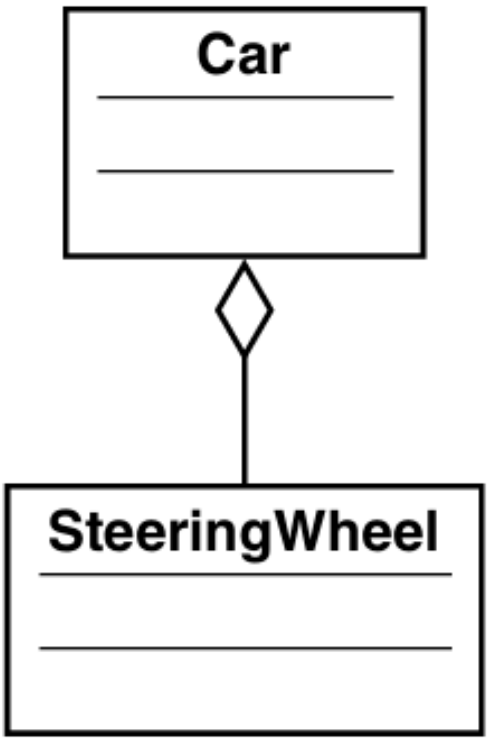


Figure 7.7 Representing composition in UML.

الفكرة الأساسية هنا هي بناء أنظمة معقدة ومرنة عن طريق تجميع وحدات أو مكونات أصغر.
والميزة إن المكونات دي ممكن نستخدمها في حتت تانية أو نبدلها.

ممكن نبدل الموتور، نغير الكاسيت.
ده بيدي مرونة كبيرة زي فكرة الأجزاء المعيارية في المصانع.
كأننا بنلعب ليجو بالكود: قطع مختلفة نعمل بيها أشكال مختلفة.

Figure 7.8 shows the object model for the car, with the engine, stereo system, and door included.

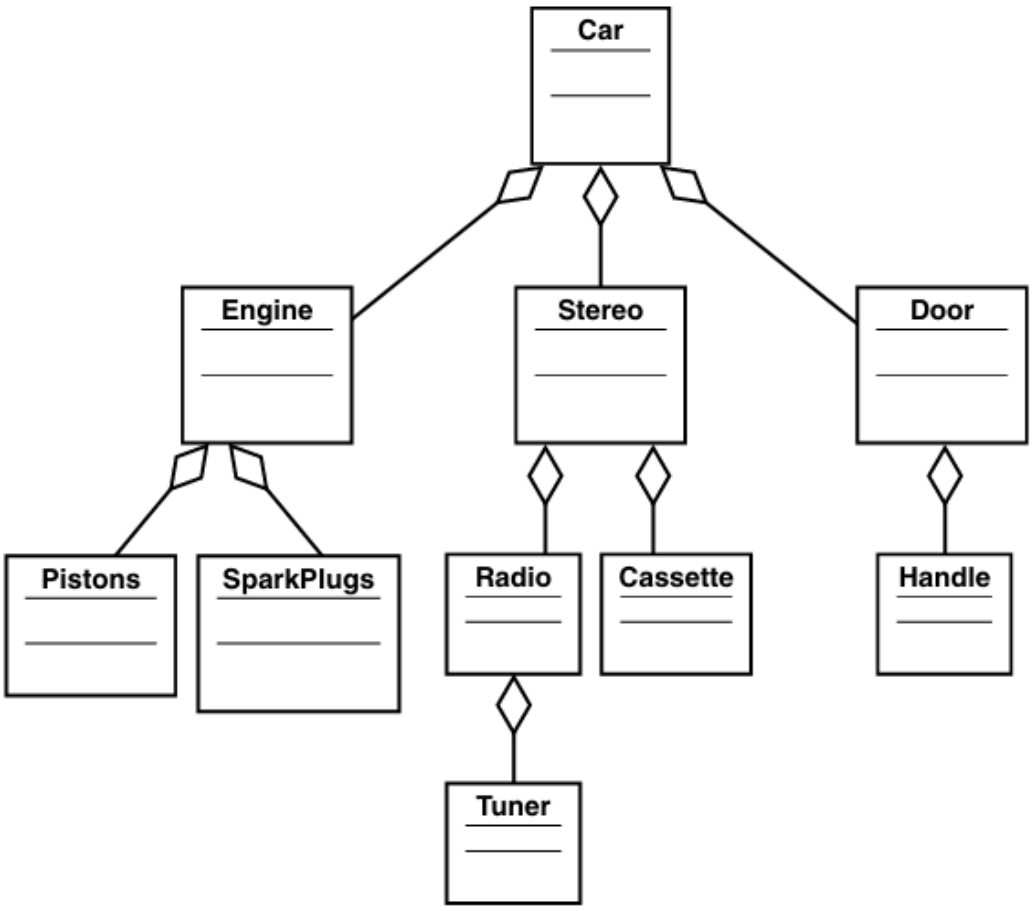


Figure 7.8 The `Car` class hierarchy.

الكتاب كمان بيدخل في تفاصيل صغيرة: الفرق بين **التجميع (Aggregation)** و**الارتباط (Association)**.
الفرق بينهم دقيق شوية، بس باختصار:
التجميع **Aggregation** بيكون ارتباط أقوى، الجزء مالوش معنى قوي من غير الكل، زي التلفزيون والشاشة.

والارتباط **Association** أضعف؟
أيوة، الأجزاء ممكن تكون مستقلة أكتر.
زي الكمبيوتر المكتبي والشاشة والكيبورد.
ممكن أغير الشاشة بسهولة.

بس على أرض الواقع، الأنظمة بتخلط بين ده وده وعلاقات تانية.
يعني العربية فيها موتور (تجميع قوي)، وفيها حامل أكواب (ارتباط أضعف).

المهم هو مبدأ **has-a**.
تمام، كده الصورة أوضح.

ودلوقتي وصلنا للنقطة المهمة أوي.
عنوان الفصل: **إتقان الوراثة والتكوين**.

والكتاب واضح إنه بيقول فيه جدل كبير بينهم.

زمان، الوراثة كانت هي أساس الـ OOP.
إيه اللي اتغير؟
هنا بقى الموضوع بيبقى شيق فعلًا.

اللي حصل إنه مع التجربة العملية في أنظمة كبيرة بدأت تبان مشاكل الوراثة.
المشكلة الكبيرة بصراحة إن الوراثة، خصوصًا **وراثة التنفيذ Implementation Inheritance** ، اللي هي لما الابن ياخد الكود نفسه بتاع الأب.
أيوة.
دي ممكن تكسر مبدأ مهم قوي اسمه **التغليف (Encapsulation)**.

آه، مبدأ إن كل Class يخبي تفاصيله الداخلية.
بالظبط.
الوراثة بتعمل رابطة قوية جدًا **tight coupling** بين الأب والابن.
أي تغيير بسيط في طريقة شغل الأب من جوة، حتى لو تغيير داخلي المفروض ما حدش يعرفه، بيسمع في الابن على طول وممكن يبوظه؟
تمامًا كده، ممكن يكسر الابن بطرق مش متوقعة خالص.

الكتاب بيدي مثال: سواق التاكسي "Cabbie" وسواق التاكسي اللي بيشتغل دوام جزئي "Part-time Cabbie" اللي بيورّث منه.

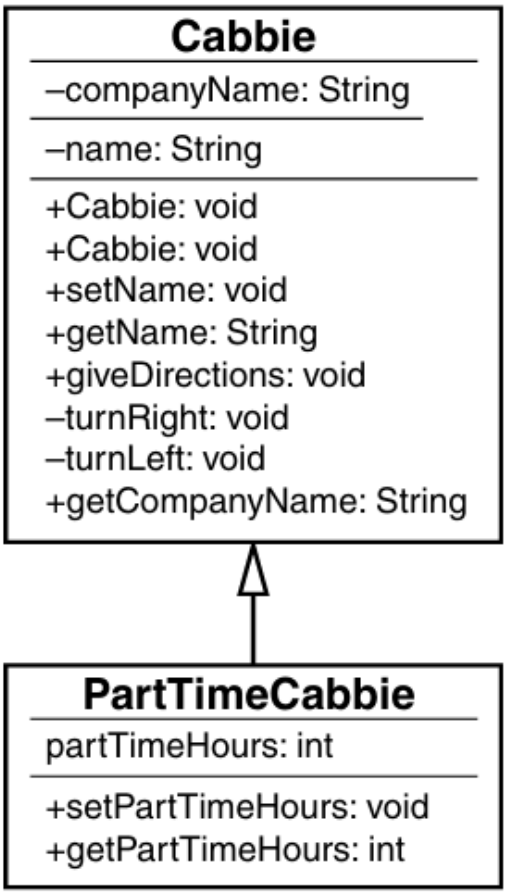


Figure 7.10 A UML diagram of the Cabbie/PartTimeCabbie classes.

لو غيّرت طريقة حساب الأجرة في Class الأب، ممكن ده يبوظ منطق الشغل بتاع الابن اللي هو بتاع الدوام الجزئي.

ايعقل لا لا مش ممكن متهزرش يا أبو السيد! يعني التغييرات بتنتشر زي العدوى كده في شجرة العيلة دي؟
هي دي بالظبط.
بيسموها مشكلة **fragile Base Class Problem**.
الأب بقى هش.

أنت زي الفل يا ابو السيد متقولش على نفسك كده أنت مش هش ولا حاجة
يا سيد ركز معايا الله يصلح حالك ركز معايا يا حبيبي خلينا نخلص الفصل على خير

يعني كده أي تغيير **Supper Class أو Base Class** على حسب التسمية ممكن يكسر عياله.

وده بيخلي الصيانة صعبة جدًا، والاختبار كمان أصعب.
لازم تختبر كل الأبناء بعد أي تغيير في الأب.
وده عكس هدف **Encapsulation** أصلًا.

عشان كده لازم نستخدم الوراثة بحذر شديد.
بالظبط.
لازم علاقة **is-a** تكون حقيقية 100%.
الابن لازم يكون فعلًا نوع متخصص حقيقي من الأب.

الكتاب بيحذر من أمثلة غلط مشهورة، زي إننا نعمل Class شباك (Window) يورّث من مستطيل (Rectangle).
مع إن الشباك شكله مستطيل، بس هو ممكن يستخدم مستطيلات جواه.

لكن هو مش نوع متخصص من الهندسي.
الشباك له وظائف تانية خالص: فتح، قفل، تغيير حجم.
لو ورسناه من مستطيل ممكن نورث حاجات ما لهاش لازمة أو نلخبط الأكواد التانية.

يبقى الاتجاه دلوقتي إننا نفضل التكوين على الوراثة بشكل كبير جدًا
favor composition over inheritance.
أيوة، ده بقى مبدأ تصميمي معروف.
المبرمجين الشاطرين بيفضلوا التكوين كحل أول.
ليه؟ عشان غالبًا بيخلي مكونات النظام مرتبطة ببعض بشكل أخف (Loosely Coupled) وأكتر مرونة.

لما بتبني كائن عن طريق تجميع كائنات تانية، كل واحد بيفضل مستقل بنفسه، ولو غيرت في واحد التأثير على الباقي بيكون أقل بكثير طالما الواجهة بتاعته (Interface) ثابتة.
وده بيحل مشكلة الهشاشة بتاعة الوراثة تمامًا.

ده مش معناه إن الوراثة وحشة، لأ، بس استخدامها يكون للحالات اللي محتاجة "is-a" حقيقية وتعدد أشكال بجد.

والكتاب بيلمح لحاجة هنتكلم عنها بعدين، اللي هي وراثة الواجهات **(Interface Inheritance).**
دي طريقة تانية ناخد بها فوايد الـ **Polymorphism** من غير مشاكل وراثة التنفيذ.

الخلاصة، زي ما الكتاب هيوضح بعدين:
الوراثة بحذر شديد للحالات الخاصة بتاعتها، والتكوين هو الحل العام الأكثر مرونة لبناء السلوكيات المعقدة.

طيب، لو حبينا نلخص كل الكلام ده:
نقدر نقول إن الوراثة والتكوين هما أدوات في شنطة العدة بتاعتنا كمبرمجين عشان ننظم ونُعيد استخدامها.
بس يعني، يبدو إن الكفة بتميل أكتر ناحية التكوين في معظم الحالات عشان المرونة والحفاظ على التغليف.

ده ملخص ممتاز فعلًا. الوراثة قوية للتخصص وتعدد الأشكال، بس بتعمل ارتباط قوي وممكن تسبب هشاشة.
نستخدمها لما يكون الابن فعلًا نوع من الأب.

التكوين بقى فكرته أبسط، زي الليجو، بنركب الأجزاء.
غالبًا بيخلي التصميم مرن أكتر، الأجزاء مستقلة والصيانة أسهل.

مفتاح الإتقان بقى هو إننا نفهم إمتى نستخدم دي وإمتى دي. ودايمًا نسأل نفسنا: هنا في الموقف ده بالذات، إيه الأنسب؟ وراثة ولا تكوين؟

جميل جدًا. وقبل ما نختم، في حتة صغيرة الكتاب عنها لفتت نظري: إن زمان بعض أنظمة البرمجة اللي كانت بتعتبر نفسها كائنية ما كانتش بتدعم الوراثة الكاملة أصلًا، وكانت بتعتمد أكتر على التكوين أو وراثة الواجهات.

تفتكر ده يقول لنا إيه عن تطور طريقة تفكيرنا في البرمجة؟ وإيه المبادئ اللي فعلًا بتعيش وتثبت قوتها مع الوقت؟
سؤال حلو نفكر فيه.

بكده نكون خلصنا الفصل السابع سلام.

الفصل الثامن Frameworks and Reuse: Designing with Interfaces and Abstract:

النهاردة هنشوف مع بعض فصل جديد،
الفصل الثامن من الكتاب.

مهمتنا النهارده هي إننا نفهم إزاي نبني برامج مش بس شغّالة وخلاص،
لأ، الأهم إنها تكون مرنة،
يعني قابلة للتطوير بسهولة في المستقبل.

بالظبط.
هنركز قوي على فكرة **إعادة استخدام الكود** وهنشوف إزاي أدوات زي الواجهات **Interfaces** والفئات

المجرّدة **Abstract Classes** بتساعدنا نحقق ده.

أهلاً.
فعلاً الفصل ده مهم، يعني هو بينقلنا لمستوى تاني خالص في التفكير. لو فاكرين كلامنا قبل كده عن الوراثة **Inheritance** والتكوين **Composition** ، أكيد في الفصول اللي فاتت كانوا طرق لإعادة الاستخدام برضه.

تمام.
النهاردة بقى هنبني على الأفكار دي.
هنشوف إزاي ننظّم عملية إعادة الاستخدام دي بشكل أحسن، بشكل مُمنهج.
ونضمن إن الكود بتاعنا مش بس بيتكتب مرة واحدة ويتنسي،
لأ، المهم إنه يفضل سهل في صيانته وتعديله بعد كده.

طيب، نبدأ من الأول خالص: ليه إعادة استخدام الكود دي مهمة كده؟
يعني ماشي، بتوفر وقت كتابة الكود، ده مفهوم.
بس إيه القيمة الحقيقية التانية؟

بُص يا سيد، توفير الوقت ده مهم أكيد،
بس يمكن مش أهم حاجة.
الأهم منه هو الموثوقية والصيانة.

الموثوقية؟
إزاي؟

الكود اللي بيتكتب مرة واحدة وبيستخدم في أماكن كتير **غالباً بيكون اتجرّب أكتر واتراجع أكتر**،
فبيبقى آه، موثوق فيه أكتر.
يعني أقل عرضة للأخطاء اللي ممكن ما تكونش باينة.

وكمان الصيانة أسهل، صح؟ لو فيه غلطة أو تطوير.

بالظبط.

لما تحتاج تصلح غلطة أو تطور ميزة، بتعمليها في مكان واحد، مش بتقعدي تدوري عليها في عشر أماكن مختلفة، وممكن تنسي مكان منهم والدنيا تبوظ.

تمام.
الفصل ده بقى بيركز على طريقتين أساسيتين عشان نوصل لإعادة الاستخدام المُنظمة دي، صح؟

مظبوط.
الطريقتين دول هم الـ Frameworks والعقود أو الـ Contracts.

طيب نمسك أول واحدة: الـ Framework أو إطار العمل.
بسمع المصطلح ده كتير أوي.
يعني إيه بالظبط Framework؟

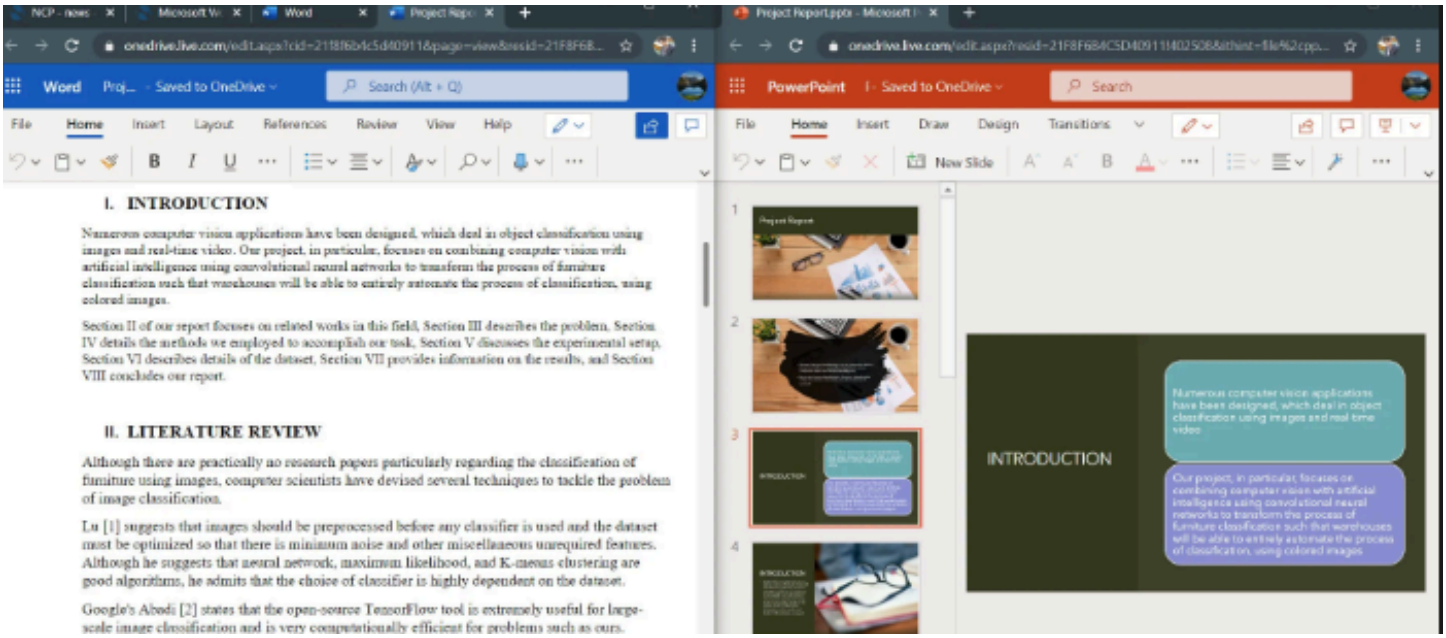
تخيله كده زي صندوق عِدّة كامل متكامل،
بيقدمه حد خبير لحد تاني عايز يعمل حاجة معينة،
بدل ما كل واحد يبدأ من الصفر ويعيد اختراع العجلة كل مرة.

يعني هو بيديني أساسات جاهزة؟
أيوة، بيديكي أساسات ومكونات جاهزة ومجرّبة،
والأهم إنه كمان بيحدد الطريقة اللي المفروض تستخدمي بيها المكونات دي
عشان الجزء بتاعك يشتغل جوه النظام الأكبر.

الفكرة كلها هي توحيد طريقة البناء،

طب ممكن أمثلة من حياتنا عشان الصورة تبقى أوضح؟
في أمثلة كتير حوالينا.

الكتاب ذكر مثال واضح قوي:



برامج الـ Office زي **PowerPoint g Word**، لو أخدت بالك هتلاقي شكل القوائم والأزرار الأساسية وطريقة التعامل معاها شبه بعض جداً بينهم.

فعلاً صح، ده غالباً لأنهم مبنيين باستخدام نفس الـ Framework الداخلي لواجهة المستخدم.

مثال تاني:
أنظمة التشغيل Windows أو Mac أو Linux كلها بتوفر Frameworks لتصميم واجهات المستخدم الرسومي.
عشان كده كل الشبابيك أو الـ Windows ليها شكل أساسي موحّد إلى حد كبير،
يعني زي شريط العنوان فوقه وأزرار التصغير والتكبير والإغلاق.

بالظبط كده.
فهمت.

يعني الـ Framework ده بيخدم المستخدم النهائي في نفس الوقت.المستخدم بيلاقي تجربة استخدام موحّدة وسهلة، والمبرمج بيستفيد من شغل جاهز يوفر عليه وقت ومجهود.

تماماً.

المبرمج بيلاقي أدوات جاهزة لوظايف بتتكرر كتير،
زي مثلاً التعامل مع الملفات (فتح وقفل وحفظ)،
أو رسم العناصر الأساسية على الشاشة،
أو إدارة الأحداث زي الكليكات.

فبيقدر يركز مجهوده أكتر على الجزء المميز في البرنامج بتاعه هو.
وده طبعاً بيسرّع الشغل وبيقلل الأخطاء.

أكيد.

والـ Framework ده عادة مش بيجي لوحده كده،
بيجي معاه زي كتالوج أو وثائق مهمة أوي اسمها الـ API.

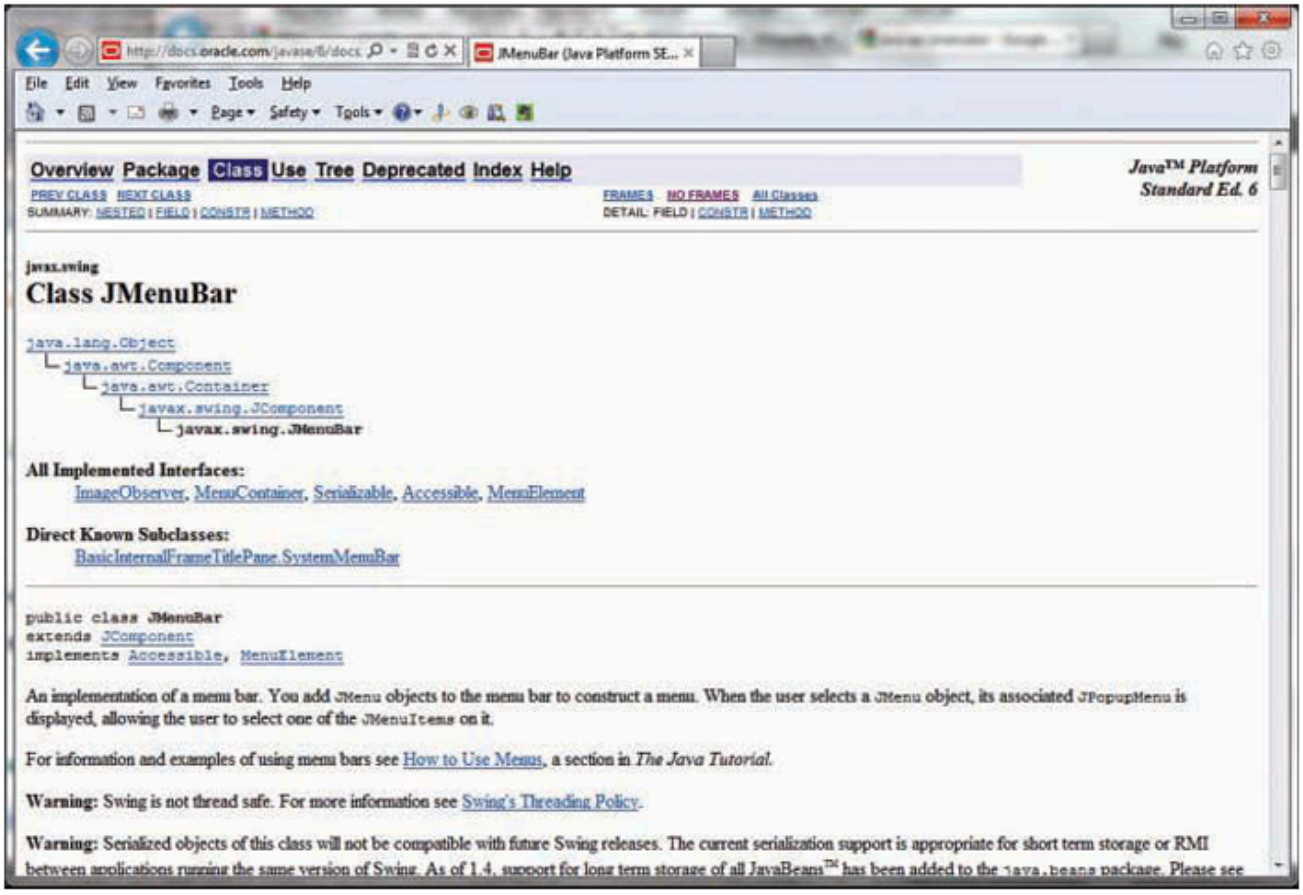


Figure 8.2 API documentation.

الـ API؟ برضو بسمع الكلمة دي كتير.
دي عبارة عن إيه بقى الـ API؟

الـ API أو واجهة برمجة التطبيقات (Application Programming Interface)،
ببساطة شديدة هي اللي بتشرح للمبرمج إيه الأدوات المتاحة في الـ Framework ده؟
وإزاي يقدر يستخدمها ويتعامل معاها.

زي دليل الاستخدام كده،
أو قايمة الأوامر اللي النظام بيفهمها.

يعني هي اللي بتحدد القواعد اللي لازم المبرمج يمشي عليها
عشان يعرف يتكلم مع الـ Framework صح ويستخدم إمكانياته ؟

الله ينور عليك يا سيد،
هي دي بالظبط.

وده بقى بينقلنا للمفهوم التاني المهم أوي في الفصل ده،
اللي هو العقد أو الـ Contract.

العقد؟ زي العقد اللي بنمضيه؟
قريب من المعنى ده،
العقد هنا مش ورقة بنمضي عليها طبعاً،
لكنه أي آلية أو طريقة بنستخدمها في الكود بتاعنا
عشان نضمن إن المبرمج اللي بيستخدم الـ API بتاعتنا أو الـ Framework بتاعنا
هيلتزم بقواعد وشروط معينة إحنا حاطينها.

يعني زي العقد القانوني بينظم العلاقة بين طرفين،
العقد البرمجي بينظم العلاقة بين اللي عمل المكون زي الـ Framework واللي بيستخدمه.

بالظبط كده.

بس استنى لحظة، ليه لازم الإلزام ده؟
يعني مش ممكن ده يحد من إبداع المبرمج أو يخليه حاسس إنه متكتّف؟
ليه ما نسيبش كل واحد يعمل الشغلانة بطريقته طالما في الآخر بتأدي نفس الوظيفة؟

سؤال حلو جداً.
هو الإلزام هنا مش هدفه التقييد في حد ذاته،
هدفه الأساسي هو ضمان التناسق والصيانة على المدى الطويل.

التناسق؟
أيوة.
تخيل لو بنبني بيت وكل عامل بيستخدم مقاسات طوب مختلفة،
أو طريقة توصيل كهربا مختلفة تماماً عن التاني.
ممكن كل جزء لوحده يبقى شغّال زي الفل،
بس لما نيجي نجمع الأجزاء دي مع بعض أو نيجي نعمل صيانة بعد كده،
هتبقى كابوس.

فهمتِ؟
بالظبط.

العقود دي بتضمن إن كل أجزاء النظام بتتكلم نفس اللغة،
وإن الكل ماشي على نفس المعايير الأساسية المتفق عليها.
ده بيمنع إن النظام يتحول مع الوقت لمجموعة جزر منعزلة كل واحدة ماشية في وادي.
وبيخلي عملية التطوير الجماعي والصيانة بعد كده أسهل بكتير.

لأ، تمام، النقطة دي واضحة جداً ومقنعة.

طيب، إزاي بقى بنطبق فكرة العقد دي عملياً جوه الكود؟
الكتاب قال فيه طريقتين أساسيتين في البرمجة كائنية التوجه زي Java مثلاً.
فعلاً عندنا أداتين رئيسيتين عشان نعمل العقود دي ونفرض الالتزام بيها:
الأداة الأولى هي الفئات المجرّدة أو Abstract Classes،
والأداة التانية هي الواجهات أو Interfaces.

ودول بقى هم قلب الفصل ده فعلاً.

أوكي.

نبدأ بالأول: الفئة المجرّدة Abstract Classes.
اسمها غريب شوية: "مجرّدة"،
يعني مش حاجة ملموسة أوي.

هو كده بالظبط.
الفئة المجرّدة هي فئة يعني Class،
بس ما ينفعش ناخد منها نسخة مباشرة أو نعمل منها كائن object.
هي زي قالب أو نموذج أولي كده.

قالب يعني لازم فئة تانية تيجي تورثه الأول عشان نقدر نستخدمه؟
بالظبط، لازم فئة تانية تيجي وتورّث القالب ده الأول.

وإيه بقى اللي بيميز القالب ده عن الكلاس العادي؟
أهم حاجة بتميزه إنه ممكن يحتوي على خليط من حاجتين:

- النوع الأول هو وظائف مجردة أو **Abstract Methods.**
  ودي بتكون مجرد إعلان عن وظيفة،
  يعني بنقول إن أي فئة هتيجي تورّث القالب ده
  لازم يكون عندها وظيفة بالاسم ده وبنفس المدخلات والمخرجات دي.
  بس **إحنا هنا مش بنقول الوظيفة دي بتشتغل إزاي أو الكود بتاعها إيه**،
  يعني بنسيب الفئة اللي بتورّث هي اللي تكتب الكود الفعلي بتاع الوظيفة دي.

بتكون مُجبرة تعمل كده؟
أيوة، بتكون مجبرة.
لو ما ورّثتش وكتبت الكود، كله مش هيشتغل أصلاً.

تمام.
ده النوع الأول.

- **النوع التاني هو وظائف مكتملة أو Concrete Methods.**
ودي بتكون وظائف عادية جداً فيها كود كامل وشغّال وجاهز للاستخدام.
الفئة اللي بتورّث القالب ممكن تستخدم الكود ده زي ما هو بالظبط،
أو لو حبت ممكن تعدّل عليه أو تعمل له Override وتكتب نسختها الخاصة من الوظيفة دي.

آه، يعني الـ Abstract Class دي كأنها بتقدملنا هيكل عظمي بالوظائف المجرّدة الإلزامية دي،
وكمان بتقدّم شوية لحم وعضلات بالوظائف المكتملة دي.

يا سلام عليك يا سيد ايه الجمال ده! هو ده التشبيه بالظبط!

طب إيه فايدتها العملية بقى؟
الفايدة هنا مزدوجة ومهمة جداً:
أولاً: هي بتفرض هيكل أو تصميم أساسي مشترك على كل الفئات اللي بتورّثها،
يعني بتضمن إن كلهم عندهم نفس الوظائف الأساسية المطلوبة.
ثانياً: هي بتوفر كود مشترك جاهز لإعادة الاستخدام عن طريق الـ Concrete Methods،
فبتقلل التكرار وتسهّل الصيانة جداً.

والمثال بتاع الـ Shape اللي في الكتاب كان بيوضّح ده؟
أيوة.

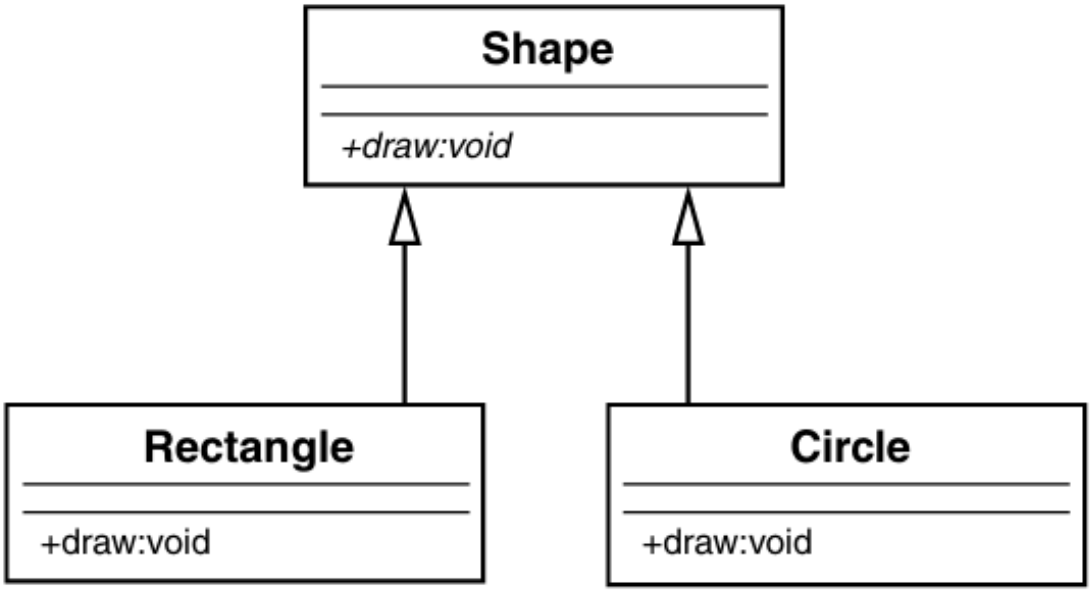


Figure 8.3 An abstract class hierarchy.

ممكن نعمل فئة اسمها Shape (الشكل)،
نحط فيها Abstract Method اسمها Draw (ارسم).

فكده نجبر كل الأشكال اللي هتورّثها زي Circle (الدائرة) وRectangleg (المستطيل)،
إن كل واحدة منهم لازم تحدد بنفسها هترسم نفسها إزاي.

وفي نفس الوقت ممكن نحط فيها وظيفة جاهزة زي SetColor مثلاً.
بالظبط، ممكن نحط Concrete Methods SetColor (تحديد اللون)،
تكون شغّالة لكل الأشكال بنفس الطريقة،
فيستفيدوا منها كلهم.

وهنا بقى بنشوف تطبيق مباشر لفكرة الـ Polymorphism أو تعدد الأشكال اللي كنا اتكلمنا عنها زمان.
نفس الوظيفة، بس بتشتغل بشكل مختلف تماماً حسب مين اللي بينفّذها.

ولما نحب نضيف شكل جديد بعد كده زي مثلث، مش محتاجين نعدل في الـ Shape نفسها، صح؟
لا خالص، بس نعمل Class جديد اسمه Triangle ونكتب الكود الخاص بيه هو.

وده بيخلي النظام مرن جداً وقابل للتوسع.

ممتاز.
كده واضحة جداً.

طيب، نخش على البديل التاني اللي الكتاب طرحه:
الـ Interface – الواجهة.

إيه حكايتها دي كمان؟ تبدو مشابهة شوية.
هي فيها شبه، بس فيه فرق جوهري.

الـ Interface دي بقى ممكن تعتبريها **"عقد صافي"**،
يعني التزام خالص.
هي بتمثل اتفاق على قدرات معينة أو سلوكيات محددة.

يعني إيه عقد صافي؟ وإيه الفرق الأساسي بينها وبين الـ Abstract Class؟
يعني ليه أصلاً نحتاجها طالما الـ Abstract Class ممكن تعمل نفس الحاجة وكمان بتقدّم كود جاهز؟
مش ده أحسن؟

سؤال ممتاز، وهو ده مربط الفرس.

الفرق الأساسي إن الـ Interface بتحتوي فقط على Abstract للـ Methods،
في معظم اللغات التقليدية زي Java (قبل التحديثات الأخيرة عشان نبقى دقيقين).
يعني هي مجرد تعريف للوظائف المطلوبة:
أسماء وتوقيعات، من غير أي سطر كود تنفيذ جواها على الإطلاق.

ما فيهاش Concrete Methods خالص؟
لأ.
هي بتحدد إيه القدرات اللي الفئة لازم توفرها،
لكن مش بتقول إزاي توفرها، ومش بتدي أي مساعدة في التنفيذ.

طيب ليه بقى نحتاجها؟ فين الميزة؟
الميزة الجبارة واللي بتخلي الـ Interface أداة لا غنى عنها في التصميم،
هي إن **الكلاس الواحد يقدر ينفّذ أو يعمل Implement لأكتر من Interface في نفس الوقت**.

آه، هنا بقى النقطة.
يعني بنورّثها؟
هي مش وراثة بالمعنى التقليدي.

الـ Abstract Class بتمثل علاقة "is-a" –
زي: الكلب هو حيوان.
وغالباً في لغات زي Java و C# بنورّث من أب واحد بس، من Class واحدة بس.
صح، وراثة واحدة في الغالب.

لكن الـ Interface بنفّذها، ودي ممكن نعتبرها علاقة "can-do":
يعني الكلب يقدر ينبح،
والكلب يقدر يكون له اسم.
وممكن الكلاس الواحد ينفّذ أكتر من Interface،
كأنه بياخد قدرات أو رخص متعددة من مصادر مختلفة.

بالظبط، بالظبط كده.
بتسمح لنا نحقق شكل من أشكال الوراثة المتعددة للسلوك،
من غير ما ندخل في مشاكل وتعقيدات وراثة الكود المتعددة
اللي ممكن تعمل لخبطة كبيرة – مشكلة الـ Diamond Problem الشهيرة.

طب والمثال اللي في الكتاب؟
المثال بسيط ويوضح الفكرة كويس،
بسيطة جداً.

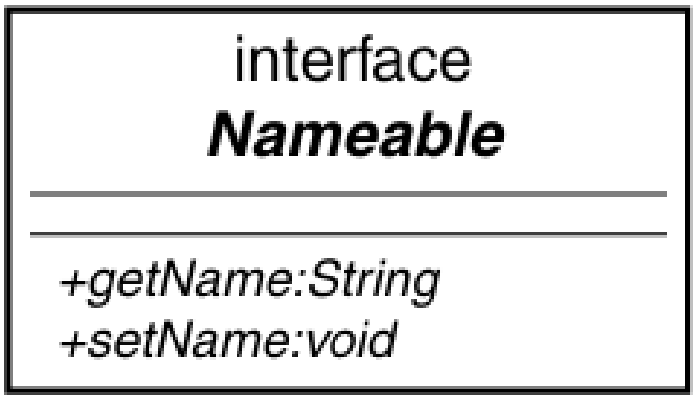


Figure 8.4 A UML diagram of a Java interface.

فيها بس وظيفتين مجردتين:
SetName و getName

أي كلاس بقى أياً كان نوعه، عايز يكتسب القدرة على أن يكون له اسم ونتعامل مع الاسم ده بطريقة موحدة يقدر يعمل Implement للانترفيس دي.

زي إيه كلاسات ممكن تحتاج دي؟ زي Planet (كوكب) أو Car (عربية) أو Dog (كلب) زي ما الكتاب قال. أي واحد من دول يقدر يعمل Implement، وبكده هيكون مجبر إنه يكتب الكود الفعلي بتاع الوظيفتين دول.

والمهم هنا إن الكوكب والعربية والكلب ما فيش بينهم أي علاقة وراثة مباشرة. صح؟ كل واحد من عالم مختلف.
تماماً. لكن كلهم ممكن يحتاجوا القدرة أو السلوك ده، وهي إن يكون لهم اسم نقدر نجيبه ونغيره.

الانترفيس هنا بتوفر طريقة عشان نعرف القدرة دي بشكل مستقل تماماً عن طبيعة الكلاس نفسه.
الصورة كده بدأت توضح أكتر بكتير.

يبقى السؤال المنطقي اللي بيطرح نفسه دلوقتي: بناءً على الفروقات دي أنا كمصممة برامج، **إمتى أختار أستخدم Abstract Class؟ وإمتى أختار أستخدم Interface؟ هل في قاعدة عامة ممكن نمشي عليها؟**

في قاعدة عامة مشهورة ومفيدة جداً، ممكن نعتبرها نقطة بداية كويسة:

- لو بتصمم مجموعة كلاسات بينها علاقة وراثة قوية ومنطقية، يعني علاقة أزقة واضحة جداً زي "is-a"، وفي خصائص أو سلوكيات (يعني كود فعلي) مشترك كتير بينهم وعايزة توفريه للفئات دي عشان تستخدمه أو تبني عليه، يبقى Abstract Class غالباً هو الاختيار الأنسب والأقوى.

- لكن لو عندك قدرة (Capability) ممكن كلاسات مختلفة تماماً عن بعضها، ما لهاش أي علاقة وراثة مباشرة ببعض، وممكن تحتاج تلتزم بالقدرة دي وتنفذها كل واحدة بطريقتها الخاصة، وما فيش بالضرورة كود فعلي مشترك كتير بينهم في القدرة دي نفسها، يبقى الInterface هو الحل الأمثل والأكثر مرونة.

زي اللي شفناها (Nameable).

طيب، وهل ممكن نستخدمهم سوا في نفس التصميم؟ يعني مش لازم أختار يا ده يا ده وخلاص؟
طبعاً، طبعاً. وده الشائع جداً جداً في التصميمات الكبيرة والقوية.

المثال اللي في الكتاب بتاع الكلب كان بيعمل كده بالظبط.
ممكن يورث (Extends) من Mammal عشان ياخد منها كل الخصائص والسلوكيات الأساسية المشتركة بين كل الثدييات زي إنه بيتنفس وعنده قلب مثلاً.
وفي نفس الوقت ينفذ (Nameable (Implements اللي هي عشان يوفر القدرة على التعامل مع اسمه.

آه يعني هو حيوان من Mammal وكمان قابل للتسمية Nameable. بياخد من هنا ومن هنا.
بالظبط.

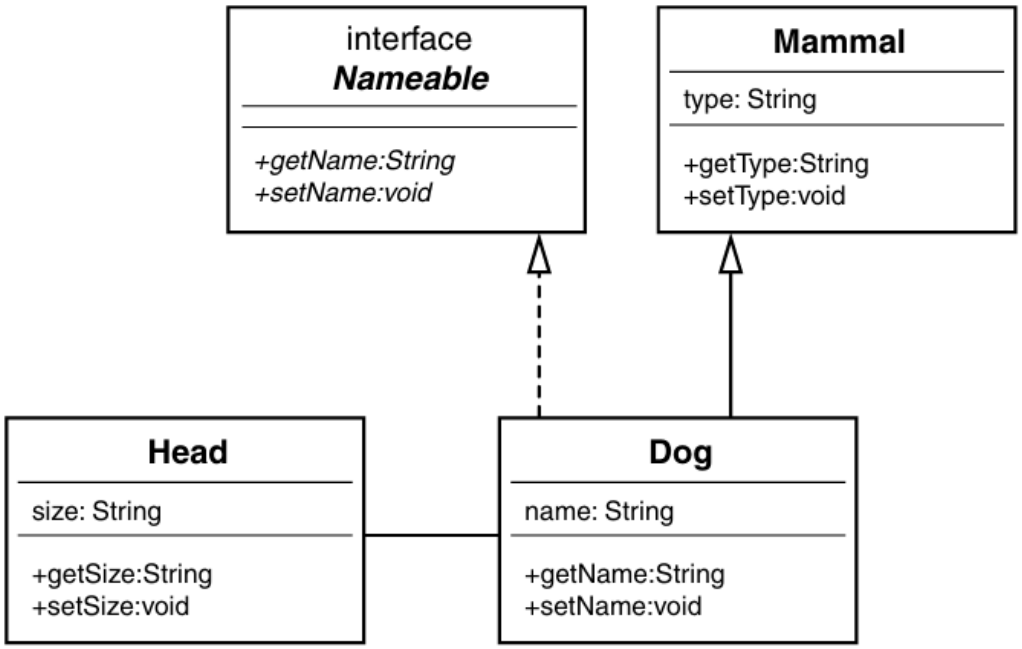


Figure 8.5 A UML diagram of the sample code.

ده بيسمح لنا نستفيد من أفضل ما في العالمين:
قوة الوراثة لتحديد الهيكل الأساسي والكود المشترك، وقوة الواجهات لتحديد القدرات والسلوكيات المطلوبة بشكل مرن وقابل للتوسع عبر أنواع مختلفة جداً من الكلاسات.

يعني لو حبينا نلخص الفرق ده تاني في جملة بسيطة أو تشبيه عشان المعلومة تثبت:
ممكن نقولها كده:ال Abstract Classes زي الأب أو الأم اللي بيورثوا لأولادهم الصفات الأساسية بتاعة العيلة، وممكن كمان يدوهم شوية حاجات جاهزة ممكن يستخدموها زي ما هي أو يغيروا فيها براحتهم.

تمام.

أما الInterface فا زي ما تكون شهادة خبرة أو رخصة قيادة. الشهادة دي بتقول إن اللي معاه الشهادة دي (اللي بينفذ الInterface) بيقدر يعمل حاجة معينة زي القيادة أو السباحة أو إنه يكون له اسم.
وممكن الواحد (الكلاس) ياخد أكتر من شهادة أو رخصة من دول.

فيه طريقة تانية برضه ممكن نفكر بيها، وهي طريقة قوية جداً ومفيدة في التصميم:
إننا نعتبر الAbstract Class أو الInterface زي نقط توصيل أو فيشات كهرباء مرنة.

يعني في النظام بتاعنا...
فيشات كهربا؟ إزاي يعني؟
يعني بدل ما جزء من الكود (نسميه العميل أو الClient) يكون معتمد بشكل مباشر ومربوط جامد قوي

بكلاس معين بعينه، زي مثلاً PizzaShop، لأ... العميل ده بيعتمد على العقد أو الفيشة اللي الكلاس ده ملتزم بيها، سواء كان العقد ده Abstract Class زي Mammal مثلاً أو Interface زي Nameable.

النقطة دي مهمة جداً فعلاً.
تقصد إن ده بيخليني أقدر أغير الحاجة اللي راكبة في الفيشة دي من غير ما أغير الجهاز نفسه اللي بيستخدم الكهرباء اللي جاية من الفيشة. يعني ممكن أفصل PizzaShop وأركب مكانه مثلاً CoffeeShop، والكود اللي بيستخدم الخدمة دي مش هيحس بأي فرق خالص.

بالظبط كده
طالما الكلاس الجديد اللي هتركبيه في المسالك بيركب في نفس الفيشة، يعني بيلتزم بنفس العقد سواء كان بيورث نفس الAbstract Class أو بينفذ نفس الInterface، الكود الأصلي اللي كان بيتعامل مع الفيشة دي مش محتاج يتغير ولا يعرف أصلاً مين اللي راكب دلوقتي.

ده بيقلل بشكل كبير حاجة اسمها الاعتمادية **coupling** بين أجزاء النظام، صح؟
تماماً.

coupling المصطلح ده مهم، وببساطة معناه: درجة إن أجزاء البرنامج المختلفة معتمدة على تفاصيل بعض الداخلية بشكل كبير.
كل ما ده يقل، كل ما يبقى أحسن.

ليه يبقى أحسن؟ لأنه بيخلي تغيير جزء في النظام ما يأثرش بشكل كبير أو غير متوقع على الأجزاء التانية. لو غيرت حتة في الكود ما تضطرش تغير عشر حتت تانية معاها عشان كله يفضل شغال.

زي ما بتقدر تغير نوع كاوتش العربية من غير ما تغير الموتور بتاعها، لأن فيه عقد أو مواصفات قياسية متفق عليها لنقطة التوصيل.

بس سؤال: مش ممكن كتر استخدام Abstract Classes وInterfaces يخلي الواحد يتوه ويحس إن الكود بقى معقد زيادة عن اللزوم بدل ما يتبسط؟

أكيد في توازن مطلوب. نقطة ممتازة وفي محلها تماماً. أكيد لازم نوازن.
ما فيش حاجة ببلاش.

زي أي أداة قوية،
الإفراط في استخدامها أو استخدامها في غير مكانها الصح ممكن يؤدي لتعقيد مبالغ فيه، ويصعب فهم الكود وصيانته بدل ما يسهله.

يعني لو عملنا Interface لكل حاجة صغيرة وخلاص...
بالظبط، ممكن نلاقي نفسنا غرقانين في طبقات فوق طبقات من الAbstraction اللي ممكن ما تكونش ليها لازمة حقيقية.

الحكمة هنا هي في تحديد العقود أو نقاط التوصيل الأساسية والمهمة فعلاً في النظام بتاعنا، اللي إحنا فعلاً محتاجين عندها المرونة دي، ونستخدمها بشكل مدروس ومناسب للمشكلة اللي بنحلها.

تمام.

طيب عشان نربط كل الكلام النظري الجميل ده بالواقع أكتر، ونشوف فايدته العملية، خلينا نرجع لمثال موقع طلبات البيتزا والدوناتس اللي الكتاب استخدمه عشان يوضح الفكرة دي.

تخيل بدأنا المشروع ده فعلاً، وصاحب محل البيتزا وصاحب محل الدوناتس، كل واحد فيهم عملنا له نظام إلكتروني منفصل تماماً عن التاني، من غير ما نفكر خالص في المفاهيم اللي كنا بنتكلم فيها دي.

For example, consider two separate applications, as represented by the UML diagram in Figure 8.6.

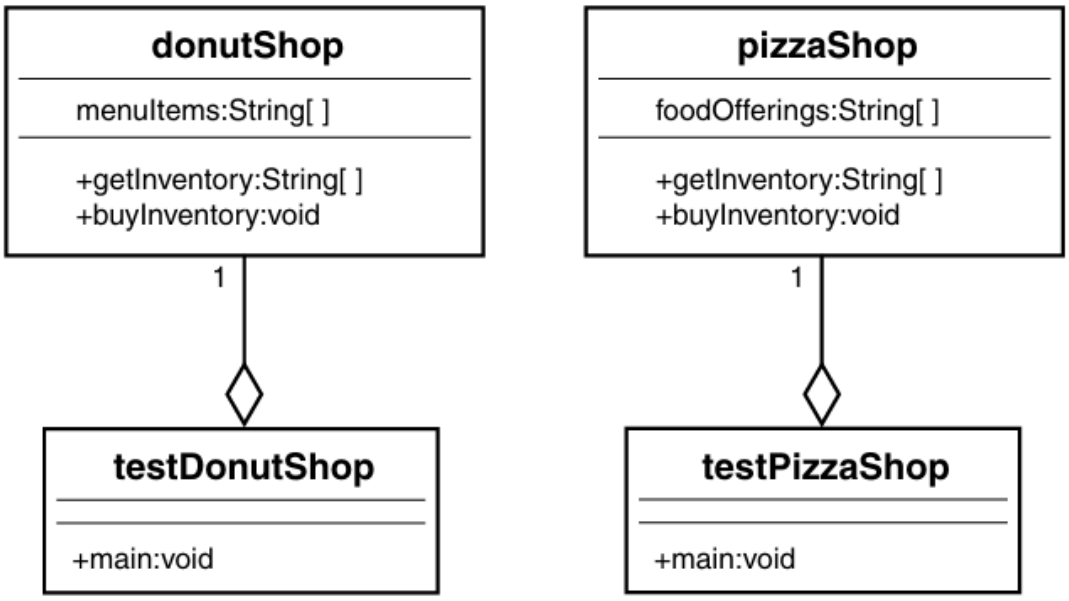


Figure 8.6 Applications on divergent paths.

إيه الكارثة اللي كانت ممكن تحصل؟
الكارثة الكلاسيكية اللي للأسف بتحصل في مشاريع كتير جداً: **تكرار الكود** بشكل مرعب.

هتلاقي نفسك بتكتب كود عرض المنتجات، وكود عملية الشراء، وكود حساب الضرائب، وكود إدارة العملاء **مرتين منفصلين تماماً:**
مرة في نظام البيتزا، ومرة تانية طبق الأصل بالظبط في نظام الدوناتس.

والمصيبة الأكبر بتيجي مع الصيانة والتطوير بعد كده، ودي هي الكارثة الحقيقية.

لو اكتشفت غلطة مثلاً في طريقة حساب الضرائب في كود البيتزا وصلحتها، نسبة إنك تنسى و تروح تصلحها بنفس الطريقة بالظبط في كود الدوناتس كبيرة جداً.

> **Tracking Down a Bug**
>
> The fact that the bug turned up in the pizza system does not mean that it will also turn up in the donut system. Even though the bug caused a crash in the pizza shop, the donut shop might never encounter it. It may be that the fix to the pizza shop's code is more dangerous to the donut shop than the original bug.

مع الوقت هتلاقي النسختين دول بيبعدوا عن بعض تماماً، وكل نسخة ماشية في سكة لوحدها.
زي ما الكتاب سماها بشكل ساخر: **Version 2.01papa** و **Version 2.03dad**.

وساعتها هتبقى عملية إضافة أي ميزة جديدة أو حتى إصلاح الأخطاء البسيطة **كابوس حقيقي**، مضاعفة ومجهود ضايع.

يعني الحل المنطقي والعملي – زي ما الكتاب بيقترح – إننا من الأول خالص نفكر بطريقة Object-Oriented، ونستخدم العقود اللي اتعلمناها دي.

بالظبط.
بدل ما نبني نظامين منفصلين، لأ... إحنا نبني **إطار عمل واحد** للمتاجر الإلكترونية دي بشكل عام.

وأول خطوة في بناء الإطار ده هي إيه؟
إننا نحدد إيه هي الوظائف الأساسية المشتركة اللي أي محل إلكتروني هيحتاجها، زي مثلاً إنه يعرض المنتجات اللي عنده ويسمح للعميل يشتريها.

تمام.

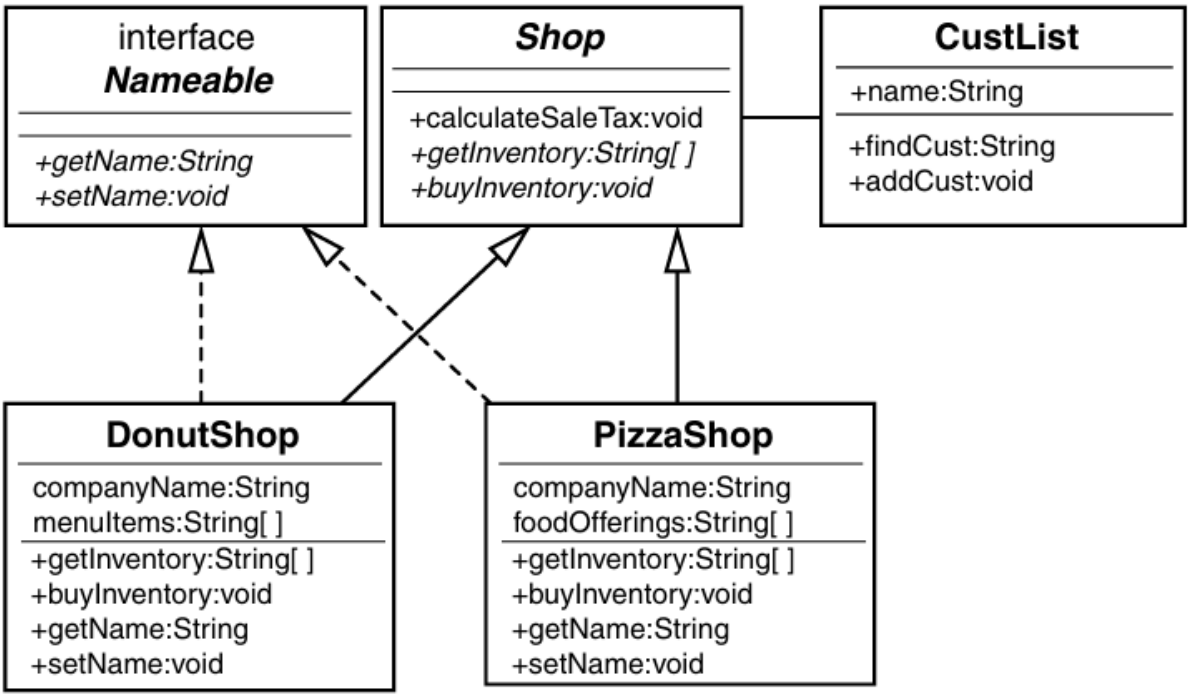


Figure 8.7 A UML diagram of the Shop model.

زي وظيفة عرض المنتجات المتاحة getInventory، وظيفة شراء منتج buyInventory... ونروح عاملين Abstract Class اسمها **Shop**.
ده بيمثل مفهوم المحل بشكل عام ومجرد.

وال**Shop** Abstract Class دي هيكون جواها إيه؟
هيكون جواها الAbstract Methods بتاعة الوظائف الأساسية دي: **getInventory()** و**buyInventory()**، عشان نجبر أي محل حقيقي هيجي بعد كده زي **PizzaShop** أو **DonutShop** إنه لازم يوفر طريقته الخاصة لتنفيذ الوظائف دي.

كل محل بيعرض وبيبيع بطريقته.

وممكن كمان نحط فيها كود جاهز مشترك؟
أكيد، ممكن نحط فيها أي كود مشترك ومفيد لكل المحلات، زي مثلاً دالة جاهزة لحساب ضريبة المبيعات **calculateSaleTax**، لأنها غالباً هتكون نفس الطريقة لكل المحلات.

وممكن كمان نستخدم Interface زي **Nameable**، عشان نضمن إن كل محل له طريقة موحدة نقدر نعرف بيها اسمه ونغيره لو حبينا.

تمام.

والفئات الحقيقية بقى – الكلاسات اللي بتمثل المحلات الفعلية زي **PizzaShop** و**DonutShop** – دي هتعمل إيه بالظبط؟

الموضوع بقى بسيط دلوقتي:
كل فئة منهم ببساطة شديدة هتورث **Abstract Class Shop** (يعني تعمل extends)، وفي نفس الوقت هتنفذ ال**Nameable** Interface (يعني تعمل implements).

وبكده هيحصل حاجتين:

1. الحاجة الأولى: هتكون مجبرة إنها تكتب الكود الخاص بيها للوظائف المجردة اللي ورثتها من **Shop** ونفذتها من **Nameable**، زي **getInventory()** و**buyInventory()** – كل محل بطريقته الخاصة.
2. الحاجة التانية: هتقدر تستفيد ببلاش من أي كود جاهز موجود في **Shop**، زي **calculateTax()** مثلاً، من غير ما تضطر تكتبه تاني أو تكرره.

وممكن كمان **Shop** يحتوي على مكونات تانية زي قايمة العملاء؟
طبعاً، ممكن **Shop** يحتوي على كلاسات تانية عن طريق **Composition**، زي مثلاً يحتوي على Object من Class اسمه **CustList** لإدارة قايمة العملاء بتوعه.
فكل المحلات اللي بتورث **Shop** بتورس القدرة دي كمان بشكل غير مباشر.

الأهم بقى من كل ده إن الكود الرئيسي بتاع الموقع – اللي بيعرض الحاجة للمستخدم أو اللي بينفذ عمليات الشراء الفعلية – مش محتاج يعرف هو بيتعامل مع محل بيتزا ولا محل دوناتس بالتحديد.

هي دي النقطة الجوهرية والفايدة الكبرى.

الكود الرئيسي في التطبيق هيتعامل مع العقد أو الواجهة الموحدة – اللي هي **Shop**، أو لو كان محتاج الاسم بس مع **Nameable**.

يعني الكود هيقول ببساطة:

"يا أيها الShop اللي قدامي، هات لي المنتجات بتاعتك (**getInventory()**)... أو يا أيها الShop نفّذ عملية الشراء دي (**buyInventory()**)."

هو مش مهتم مين الShop ده تحديداً – بيتزا ولا دوناتس ولا غيره.

وده بيفتح الباب لإضافة أنواع محلات جديدة في المستقبل بسهولة جداً.

بمنتهى السهولة، لو بكرة الصبح حبينا نضيف محل ورد أو مكتبة أو أي نوع محل جديد، كل اللي هنعمله إننا نعمل Class جديد للمحل ده ونخليه يلتزم بنفس العقد (يورث **Shop** وينفذ **Nameable**).

الكود الرئيسي بتاع الموقع مش هيحتاج أي تعديل عشان يقدر يتعامل مع المحل الجديد ده فوراً.

ده حتى الكتاب أشار لنقطة متقدمة شوية وهي إمكانية تحديد نوع المحل الفعلي اللي هيتم استخدامه وقت تشغيل البرنامج نفسه.

أيوة، ديناميكياً **وقت الRuntime.**

وده بيزود المرونة لأقصى درجة ممكنة.
يعني نفس الكود يقدر النهاردة يعرض محل بيتزا، وبكرة يعرض محل دوناتس – حسب اختيار المستخدم مثلاً – **من غير ما نغير سطر واحد في الكود الأساسي.**

بكده فعلاً تكون الصورة اكتملت تماماً
فهمنا منها إزاي مش مجرد مصطلحات نظرية معقدة بنحفظها وخلاص، لا طبعاً.
دي أدوات عملية وقوية جداً بنستخدمها كل يوم عشان نبني كود منظم، مرن في صيانته وتطويره، والأهم قابل لإعادة الاستخدام بجد وفعالية.

يعني نقلة نوعية في التفكير، بالظبط المفاهيم دي هي اللي بتفرق بين واحد بيكتب كود بيؤدي الغرض المطلوب منه دلوقتي وخلاص، وبين مهندس برمجيات بيصمم وبيهندس نظام برمجي قادر يستحمل التغيير والتطور في المستقبل بسهولة وكفاءة.

وفن الاختيار بقى بين abstract classesg interfaces وتحديد العقود المناسبة فين بالظبط في النظام، ده جزء أساسي من الخبرة؟ أكيد.
دي مهارة بتيجي مع الخبرة والممارسة والتفكير العميق في التصميم.

والمثال بتاع البيتزا والدوناتس ده بيورينا بشكل عملي إزاي التفكير ده، حتى لو خد وقت أطول شوية في الأول، ممكن يوفر فلوس ومجهود ووقت وصداع كبير جداً على المدى الطويل.

وكالعادة نحب نسيب سؤال أخير كده للتفكير مع بعض.
ليه بنحتاج عقود (interfaces) ووجهات تعامل واضحة عشان ننظم التفاعل بين الأجزاء المختلفة ونضمن المرونة في نفس الوقت؟

هل الفكرة دي ممكن نلاقيها أو نطبقها في مجالات تانية خالص برة البرمجة؟
ممكن في حياتنا اليومية في تنظيم فرق العمل أو في أي نظام تاني بنتعامل معاه؟

بس كده سلام.

الفصل التاسع Building Objects and Object-Oriented Design:

بسم الله.
النهاردة هنكمل كلامنا في كتاب The Object-Oriented Thought Process، تركيزنا هيكون على الفصل التاسع.
الفصل ده بيبني على اللي فات،
يعني الفصل السابع والتامن اتكلمنا عن الـ Inheritance والـ Composition.

بس هنا بقى مش بنشوف بس إزاي نبني الكائن Object نفسه، لأ،
إزاي الكائنات دي يعني بترتبط ببعضها، إزاي بتتفاعل عشان تعمل سيستم كامل.

يعني الفصول اللي فاتت كنا كأننا بنتعلم نعمل الطوبة نفسها ال Object.
عرفنا علاقة الـ Inheritance (IS-A) والطوبة الجديدة اللي وارثة من قديمة.
صح.
وعرفنا علاقة الـ Composition (HAS-A) اللي هو بنستخدم طوب نبني به حيطة أكبر.
الفصل التاسع ده بيتكلم عن هندسة السيستم كله.
إزاي نرص الطوب ده جنب بعضه؟ آه، أنواع العلاقات بقى بين المكونات دي.
بالظبط كده.

وإيه الفرق الجوهري بينهم؟ نفتكر تاني: الـ Inheritance ده واحد بمفهومه بيلم صفات كائن كله.
لكن الـ Composition لأ، ده كائن أو أكتر بيبنوا كائن تاني، يعني كيانات منفصلة بتتفاعل.

طب عشان نفرق أكتر ممكن نبدأ بمثال اللي في الكتاب بتاع الـ Employee.

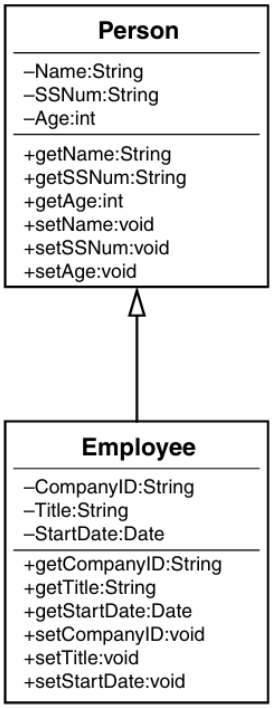


Figure 9.1 An inheritance relationship.

المثال ده بسيط ومباشر.
الموظف Employee هو بطبيعته شخص Person.
مش كده؟ تمام.
مش محتاج يعني يبعت رسالة ل Person object تاني عشان يسأله عن اسمه مثلا.
لأ.
هو نفسه جواه كل حاجة بتاعة الـ Person.
هو كيان واحد مدمج.
وده عكس الـ Composition تماما.
بالظبط.

لو كنا استخدمنا Composition كنا ممكن نعمل Employee Class جواه كائن تاني مثلا Person data، ساعتها ال Employee لازم يكلم Person data عشان ياخد منه المعلومة.
دول كيانين منفصلين بيتفاعلوا.
تمام.
والكتاب بيركز قوي على إن ال Composition ده هو طريقتنا الطبيعية في التفكير وفي بناء الأنظمة المعقدة.
يعني إحنا لما بنبص للعربية مش بنشوفها كومة حديد وتروس، صح؟ لأ طبعا.
بنشوفها عربية كوحدة واحدة.
ده تجريد Abstraction بيسهل علينا الدنيا.
في البرمجة الـ Composition بيعمل كده.
أيوة، يبني حاجة معقدة زي الـ Car Object من حاجات أبسط زي الـ Wheel والـ Engine وكده.

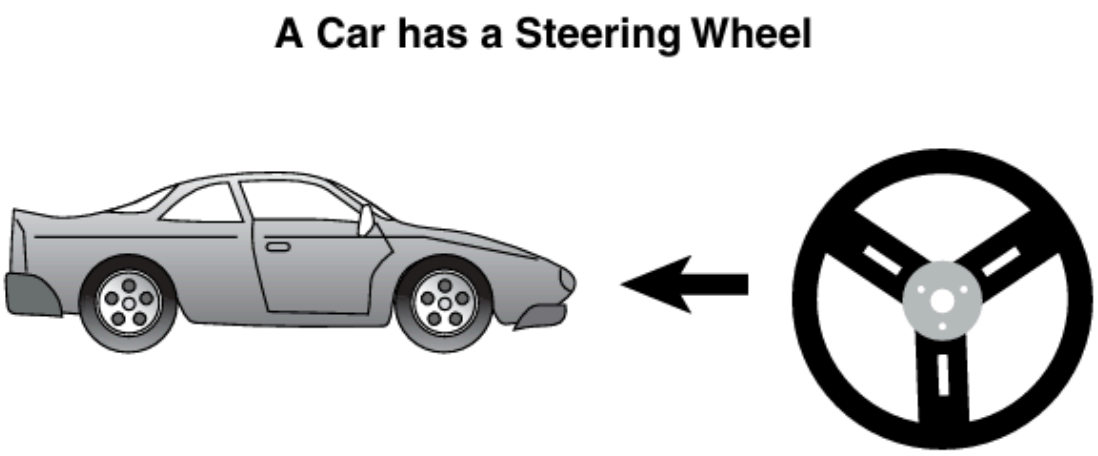


Figure 9.2 A composition relationship.

والحتة دي بتدينا مرونة كمان، زي ما الكتاب بيقول، فكرة إن الأجزاء قابلة للتبديل.
زي كاوتش العربية نغيره أو نركب موتور أقوى.
تمام.
والموتور ده ممكن نستخدمه في عربية تانية.
يعني إعادة استخدام. دي مميزات قوية قوي.
أكيد.
وده يودينا بقى لنقطة مهمة جدا:
أفكار واحد اسمه **هيربرت سايمون.**
آه، العالم اللي واخد نوبل.
بالظبط.
هو فيه مقالة ليه اسمها **The Architecture of Complexity**، حلل فيها ليه الأنظمة المستقرة سواء في الطبيعة أو المجتمع غالبا بتبقى مبنية بطريقة معينة.

- **"Stable complex systems usually take the form of a hierarchy, where each system is built from simpler subsystems, and each subsystem is built from simpler subsystems still."**—You might already be familiar with this principle because it forms the basis for functional decomposition, the method behind procedural software development. In object-oriented design, you apply the same principles to composition—building complex objects from simpler pieces.
- **"Stable, complex systems are nearly decomposable."**—This means you can identify the parts that make up the system and can tell the difference between interactions between the parts and inside the parts. Stable systems have fewer links between their parts than they have inside their parts. Thus, a modular stereo system, with simple links between the speakers, turntable, and amplifier, is inherently more stable than an integrated system, which isn't easily decomposable.

- **"Stable complex systems are almost always composed of only a few different kinds of subsystems, arranged in different combinations."**—Those subsystems, in turn, are generally composed of only a few different kinds of parts.
- **"Stable systems that work have almost always evolved from simple systems that worked."**—Rather than build a new system from scratch—reinventing the wheel—the new system builds on the proven designs that went before it.

والكتاب بيستخدم الأفكار دي عشان يورينا قوة الـ Composition في البرمجة.
طب إيه أهم ملاحظات سايمون دي؟
بص، هو لاحظ كذا حاجة:
أولا: الأنظمة المعقدة اللي شغالة كويس دي غالبا **بتكون هرمية**.
يعني إيه؟ يعني بتتكون من أنظمة فرعية أصغر، وكل نظام فرعي ممكن يتكون من أنظمة أصغر منه وهكذا.
زي جسم الإنسان كده: أجهزة وأعضاء وأنسجة.
تمام.
ثانيا: الأنظمة دي **قابلة للتجزئة** بشكل كبير **(Nearly Decomposable).**
يعني التفاعلات والاعتماديات جوة كل جزء بتكون أقوى بكتير من التفاعلات بين الأجزاء المختلفة.
وده اللي بيخليها مستقرة؟ بالظبط.
لو حصل خلل في حتة، تأثيره محدود على الباقي.
ثالثا: الأنظمة دي عادة **بتتكون من أنواع قليلة من المكونات الأساسية**، بس مترتبة بطرق مختلفة.
زي الحروف الابجدية كده.
الله ينور عليك.

رابعا: الأنظمة المستقرة دي غالبا **بتيجي من تطوير لأنظمة أبسط كانت شغالة أصلا**.
يعني **تطور تدريجي**، مش بنبدأ من الصفر.
فهمت؟ يعني أفكار سايمون دي كأنها بتقول لنا ليه الـ Composition كويس في البرمجة؟ **Nearly Decomposable** الحتة دي .
هي دي Loose coupling اللي بنتكلم عنها.
هي دي بالظبط.
يعني كل جزء مستقل ومعتمد على نفسه في وظيفته.
وده بيخلي التعديل والتصليح أسهل.
أكيد، مش خايف تغير حاجة هنا تبوظ الدنيا في حتة تانية.
والكتاب جايب مثال حلو قوي عشان يوضح ده:

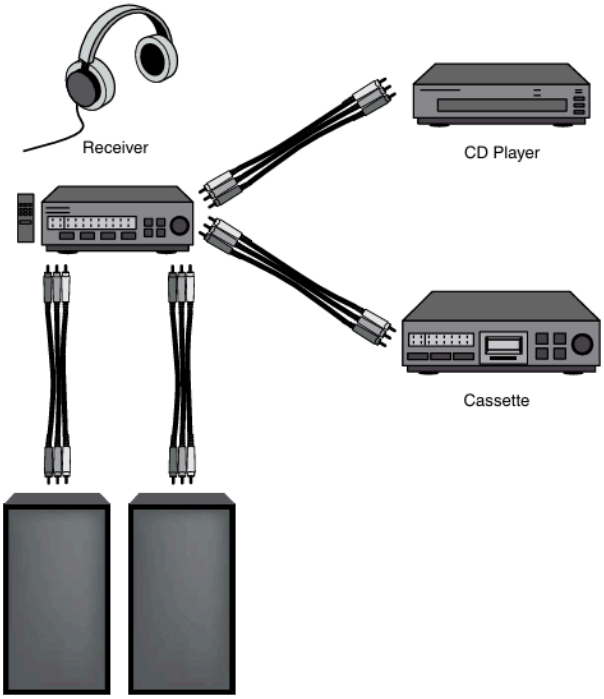


Figure 9.3 Building, testing, and verifying a complete system one step at a time.

الفرق بين جهاز الاستيريو اللي هو حتة واحدة.
آه، اللي فيه راديو وكاسيت وسي دي مع بعض.
أيوة.
وبين نظام الاستيريو اللي هو مكونات منفصلة.
الراديو لوحده.
السي دي لوحده.
السماعات لوحدها.
تمام.
في المدمج لو CD باظ، ممكن الجهاز كله يقف.
أو على الأقل مش هتعرف تشغل الباقي لحد ما يتصلح.
صح.
إنما في المكونات لو CD Player باظ، بتاخده هو تصلحه والباقي شغال عادي.
ودي ميزة كبيرة في الصيانة والاختبار.
بالظبط.
كل مكون بيتصلح لوحده وتختبره لوحده.
مثال ممتاز.
طيب، كده فهمنا أهمية الـ Composition ومبادئ سايمون.

الفصل بقى بيبدأ يفصص أنواع الـ Relationships دي نفسها.
بيركز على نوعين أساسيين:
الـ Aggregation (التجميع) والـ Association (الارتباط).

Is Composition a Form of Association?

Composition is another area in OO technologies where there is a question of which came first, the chicken or the egg. Some texts say that composition is a form of association, and some say that an association is a form of composition. In any event, in this book, we consider inheritance and composition the two primary ways to build classes. Thus, in this book, association is considered a form of composition.

نبدأ بالـ Aggregation.

Aggregation ده يعني الشكل اللي ممكن ييجي في بالنا على طول:
ال object الكبير (الـ Whole) بيتكون من object أصغر.
العلاقة هنا قوية شوية.

زي التليفزيون ومكوناته.
بالظبط.
التليفزيون فيه شاشة ودوائر وهيكل.
إحنا بنتعامل معاه كواحدة واحدة.
مش بنفكر في أجزاؤه دي طول الوقت.
أو العربية وموتورها وشاسيهها.
تمام.

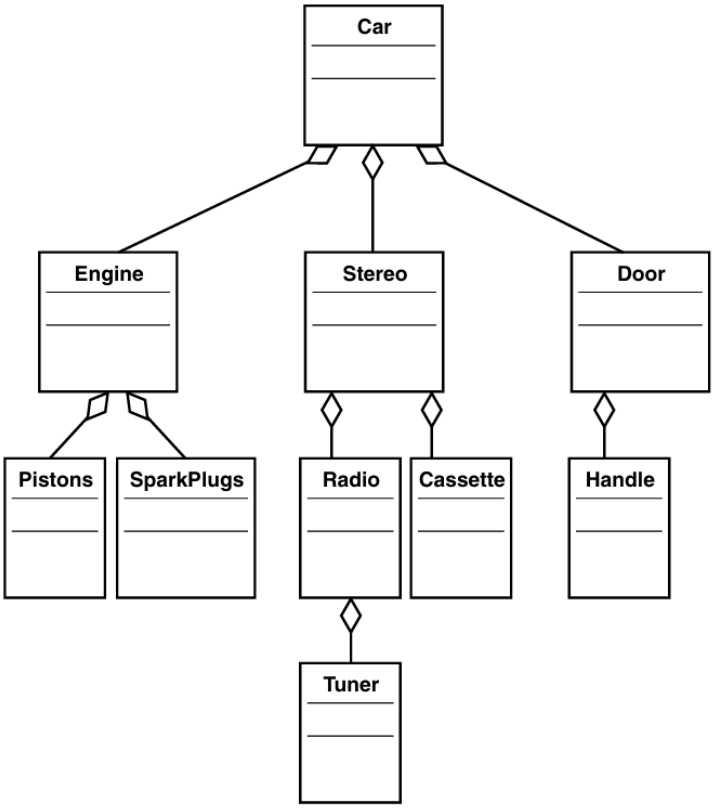
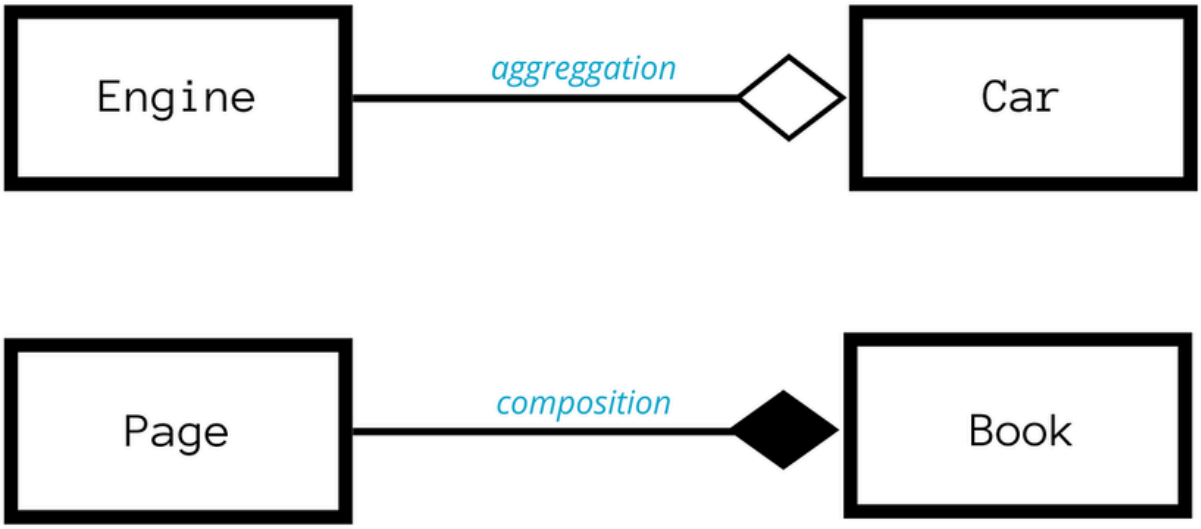


Figure 9.4 An aggregation hierarchy for a car.

في الـ UML، اللغة الرسومية دي اللي بنوصف بيها التصميم، بنرسم خط بين الكلاسين.
ونيجي ناحية الكلاس الكل (الحاوية) ونحط شكل معين كده زي المعين أو الدايموند.

بس الفكرة إن الشكل ده بيقول إن ده الكل aggregate.
وده بيوحي بالملكية أو الاحتواء.
يبقى **aggregation** دي علاقة "جزء من" (Part-of Relationship) قوية.

طب هل ده معناه إن لو الكل ده (الحاوية) اختفى، الأجزاء كمان تختفي؟
سؤال كويس.



ال aggregation هو فيه درجات: فيه شكل أقوى بيسموه Composition، وده اللي الدايموند بتاعه **أسود** في الـ UML.
ده معناه إن حياة الأجزاء مرتبطة بحياة الكل فعلا.
لو الحاوية اتدمرت، الأجزاء بتروح معاها.
تمام.

لكن ال **aggregation** بمعناها الأعم، **الدايموند الأبيض** ممكن تكون أضعف شوية.
الأجزاء ممكن نظريا تعيش لوحدها.
بس في السياق ده هي جزء أساسي من الكل.
الكتاب هنا بيستخدم Aggregation بالمعنى الواسع شوية.
المهم نفهم فكرة الاحتواء: **"جزء منه".**

زي الفل يا أبو السيد، كده وضحت.

إيه بقى النوع التاني؟ الـ **Association**.
الـ Association أو الارتباط.
لأ، دي علاقة مختلفة.
هنا الأوبجكتس بتتعاون مع بعض، بس ما فيش علاقة "جزء من" أو ملكية قوية زي الـ Aggregation.
هم زي ما يكونوا كيانات مستقلة، كل واحد له حياته، بس بيتعاملوا مع بعض.

زي إيه مثلا؟ مثال الكتاب بتاع الكمبيوتر المكتبي ومكوناته الخارجية:

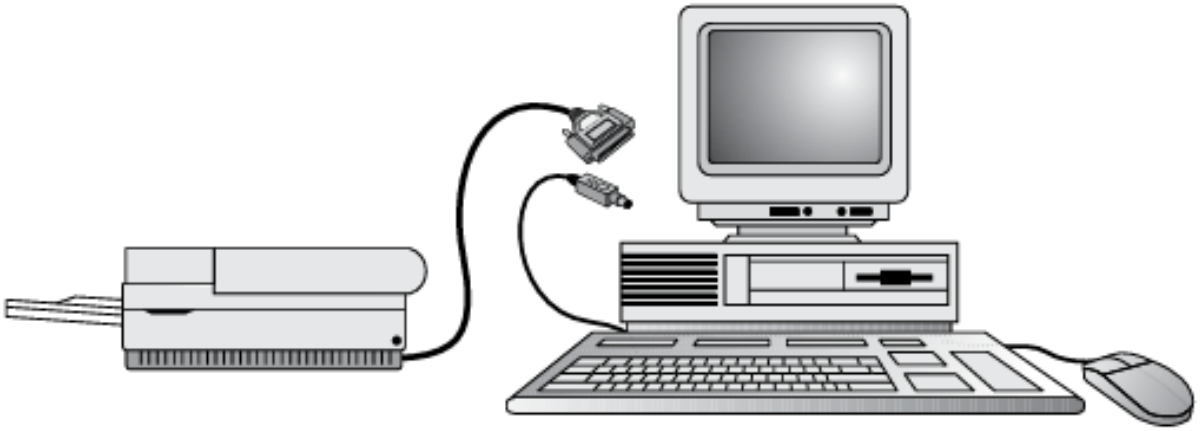
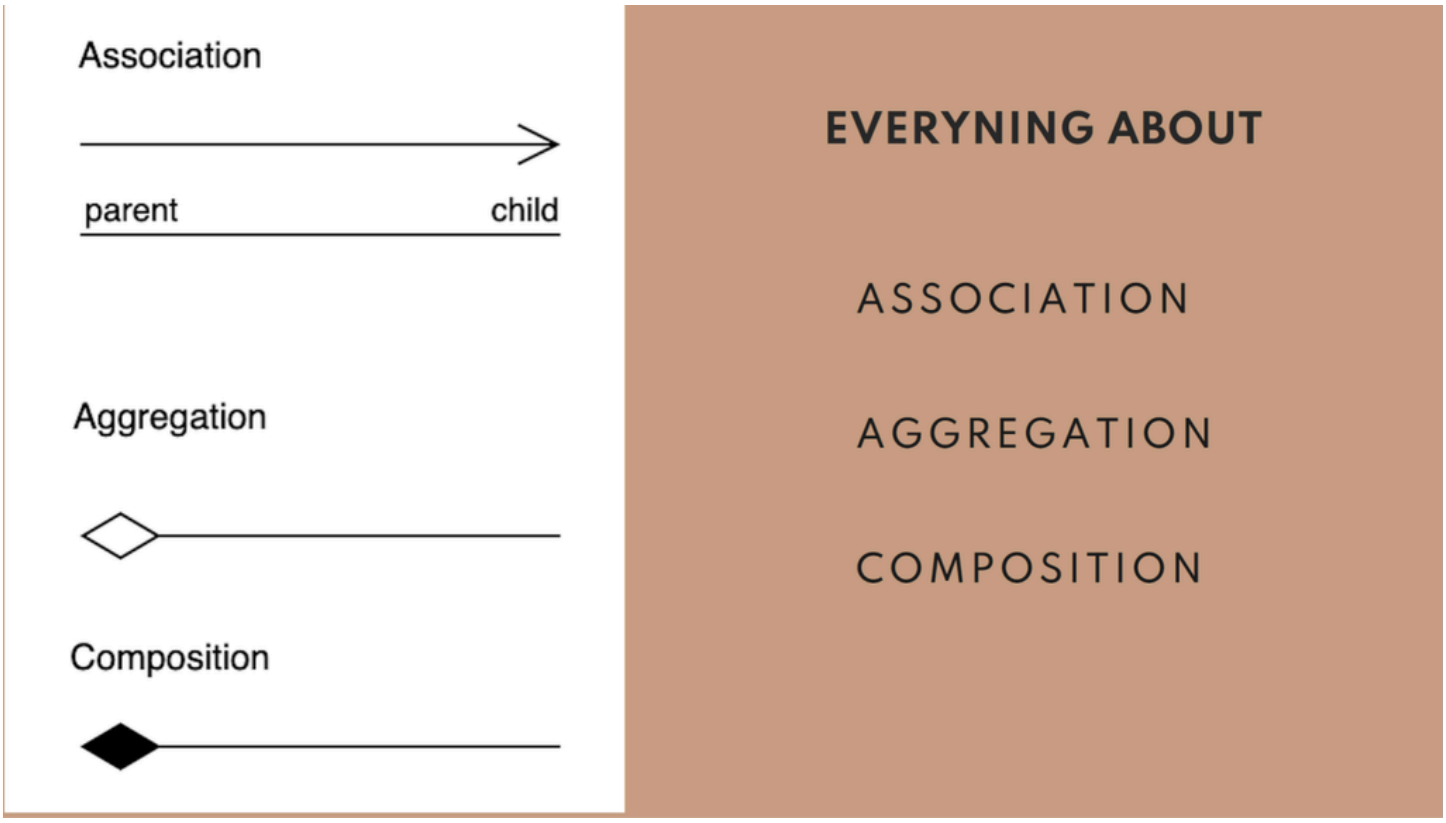


Figure 9.5 Associations as a separate service.

الكيسة، الشاشة، الكيبورد، الماوس.
الكيبورد **مش جزء من** الشاشة.
والشاشة **مش جزء من** الكيسة.
بالمعنى القوي بتاع "جزء من".
آه، هم كيانات منفصلة بترتبط ببعض عشان السيستم يشتغل.
بالظبط.

الكيسة في الكمبيوتر بتحتاج خدمة من الكيبورد، والكيبورد بيطلب الخدمة دي من المكوّن (Object) المختص. في ال **UML ال Association** دي في مجرد **خط عادي بيربط بين الكلاسين**، وده معناه إن العلاقة أضعف، مجرد تعاون أو استخدام خدمة، مش علاقة "جزء من".

و ده شكلها في ال UML



ممكن نفكر فيها زي البيت والسبّاك: الغرف جزء أساسي من البيت، لكن السباك كيان مستقل بييجي يقدّم خدمة ويمشي. دي بالظبط فكرة الـ **Association**.

في المقابل، الـ **Aggregation** (الاحتواء) معناه إن الكائنات مترابطة ترابط قوي، علاقة "جزء من". مثال: الكيسة من جوة فيها بروسيسور ورام ومكونات تانية؛ دي أجزاء أساسية منها، ومش هتشتغل الكيسة من غيرها.

الكتاب وضّح بمثال تاني: الموظف (Employee Object).

- ممكن يكون جواه عنوان (Address object). العنوان هنا جزء أساسي من بيانات الموظف (Aggregation).
- في نفس الوقت، نفس الموظف ممكن يكون مرتبط بكائن تاني وهو الزوج/الزوجة (Spouse). العلاقة دي Association، **لأن الزوج كيان مستقل مش جزء من الموظف، لكنه مرتبط بيه.**

الفرق هنا مهم جدًا: اختيارك بين Associationg Aggregation مش رفاهية، ده بيعكس طبيعة العلاقة الحقيقية. لو اخترت غلط، التصميم هيبقى هش وهيعمل مشاكل بعدين.

ببساطة:

- **Aggregation** = علاقة قوية "جزء من" (لو حاجة باظت بتأثر على الكيان كله).
- **Association** = علاقة أضعف "تعاون أو ارتباط" (أكتر مرونة، الكيانات تقدر تعيش وتشتغل مستقلين).

لأن الكيانات مستقلة، ده بيرجعنا تاني لمبدأ مهم: **تقليل الاعتماديات low coupling** بين المكونات بقدر الإمكان، وفصل الـ Domains. كل ما الأجزاء كانت أكثر استقلالية، كل ما كان النظام أكثر مرونة واستقرار.

الكتاب رجع هنا لمثال الـ **TV/VCR المدمج** مقابل **النظام المكوّن من أجزاء منفصلة**:

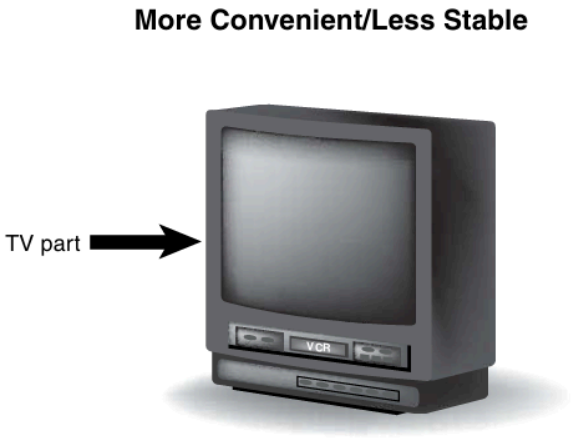


Figure 9.6 Convenience versus stability.

- المدمج أسهل في الاستخدام وأرخص كبداية.
- لكن النظام المكوّن من أجزاء مستقلة أكثر **قابلية للصيانة والتطوير**: لو جزء تعطل، الباقي يكمّل عادي، ولو عايز تحدّث جزء، تقدر تعمله بسهولة.

الموازنة هنا بين:

- **الراحة وسهولة الاستخدام**.
- **الاستقرار وقابلية الصيانة على المدى الطويل**.

مافيش حل واحد صحيح دايمًا، الاختيار بيعتمد على **الأولويات** وطبيعة التطبيق.

بعد ما خلص الحديث عن أنواع العلاقات، الكتاب بدأ يشرح جزئية مهمة جوه العلاقة نفسها وهي: **الكاردينالية (Cardinality)**.

الكاردينالية ببساطة هي **عدد الارتباطات بين الكائنات**. بنسأل:

1. مين بيتفاعل مع مين؟ (Class Bg Class A).
2. كل كائن من A بيرتبط بكام كائن من B؟
   - واحد بس؟
   - واحد أو أكتر؟
   - ممكن صفر أو واحد؟
3. هل الارتباط **إجباري** ولا **اختياري**؟

مثال من الكتاب: علاقة **الموظف (Employee)** مع **القسم (Division)**.

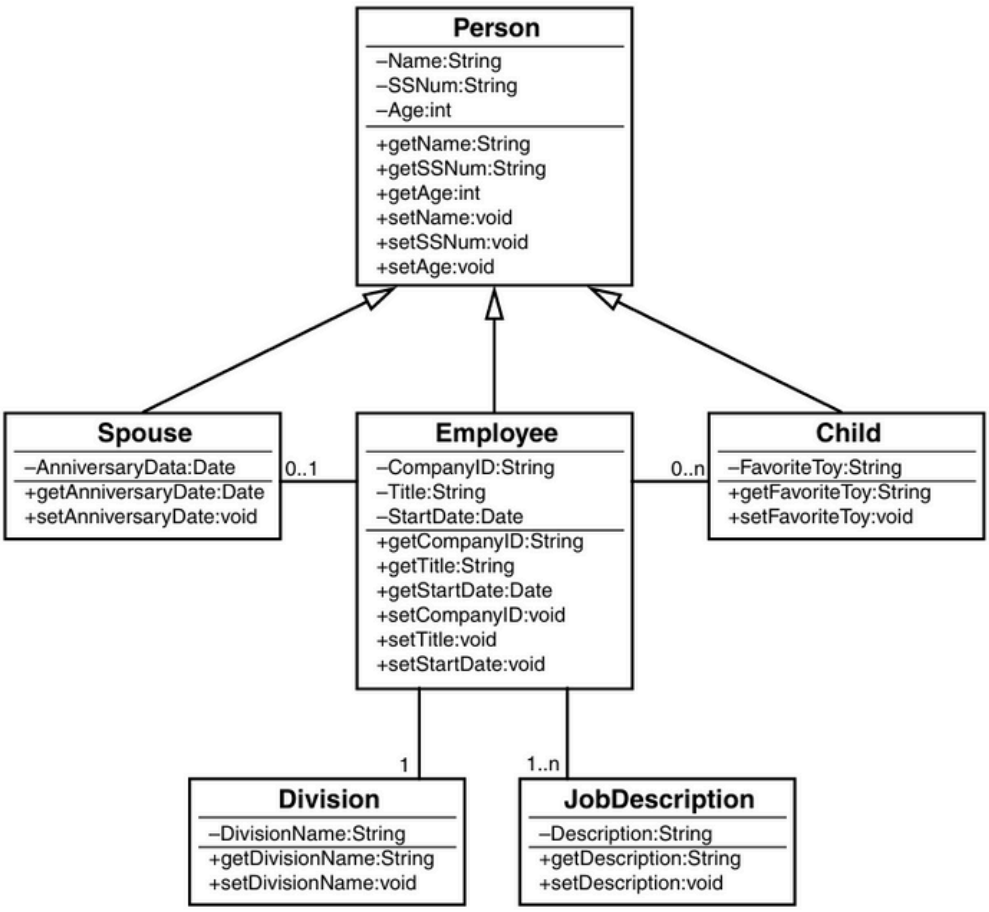
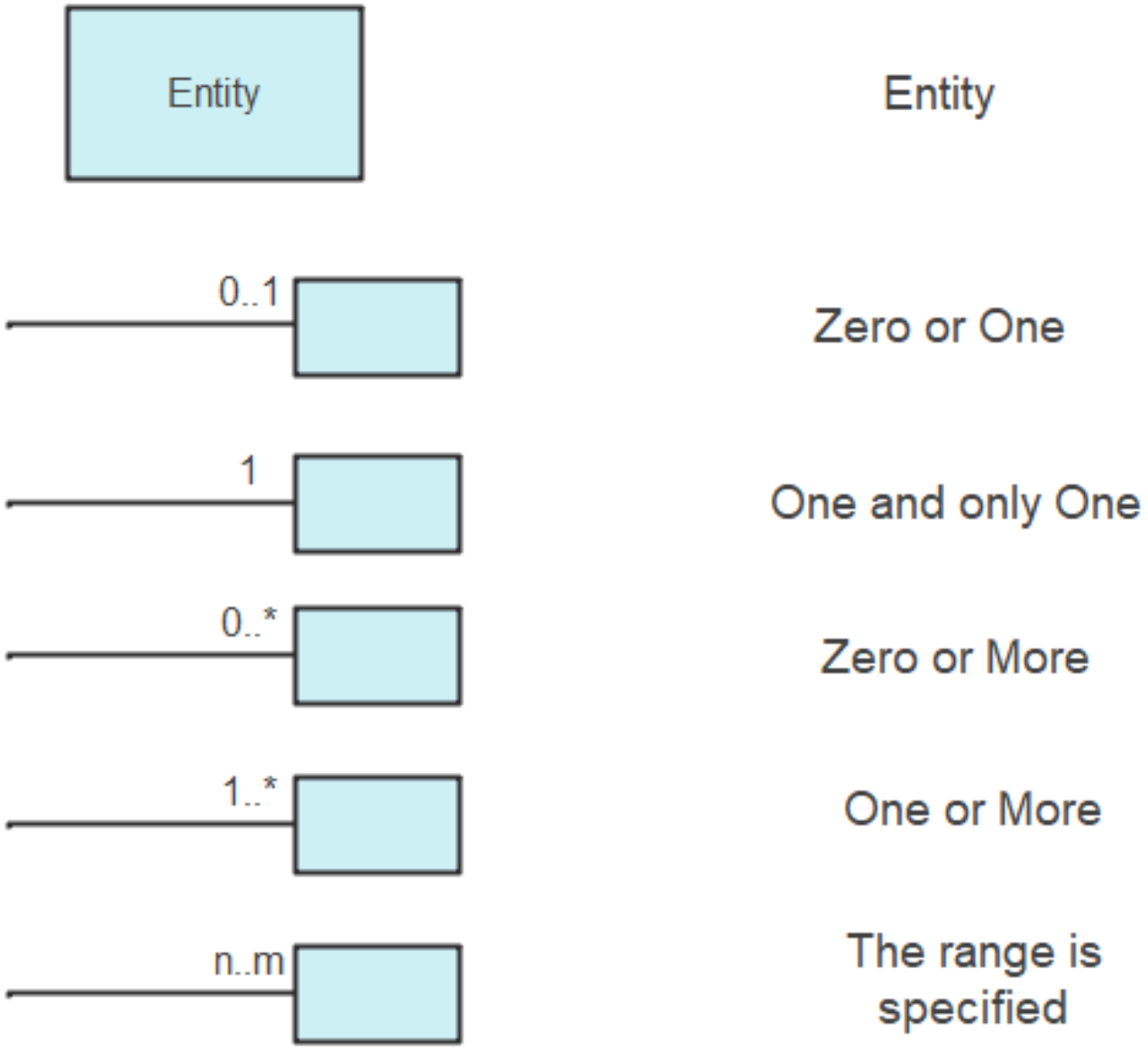


Figure 9.7 Cardinality in a UML diagram.

- كل موظف لازم يشتغل في قسم واحد بالضبط (واحد وإجباري).
- نفس الموظف لازم يكون عنده وصف وظيفي (Job Description) واحد على الأقل، لكن ممكن يكون عنده أكتر من وصف (واحد أو أكتر).

Cardinality في UML بتتكتب بالرموز:



أمثلة:

- **الموظف (Employee) ↔ الزوج/الزوجة (Spouse):**
  - موظف ممكن يكون متزوج (Spouse واحد) أو غير متزوج (ولا واحد).
  - الكارديناليتي هنا **1..0** → اختيارية.
- **الموظف (Employee) ↔ الأبناء (Children):**
  - الموظف ممكن يكون عنده أولاد (واحد أو أكتر) أو مفيش أولاد خالص.
  - الكارديناليتي هنا **n..0** → اختيارية.
- **الموظف (Employee) ↔ القسم (Division):**
  - كل موظف لازم ينتمي لقسم واحد بالضبط.
  - الكارديناليتي هنا **1** → إجبارية.

في الكود:

- العلاقات اللي فيها اكثر من object زي **n..1** أو **n..0** غالبًا بنمثّلها بـ **List / Collection**.
  - مثلًا: List<Child> children
  - لو مفيش أطفال → الليست تبقى فاضية.
  - لو فيه 1 أو 5 أطفال → الليست بتحتويهم.

- العلاقات **1..0** بتمثل كائن ممكن يكون **null** (مش موجود).
  - مثلًا: Spouse? spouse;
  - لو الموظف متزوج → فيه كائن Spouse.
  - لو مش متزوج → القيمة null.

النقطة الخطيرة (اللي الكتاب نبه عليها):

العلاقات الاختيارية (1..0 أو 0..*) لازم نتعامل معاها بحذر جدًا.

- لو الكود افترض إن العلاقة موجودة وحاول يستخدمها من غير ما يتأكد، يحصل **null pointer exception**
- ده خطأ شائع جدًا في البرمجة.



Figure 9.8 Checking all optional associations.

الحل:

- لازم نعمل **check** قبل ما نستخدم أي كائن جاي من علاقة اختيارية.
- مثال:

```
if (employee.spouse != null) {

  print(employee.spouse.name);

}
```

- بدل ما نفترض دايمًا إنه موجود.

المثال الختامي: الـ Dog

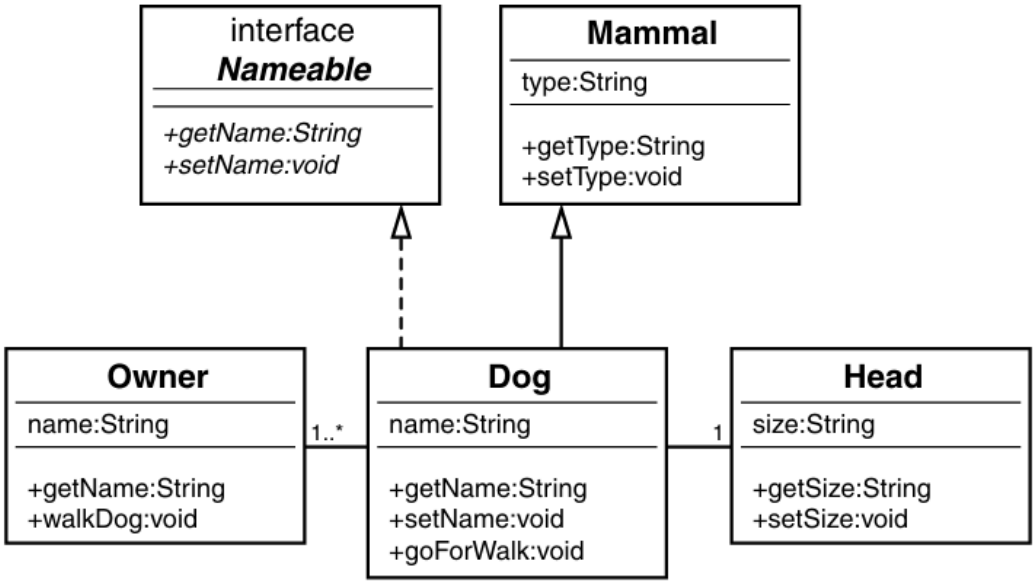


Figure 9.9 A UML diagram for the Dog example.

- **Head ↔ Dog (الرأس):**
  - علاقة **جزء من قوية** (يبقى **Aggregation** أو **Composition** بالمعنى الضيق).
  - الرأس جزء أساسي من الكلب.
  - **Cardinality:** واحد فقط (**1**).
- **Owner ↔ Dog (المالك):**
  - علاقة **Association (ارتباط/تعاون)** علشان الكلب مش جزء من المالك.
  - الكلب يحتاج خدمات من المالك (أكل، تمشية).
  - **Cardinality:** واحد أو أكثر (**1..***).
  - الكتاب افترض إنه لازم يكون فيه مالك واحد على الأقل.

الخلاصة العامة للفصل:

1. **فهم العلاقات** بين الكيانات:
   - **Aggregation / Composition (جزء من)**: علاقة احتواء قوية.
   - **Association (ارتباط/تعاون):** تعاون بين كيانات مستقلة.
2. **أفكار هيربرت سايمون** بتوضح ليه النظام الهيراركي (المركّب من أجزاء مستقلة نسبيًا **Composition**) بيؤدي إلى **أنظمة مستقرة وقابلة للصيانة**.
3. **الكاردينالیتي (Cardinality):**
   - بتحدد العدد (1 / 1..* / 0..1 / 0..*).
   - بتحدد كمان إذا العلاقة اختيارية ولا إجبارية.
4. **التعامل مع العلاقات الاختيارية:**
   - لازم نستخدم **برمجة دفاعية** (Defensive Programming).
   - ما ينفعش نفترض وجود البيانات (مثال: سباوس ممكن يكون null).
5. **الأثر على التصميم:**

- اختيار نوع العلاقة (جزء من Vs ارتباط) بيأثر على:
  - **المرونة (Flexibility).**
  - **قابلية الصيانة (Maintainability).**
  - **قدرة النظام على التكيف مع التغيير (Adaptability).**
  - **درجة التعقيد الكلي (Complexity).**
- مفيش إجابة واحدة صح، **السياق والمتطلبات** هما اللي بيحددوا الأنسب.

بس كده سلام.

الفصل العاشر Design Patterns:

النهاردة هنكمل كلامنا وندخل في موضوع جديد
وهو الفصل العاشر من الكتاب. الفصل ده بقى بيركز على حاجة اسمها **Design Patterns** أو أنماط التصميم.
هنحاول نفهم مع بعض إيه هي ونبسطها،
خصوصا للي لسه بيبدأ في عالم الـ OOP.

و ال **Design Patterns** ممكن نعتبرها كده زي خلاصة تجارب وخبرات المبرمجين اللي قبلنا،
يعني حلول لمشاكل بتقابلنا كتير وإحنا بنصمم الـ Software.

بالضبط.
يعني بدل ما كل واحد يقعد يخترِع العجلة من الأول تاني.
تماما.
الفكرة إن فهمها بيساعدنا نكتب كود منظم أكتر،
نقدر نعيد استخدامه بسهولة،
وطبعا الأهم يبقى سهل في الصيانة بعد كده.

طيب خلينا من الأول خالص،
يعني إيه أصلا **Design Patterns** ؟ المصطلح ده لما نسمعه المفروض ييجي في دماغنا إيه؟
هل هو كود جاهز كده بننسخه وخلاص؟
لا لا، هو مش كود جاهز خالص.

بُص يا سيد، الفكرة أعمق شوية.
نمط التصميم ده عبارة عن وصف لحل قياسي،
زي Template كده لمشكلة تصميم معينة بتتكرر كتير.
شبه مثلا المبادئ بتاعة الهندسة المعمارية،
يعني تصميم معين للعمود عشان يشيل وزن بطريقة معينة.
المهندس مش بياخد كود العمود.
لأ.

هو بياخد الفكرة أو النمط ويطبقه.
آه فهمتك.

يعني زي مبدأ أو Approach للحل مش الحل نفسه بكود معين.

والكتاب اللي دايما بيجي ذكره مع الموضوع ده هو بتاع الـ Gang of Four (GoF) صح؟
مظبوط. كتاب *Design Patterns (Elements of Reusable Object-Oriented Software)*
اللي اتشهر باسم كتاب الجانج أوف فور،
هو اللي رسخ المصطلح ده بشكل كبير.

وهما قالوا إن أي نمط تصميم له أربع مكونات أساسية بتوصفه.
حلو قوي.
إيه هي الأربع مكونات دي؟ خلينا نمشي فيهم واحدة واحدة.

> **The Four Elements of a Pattern**
>
> The GoF describe a pattern as having four essential elements:
>
> - The *pattern name* is a handle we can use to describe a design problem, its solutions, and consequences in a word or two. Naming a pattern immediately increases our design vocabulary. It lets us design at a higher level of abstraction. Having a vocabulary for patterns lets us talk about them with our colleagues, in our documentation, and even to ourselves. It makes it easier to think about designs and to communicate them and their trade-off to others. Finding good names has been one of the hardest parts of developing our catalog.
> - The *problem* describes when to apply the pattern. It explains the problem and its content. It might describe specific design problems, such as how to represent algorithms as objects. It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
> - The *solution* describes the elements that make up the design, their relationships, responsibilities, and collaborations. The solution doesn't describe a particular concrete design or implementation, because a pattern is like a template that can be applied in many situations. Instead, the pattern provides an abstract description of a design problem, and how a general arrangement of elements (classes and objects in our case) solves it.
> - The *consequences* are the results and trade-offs of applying the pattern. Although consequences are often unvoiced, when we describe design decisions, they are critical for evaluating design alternatives and for understanding the costs and benefits of the applying pattern. The consequences for software often concern space and time trade-offs. They might address language and implementation issues as well. Because reuse is often a factor in object-oriented design, the consequences of a pattern include its impact on a system's flexibility, extensibility, or portability. Listing the consequences explicitly helps you understand and evaluate them.

أول حاجة: الاسم (*Pattern Name*).
لازم يكون اسم مميز ومختصر عشان يبقى سهل المبرمجين يتكلموا به مع بعض.
زي ما نقول *Factory Pattern* أو غيره.
مهم عشان التواصل، صح؟ يعني بدل ما أقعد أشرح فكرة معقدة لزميلي،
أقول له: "هنستخدم Factory Pattern هنا"، وهو يفهم قصدي على طول.
بالضبط. بيوفر وقت وبيقلل سوء الفهم.

تاني حاجة: المشكلة (*Problem*).
يعني بنوصف بالضبط الموقف أو المشكلة اللي النمط ده بيحلها.
إمتى نستخدمه؟ إيه الظروف اللي بتخليه حل مناسب؟
تمام. عرفنا الاسم والمشكلة.

اللي بعده: تيجي بقى للحل (*Solution*)، وده أهم حتة.
الحل هنا مش كود بلغة برمجة معينة،
لأ. ده وصف مجرد للكلاسات، والأوبجكتس، والعلاقات اللي بتحل المشكلة دي.
يعني بيشرح إزاي النمط ده بيشتغل كهيكل عام.
يعني زي *Blueprint* كده بيديني الهيكل العام، وأنا أطبقه بالكود بتاعي.
تماما.

رابع وآخر حاجة، ودي مهمة قوي،
هي النتائج والعواقب (*Consequences*).
يعني إيه المزايا اللي هاخدها لما أطبق النمط ده؟
وإيه العيوب أو التضحيات اللي ممكن أقدمها؟

زي إيه مثلا؟
يعني مثلا هل النمط ده هيزود مرونة السيستم؟
هيخليه أسهل نكبره بعدين؟ طيب، هل هيأثر على الأداء؟
هيستهلك ميموري أكتر؟ هيصعب الكود شوية؟
لازم نفهم trade-offs دي كويس.

النقطة دي خطيرة فعلا،
عشان ما فيش حل ببلاش. يعني كل حاجة لها تمن.

طيب فهمنا كده إيه هي الأنماط ومكوناتها ؟ نرجع للسؤال المهم: ليه أستخدمها أصلا؟
إيه الفايدة العملية لي كمبرمج؟

الفوايد كتيرة وملموسة.
أولا، زي ما قلنا: بتوفر وقت ومجهود.
حلول مجربة وفعالة لمشاكل بتتكرر.
مش هنقعد نخترع العجلة.

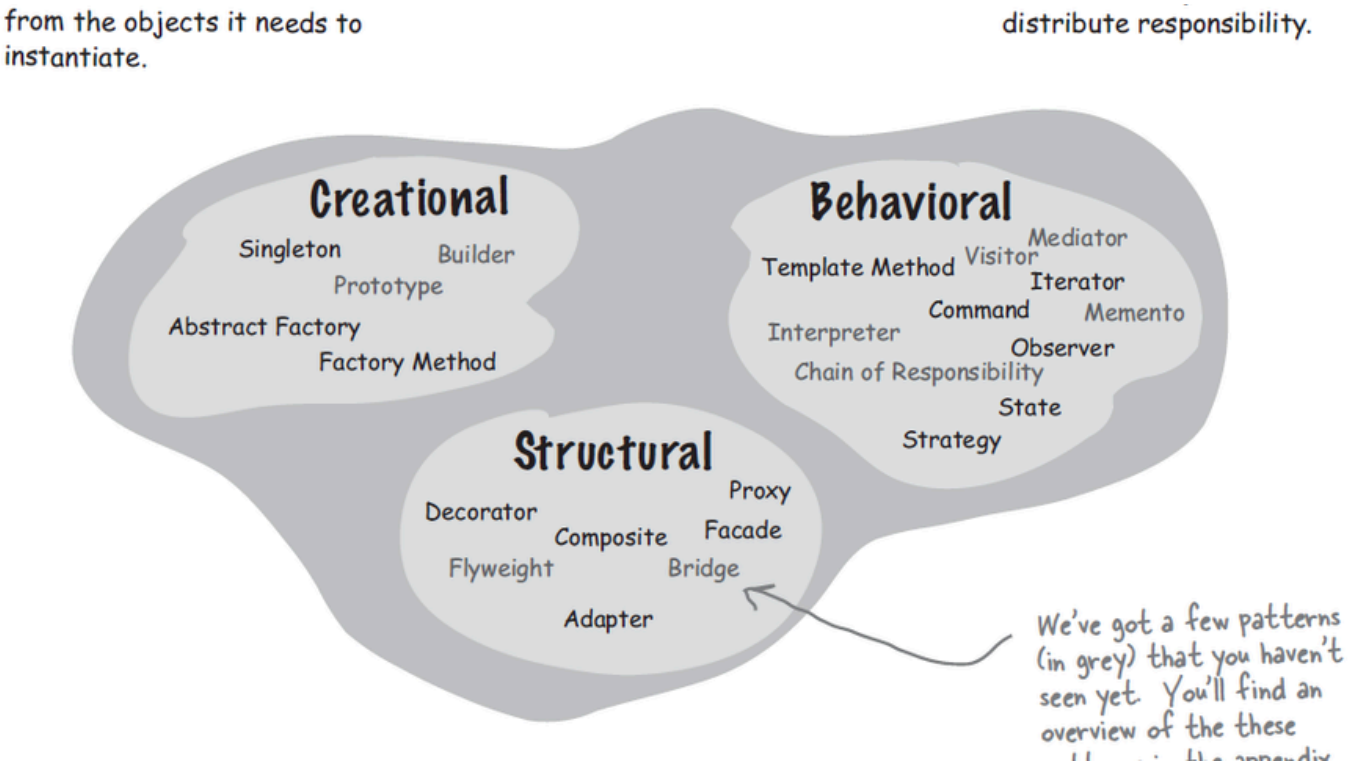
ثانيا: بتحسن التواصل جدا بين الفريق.
لما كلنا نبقى فاهمين يعني إيه *Singleton* أو *Observer*، بنتكلم نفس اللغة.
وده بيخلي الشغل أسرع وأدق.

وده أكيد بيظهر في الكود نفسه.
يعني الكود اللي فيه *Design Patterns* معروفة،
بيبقى أسهل في القراءة والفهم والصيانة،
حتى لو حد جديد جه الفريق.

بالظبط.

بيقدر يتعرف على الـ Pattern ويفهم الحتة دي بتعمل إيه بسرعة.
وده بيساعدنا نصمم ونفكر على مستوى أعلى من التجريد Abstraction.
تمام.

طيب، الكتاب بتاع الـ GoF بيقسم الأنماط دي لتلات فئات رئيسية عشان يسهل فهمها.
ممكن نعدي عليهم بسرعة؟
أكيد، التقسيمة دي بتوضح الهدف من كل مجموعة.



أول نوع: الأنماط الإنشائية (*Creational Patterns*).
دي اللي لها علاقة بإزاي بننشئ الـ Objects.
هدفها تخلي عملية الإنشاء دي مرنة أكتر وتفصل الكود اللي بيستخدم الـ Objects عن تفاصيل إنشاؤه.
يعني بتدينا طرق ذكية نصنع بيها الـ Objects.

بالظبط.

تاني نوع: الأنماط الهيكلية (*Structural Patterns*).
دي بتهتم بإزاي نركب الكلاسات مع بعض عشان نعمل هياكل أكبر ومنظمة أكتر،
زي تركيب الليجو كده.

تمام.

والتالتة: الأنماط السلوكية (*Behavioral Patterns*).
دي بقى تركز على إزاي الـ Objects بتتفاعل مع بعض،
إزاي بيتكلموا، وإزاي بنوزع المسؤوليات بينهم.
يعني بتهتم بالتفاعلات اللي بتحصل وقت التشغيل.

طيب عشان نفهم أكتر،
خلينا ناخد أمثلة عملية من الكتاب لكل نوع.

نبدأ بنمط إنشائي مشهور: *Factory Method*
إيه قصته ده؟
ده بيحل مشكلة شائعة جدا.

تخيلي عندك كود رئيسي محتاج يعمل Objects من أنواع مختلفة،
زي برنامج رسامي بيرسم دواير ومربعات ومثلثات.
إزاي نخليه يطلب شكل معين من غير ما الكود الرئيسي ده يبقى عارف اسم الكلاس المحدد بتاع الدايرة أو المربع؟

عايزين نعمل *Decoupling* أو فك ارتباط،
عشان لو حبيت أزود شكل جديد زي مستطيل مثلا،
مش عايزة أرجع أعدل في الكود الرئيسي اللي بيرسم.

المثال بتاع Shape في الكتاب بيوضح ده إزاي؟
أيوة، هو سيناريو بسيط وواضح.

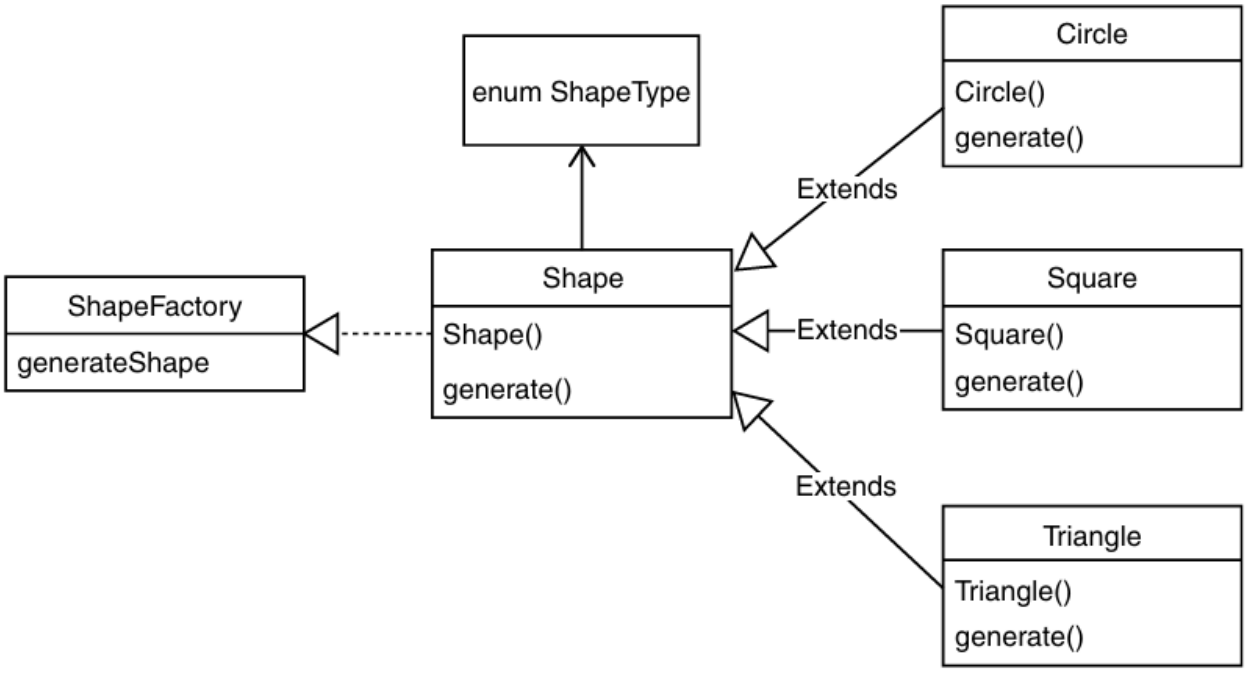


Figure 10.2 Creating a factory for the Shape class.

أول حاجة: ممكن يكون عندنا *enum ShapeType* بأنواع الأشكال.
بعدين نعمل *Abstract Class* أو *Interface* اسمه Shape،
ده بيعرف العمليات المشتركة زي مثلا ()Shape
دي بتكون زي *Abstract method*، يعني بنقول: كل شكل لازم يعرف يرسم نفسه،
بس نسيب التفاصيل لكل شكل.

بالظبط. بعدين نعمل الكلاسات بقى بتاعة كل شكل،
كل واحد بيعمل Implement لـ Shape،
وبيدي الكود الخاص بيه.

هنا ييجي دور الـ Factory.
نعمل كلاس اسمه ShapeFactory.
الكلاس ده فيه ميثود نسميها generateShape،
دي بتاخد نوع الشكل اللي عايزينه.
جواها بقى بيبقى فيه Logic بسيط يقرر بناءً على النوع ده يعمل new لأنهي كلاس ويرجع الـ Object بتاعه.

```
class ShapeFactory {
    public static Shape generateShape(ShapeType sType) {
        Shape shape = null;
        switch (sType) {

        case CIRCLE:
            shape = new Circle();
            break;

        case SQUARE:
            shape = new Square();
            break;

        case TRIANGLE:
            shape = new Triangle();
            break;

        default:
            // throw an exception
            break;
        }
        return shape;
    }
}
```

آه، يعني الكود الرئيسي بتاعي بدل ما يقول new Circle()،
يقول: "هات Shape من الـ Factory" ... يعني مثلا
ShapeFactory.generateShape(ShapeType.CIRCLE)
والـ Factory هو اللي بيعمل الـ new دي وبيختار الكلاس الصح.

```
public class TestFactoryPattern {
    public static void main(String[] args) {

        Circle circle = new Circle();
        Square square = new Square();
        Triangle triangle = new Triangle();

    }
}
```

However, properly using the factory requires that the programmer use the ShapeFactory class to obtain any Shape object:

```
public class TestFactoryPattern {
    public static void main(String[] args) {

        ShapeFactory.generateShape(ShapeType.CIRCLE);
        ShapeFactory.generateShape(ShapeType.SQUARE);
        ShapeFactory.generateShape(ShapeType.TRIANGLE);

    }
}
```

وده بيستخدم كمان مع *Polymorphism*، صح؟
عشان الكود الرئيسي بيتعامل مع اللي راجع على إنه Shape وخلاص.

ممتاز.

بالظبط كده.
الكود الرئيسي ما بيشوفش خالص كلاسات Circle ولا Square.
هو بيتعامل مع الـ Factory والـ Interface Shape.

لو حبينا نضيف Rectangle، هنعمل تعديل بسيط في الـ Factory.
الكود الرئيسي مش هيتغير خالص.
دي قوة الـ Factory.

طيب، ده بالنسبة Creational Patterns.
نيجي بقى Structural Patterns.

لو عندنا كلاسات موجودة بس الـ Interfaces بتاعتها مش متوافقة،
إزاي نخليهم يشتغلوا مع بعض؟

هنا بيجي دور باترن شبه الـ Adapter.

تشبيهه المشهور هو الأبليكة بتاعة الكهربا أو محول الفيشة.



عندك فيشة شكلها مختلف عن الجهاز،
فبتستخدمي Adapter في النص عشان يوصلهم ببعض.

في البرمجة Adapter Patterns بيعمل نفس الحاجة:
بيخلي Classes لها واجهات مختلفة تقدر تشتغل سوا.
يعني زي مترجم بينهم.

مثال الـ MailTool في الكتاب ممكن يوضح أكتر؟
أكيد.

```
package MailTool;
public class MailTool {
   public MailTool () {
   }
   public int retrieveMail() {

    System.out.println ("You've Got Mail");

         return 0;
   }
}
```

تخيل عندك كلاس قديم اسمه MailTool،
فيه ميثود اسمها retrieveMail().
وانت في النظام الجديد عايز تستخدم Interface تانية،
اسمها MailInterface،
وفيها ميثود اسمها getEmails().

الأسماء والطريقة مختلفة.
يعني الكود الجديد عايز ينادي getEmails()،

بس الكود القديم هو اللي بيعمل الشغلانة فعلا في retrieveMail().
مش هيركبوا على بعض.

```
package MailTool;
interface MailInterface {
   int getMail();
}
```

You can now create your own mail tool that wraps the original tool and provide your own interface:

```
package MailTool;
class MyMailTool implements MailInterface {
```



```
    private MailTool yourMailTool;
    public MyMailTool () {
     yourMailTool= new MailTool();
         setYourMailTool(yourMailTool);
    }
    public int getMail() {
       return getYourMailTool().retrieveMail();
    }
    public MailTool getYourMailTool() {
       return yourMailTool ;
    }
    public void setYourMailTool(MailTool newYourMailTool) {
       yourMailTool = newYourMailTool;
    }
}
```

```
package MailTool;
public class Adapter
{
  public static void main(String[] args)
  {
    MyMailTool myMailTool = new MyMailTool();

    myMailTool.getMail();

  }
}
```

بالظبط هنا بنعمل كلاس جديد نسميه MyMailTool.
ده أولا بيعمل Implement للـ Interface الجديدة MailInterface،
يعني لازم يكون فيه ميثود getEmails().
ثانيا: جواه بيكون عنده Object من الكلاس القديم MailTool.

لما الكود الجديد ينادي على getEmails()،
كل اللي بيعمله الـ Adapter إنه جوه الميثود دي ينادي retrieveMail()
على الـ Object القديم اللي جواه.

يعني هو بيستقبل الطلب بالشكل الجديد،
ويترجمه للشكل القديم،
ويوصله للكلاس الأصلي.

كده الكود الجديد استفاد من القديم من غير ما يعرف تفاصيله.

وده شبه فكرة الـ Wrapper اللي اتكلمنا عليها قبل كده.
كأننا بنغلف الكلاس القديم بواجهة جديدة.

بالظبط.

الـ Adapter هو تطبيق للـ Wrapper،
هدفه التوفيق بين الـ Interfaces.

ممتاز.

كده شفنا مثال Creational Patterns و Structural Patterns،
فاضل Behavioral Patterns.

الكتاب بيتكلم عن الـ **Iterator**.
ده بيحل مشكلة بنقابلها كتير جدا:
إزاي نعدي على عناصر زي Array أو List أو أي حاوية بيانات،

واحد ورا التاني بطريقة موحدة،
من غير ما نعرف الحاوية دي متخزنة إزاي من جوه.

يعني عايز طريقة واحدة أقدر بيها أمشي على عناصر أي نوع Collection،
سواء كانت Array أو LinkedList أو غيره.

تماما.

الكتاب بيستخدم مثال قديم شوية في Java زي Vector.

```
package Iterator;

import java.util.*;
public class Iterator {
    public static void main(String args[]) {

        // Instantiate an ArrayList.
        ArrayList<String> names = new ArrayList();

        // Add values to the ArrayList
        names.add(new String("Joe"));
        names.add(new String("Mary"));
        names.add(new String("Bob"));
        names.add(new String("Sue"));

        //Now Iterate through the names
        System.out.println("Names:");
        iterate(names);
    }

    private static void iterate(ArrayList<String> arl) {
        for(String listItem : arl) {
                    System.out.println(listItem.toString());
                }
    }
}
```

الفكرة إن الـ Vector دي كانت بتوفر طريقتين أساسيتين:
تسأل "لسه في عناصر؟" زي hasMoreElements()،
وتقولي "هات العنصر اللي بعده" زي nextElement().

يعني الـ Iterator بيخفي تماما تفاصيل التخزين والترتيب جوه الـ Collection دي.
أنا كمستخدم كل اللي يهمني إني أقدر أطلب العناصر واحد ورا التاني.

بالظبط.
وده بيدي ميزتين:

- بساطة الاستخدام بواجهة موحدة.
- وفصل الاهتمامات.

يعني الكود اللي بيستخدم الـ Iterator مش مهتم خالص بطريقة التخزين.
ممكن نغير الـ Collection من Array لـ LinkedList،
والكود اللي بيستخدم الـ Iterator مش هيتغير.

وده طبعا بنشوفه في كل اللغات دلوقتي.
مثلا الـ for-each loop بتستخدم Iterator من ورا الكواليس، صح؟
فعلا.
الـ Iterator بقى أساسي في التعامل مع الـ Collections.

طيب، قربنا نخلص كلامنا عن الأنماط،
بس الكتاب ذكر حاجة عكسها ومهمة برضه.

Antipatterns إيه دي بقى؟
دي ببساطة الوجه الآخر للعملة.
لو الـ Patterns هي حلول كويسة ومجربة،
فالـ Anti-Patterns هي وصف لممارسات سيئة أو حلول شائعة،
لكن بتسبب مشاكل.

بندرسها عشان نتعلم من الأخطاء دي ونتجنبها.
يعني بدل ما نتعلم إيه الصح اللي نعمله،
بنتعلم إيه الغلط اللي ما نعملوش.

بالظبط.

زي ما في علامات تحذير، الكتاب ذكر مثال على كود اتعمل عشان يُعاد استخدامه لكن ما حدش استعمله فعلًا، يمكن لأنه معقد أو مش موثّق كويس. كمان بعض الأنماط زي الـSingleton لو استخدمناها غلط ممكن تتحول لـAnti-pattern.

مثال أبسط هو "الـMagic Strings" لما نكتب نفس النص (زي اسم ملف أو رسالة خطأ) في كذا مكان في الكود بدل ما نعرّفه كـ constant مرة واحدة.

المهم نعرف النوعين: **الأنماط الصحيحة** عشان نستخدمها صح، و**الـAnti-patterns** عشان نتجنبها.
بكده نقدر نقول إن أنماط التصميم أدوات قوية ومجرّبة بتساعدنا نحل مشاكل تصميم متكررة، وفهمها بيخلي الكود أنضف وأوضح وأسهل في الصيانة.

لكن لازم نفتكر إنها مش حلول سحرية لكل مشكلة. الأهم دايمًا إننا نفهم المشكلة نفسها كويس جدًا الأول، وبعدين نسأل:

- هل فعلًا فيه **pattern** مناسب لحل المشكلة دي؟
- هل الفوائد اللي هيضيفها تستاهل التعقيد اللي ممكن يجي معاه؟

القدرة على اتخاذ القرار الصح—إما استخدام **pattern** المناسب أو اختيار الحل البسيط المباشر—دي مهارة مهمة زي معرفة النمط نفسه. كمان الأنماط مش ثابتة، هي بتتطور مع الوقت.

في النهاية: التفكير النقدي واختيار الأداة المناسبة هو الأساس.

بس كده سلام.

الفصل الحادي عشر Avoiding Dependencies and Highly Coupled Classes :

النهاردة بنكمل رحلتنا سوا في الكتاب و هنركز على الفصل الحادي عشر.

لو فاكر يا سيد، إحنا قبل كده غطينا أساسيات البرمجة الكائنية وتصميم الكلاسات.

آه طبعًا.
وفي الفصل السابع تحديدًا كنا اتكلمنا عن الفرق بين Inheritance و Composition.

بالضبط.
ودول كانوا الوسائل الأساسية لإعادة استخدام الكود.
تمام أوي.

الفصل ده بقى بياخدنا خطوة لقدام شوية، ويدخل في حتة مهمة قوي، تحدي كبير بيقابل أي حد بيكتب كود: **إزاي نتعامل مع العلاقات بين الكلاسات؟**
وتحديدًا بقى مشكلة **dependencies and highly coupled classes**، اللي هو الترابط الزيادة بين الكلاسات.

آه، **Coupling** دي كلمة بتتكرر كتير.
جدا، عشان كده الفصل ده يعتبر تكملة مهمة جدًا للي قلناه قبل كده عن **Composition**.
جميل.

يبقى هدفنا النهاردة خلينا نقول إننا نستوعب التقنيات اللي الكتاب بيقولها زي **ال Dependency Injection** دي، علشان في الآخر نعرف نكتب كود مش بس شغال وخلاص
لأ طبعًا.
كود يكون مرن، نقدر نعدل فيه بسهولة، و نطور عليه بعدين من غير مشاكل كبيرة.

بالضبط كده.
يلا بينا نبدأ ونفصص الأفكار دي واحدة واحدة.

يلا بينا.
طيب، الفصل بيبدأ بنقطة مهمة، إنه بيرجع يقارن بينهم **Composition versus Inheritance**. ممكن حد يسأل: ليه بنرجع للنقطة دي تاني؟ يعني إيه لازمة المقارنة دي مرة تانية؟

سؤال كويس جدًا.
السبب إن الاختيار بين دول، بين **Inheritance** و **Composition**، له تأثير مباشر، يعني تأثير كبير قوي على مشكلة أساسية في أي سوفت وير، اللي هي الترابط **Coupling** .

الترابط **Coupling** .
آه، تخيلها كده كأن الكلاسات لازقة في بعض قوي.

ماسكين إيد بعض جامد.
لدرجة إنك لو حركت واحد تلاقي كله بيتهز معاه.
بيتسمع في كله.

بالضبط.

الكتاب بيستخدم كلام يوصف نتيجة الكوبلينج العالي ده زي إنه بيخلي الكود:

Rigit جامد يعني.

آه، جامد صعب تغييره.

و هش **Fragile**.
أي خبطة ممكن تكسره في حتة تانية خالص.
وإيه **Immobile**؟

ده اللي هو غير قابل للنقل.

أيوة، صعب تاخد حتة كود منه وتستخدمها في حتة تانية.
كأنه بيت الكوتشينة يعني، لو شديت كارت غلط كله يقع.



بالضبط كده.
هي دي المشكلة.

وده بيخلي الصيانة كابوس حقيقي.
يعني تغيير بسيط ممكن تطلبه في حتة، تلاقيه عمل مشاكل إنت مش متوقعها خالص في حتت ممكن تكون ما لهاش علاقة في الظاهر.

وكمان بتخسر ميزة مهمة قوي اللي هي إعادة الاستخدام.
الكود بيبقى متكتف كده.
مش عارف تستفيد بيه في مشاريع تانية، أو حتى في حتة تانية في نفس المشروع.

واضح أوي.
يبقى الفصل بيشوف إزاي Inheritance من ناحية، ومن ناحية تانية Composition، كل واحد فيهم بيتعامل إزاي مع مشكلة **Coupling** دي، أو يمكن بيساهم فيها.

تمام.
خلينا بس نراجع بسرعة
زي ما قلنا قبل كده، خصوصًا في الفصل الأول والسابع لو فاكر

Inheritance بيمثل علاقة **"is a".**
هو نوع من ...

بالضبط.
زي ما نقول: Dog هو نوع من Mammal.
دي طريقة كويسة أوي عشان نعيد استخدام الخصائص اللي في الـSuperclass.

تمام.
أما الـComposition بقى، ده بيمثل علاقة **"has a".**
يحتوي على.

زي العربية فيها موتور، أو الـOrder دي فيها Items.
هنا الكلاس بيبقى جواه من كلاسات تانية عشان يستخدم اللي فيها.
دول الطريقتين الأساسيتين اللي بنبني بيهم علاقات ونعيد استخدام الكود.

بس الفصل ده، زي ما فهمت منك، مش بيقدّمهم على إنهم حلول مثالية وخلاص.

- لأ، ده بيبدأ يوضح إن في مشاكل، ويمكن المشاكل دي بتظهر أكتر مع الـInheritance.

هنا بقى بحس الموضوع بيبدأ يبقى فيه تفاصيل مهمة.

- عندك حق مية في المية.
  الـInheritance آه، هو قوي وكل حاجة، بس ممكن يعمل كوبلينج عنيف جدًا.
- وإحنا لمسنا الحتة دي في الفصل السابع.
- آه، قلنا إنه بيكسر ال Encapsulation شوية.
- بالضبط، لإنه بيبقى عارف تفاصيل كتير عن Impl جوه ال parent.

المشكلة الأكبر، واللي الفصل ده بيركز عليها جامد، إنك لما تورّث مباشرة من الكود الفعلي بتاع الـMethod.

- أيوة.
- بالضبط، أي تغيير في الـParent بيسمع على طول وبشكل مباشر في كل اللي واخدين منه الوراثة دي.

لو الأب الـParent Class غير رأيه في طريقة عمل حاجة، كل ال Child لازم يغيروا طريقتهم هم كمان، حتى لو ده مش مناسب لهم.

- بالضبط.
  وهو ده اللي ممكن يعمل مشاكل مش متوقعة وسلوك غريب جدًا محدش كان عامل حسابه.

تخيل عندك مثلًا عشرة كلاسات وارثين من واحد، وغيرت حاجة صغيرة في حتة صغيرة.

- آه، إنت محتاج بقى ترجع تختبر كل العشرة كلاسات دول تاني عشان تتأكد إن مافيش حاجة ضربت.

ده بيخلي التعديل والتطوير صعب أوي ومليان مخاطر.

- بيصعّب جدًا موضوع الـMaintenance و ال Unit testing، أكيد طبعًا.
  لإن ما بقاش وحدة مستقلة بنفسها، لأ، ده بقى معتمد قوي على تفاصيل الـParent.

طيب، لو الوراثة فيها المشاكل دي، هل ده معناه إن الـComposition بقى البطل الخارق اللي ما فيهوش غلطة؟ الحل السحري ؟

- بصراحة بيقدم مرونة أعلى بكتير في معظم الحالات، بس مش دايمًا بيكون خالي تمامًا من الCoupling هو كمان، ده بيعتمد على طريقة استخدامه.

الفصل بيفرق بين نوعين أساسيين من الـComposition عشان يوضح الحتة دي.

- تمام.
- إيه هم النوعين دول؟

كنا اتكلمنا عنهم قبل كده بس هنراجع عليهم سريعا بس

النوع الأول هو الـAggregation (التجميع).
تخيّل جهاز Boombox بتاع زمان، اللي فيه كاسيت وراديو وسماعات كله في علبة واحدة.
الكلاس الأساسي هنا بيعتبر الـObjects التانية أجزاء منه.

هنا بقى ممكن يحصل كوبلينج لو الكلاس الأساسي هو نفسه اللي بيعمل الـInstantiation جواه، هو اللي بينشئها يعني.

هنا البوومبوكس بقى معتمد بشكل مباشر على وجود كلاسات اسمها Radio وSpeakers مثلا.

- تمام.

والنوع التاني هو الـAssociation (الربط).
ده بقى أشبه بمكونات الاستيريو الحديثة، كل قطعة لوحدها.

الأمبليفاير، الـCD، السماعات، وكلهم بيتوصلوا بأسلاك.
هنا بتبقى مستقلة أكتر، الكلاس الرئيسي بس بيربط بينهم أو بيستخدمهم.
الفصل بيستخدم مثال الاستيريو ده عشان يبين إزاي الـAssociation بيديكِ مرونة أعلى بكثير.

- آه أيوة، لإنك ممكن تغيّر قطعة بقطعة تانية، تغيّر السماعات مثلًا من غير ما تأثر على الباقي.
  على عكس البوومبوكس اللي أي تغيير فيه أصعب، فيه كوبلينج أعلى.

طيب، الفصل بيستخدم أمثلة عشان يطبّق الأفكار دي، زي المثال بتاع الثدييات والكلاب والحيتان اللي قلت عليه.

- ممكن نمشي مع الأمثلة دي خطوة بخطوة؟ عشان نفهم المشكلة والحل بشكل عملي أكتر.

طبعًا، الأمثلة دي هي فعلًا لب الموضوع.

المثال الأولاني بيوضح مشاكل استخدام الـInheritance لو حبينا نضيف سلوكيات جديدة لتسلسل هرمي زي بتاع الـMammals.

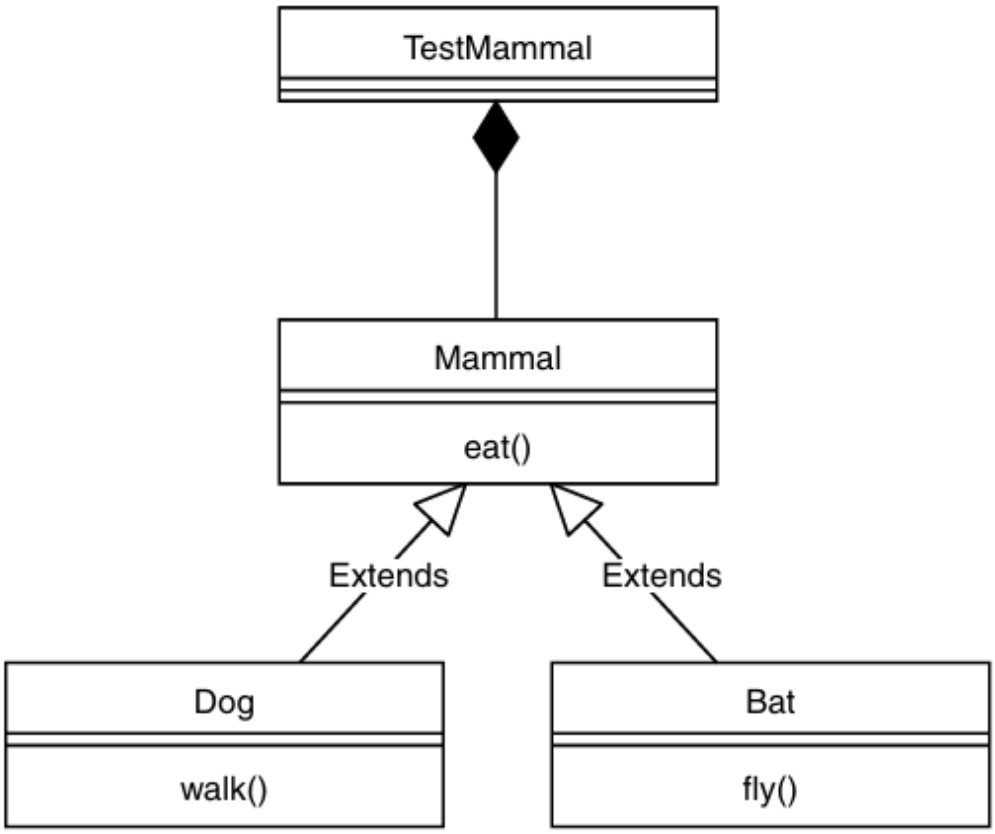


Figure 11.1 Using inheritance to create mammals.

يعني تخيل عندنا كلاس Mammal، وكلاسات زي Dog وWhaleg وارثين منه.

- لحد كده تمام.
- آه كله زي الفل.

طيب لو حبينا نضيف سلوك زي السباحة أو الطيران.
لو حطينا swim() في الـMammal، هيبقى منطقي للحوت (Whale)، بس أكيد مش منطقي للكلب.
ولو ضيفنا fly()، ممكن يبقى كويس لو عملنا Bat خفاش، بس برضه مش منطقي لا للكلب ولا للحوت.

محاولة تعديل الـHierarchy عشان تدخل السلوكيات دي بتدخلنا في حتت معقدة ومش منطقية خالص، وتكسر مبدأ مهم زي **الـLiskov Substitution Principle** اللي هو بيقول إن الابن لازم يعرف يعمل كل اللي الأب بيعمله.

- بالضبط.

المفروض أقدر أستخدم Object من ال Child مكان الـParent من غير ما أبوظ الدنيا.
لو الـMammal بقى بيطير، مش هقدر أحطه Dog مكانه في كل الحالات مثلا.

- تمام.

عشان يحل المشكلة دي، المثال التاني بيجرب يستخدم الـComposition، بدل ما نورّث السلوك، بنعمل كلاسات منفصلة للسلوكيات دي.

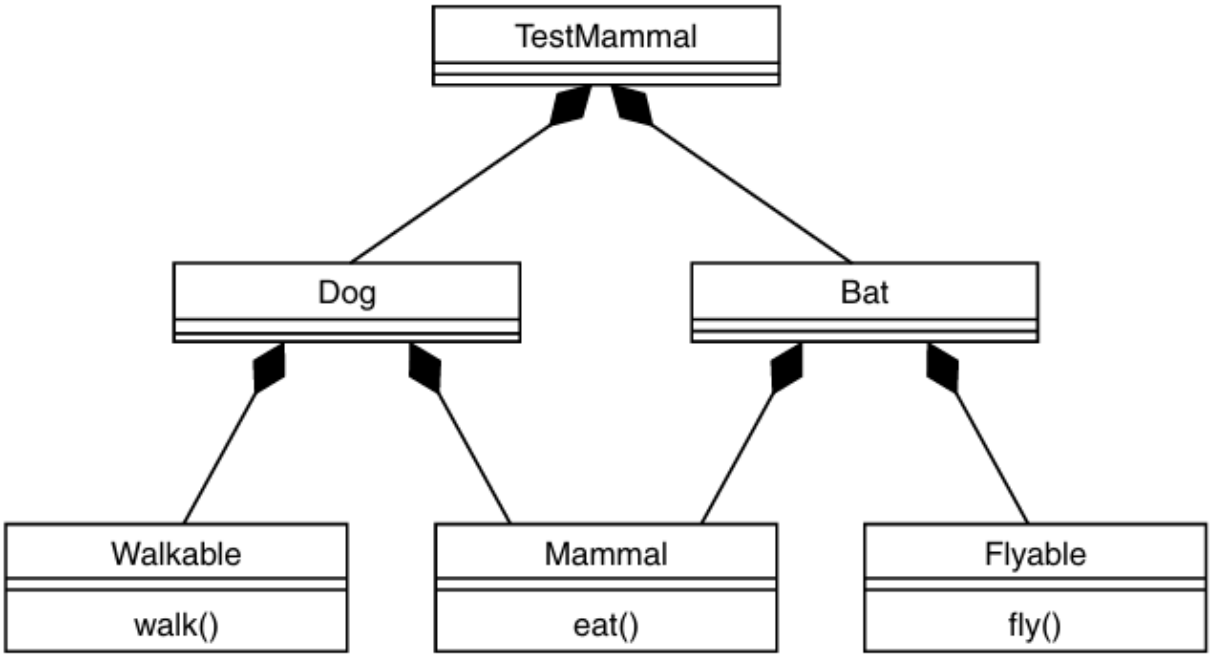


Figure 11.2 Using composition to create mammals.

زي كلاس Walkable (قابل للمشي) وكلاس Flyable (قابل للطيران).

- منطقي أكتر.

وبعدين زي Dog، يبقى جواه Object من نوع Walkable، و كلاس Bat جواه Object من نوع Flyable.
ده طبعًا حل أحسن بكتير من ناحية المنطق والمرونة.

لكن، وهنا بقى النقطة: الفصل بيوضح إن الطريقة اللي بنعمل بيها ده جوه الكلاس الأساسي لسه ممكن تعمل كوبلينج.

إزاي؟ لو كود كلاس Dog مكتوب جواه كده:

_walkable = new Walkable();

هنا ال Dog لسه معتمد بشكل مباشر على إنه هو اللي ينشئ Object من الكلاس المحدد اللي اسمه Walkable.

- فهمتك، يعني حتى مع الـComposition، لو خلينا هو اللي بيعمل الـInstantiation جواه، لسه المشكلة موجودة.
- يمكن أقل شوية، بس موجودة.

لو حبينا نغير طريقة المشي دي أو نستخدم نوع تاني من Walkable، هنضطر نرجع نغير في كود الـDog نفسه.

- بالضبط.

طيب، إيه الحل بقى؟ اللي الفصل بيقدمه عشان نتخلص من ال Coupling ده حتى مع الـComposition؟
هنا بقى **بنوصل للقلب النابض بتاع الفصل ده كله**: **Dependency Injection** أو حقن الاعتمادية.

حقن الاعتمادية **Dependency Injection**.
الفكرة ببساطة شديدة: بدل ما الـObject زي Dog هو اللي يعمل الـDependencies بتاعته جواه...

- زي new Walkable() مثلًا.
  دي بتيجي له جاهزة من برّه، بتتحقن جواه.

وده بيودينا لمفهوم مهم مرتبط بيه قوي اسمه **Inversion of Control** أو عكس التحكم (IOC).

- الاسم لوحده مثيّر للاهتمام.
- ممكن توضح أكتر يعني إيه التحكم بيتعكس؟

معناها إن الكلاس زي Dog ما بقاش هو اللي متحكم في عملية إنشاء الـDependencies اللي هو بيعتمد عليها.
زمان كان هو اللي بيقول new Walkable().
دلوقتي لأ، التحكم ده اتعكس. المسؤولية دي اتشالت من عليه واتنقلت لحد تاني، حد برّه الكلاس، كيان خارجي يعني.

- بالظبط، كيان خارجي هو اللي مسؤول إنه يعمل الـWalkable Object، وبعدين يديه للـDog Object.
  كل اللي عليه إنه يستخدم اللي جاله.

مش مهم بقى هو اتعمل إزاي أو مين اللي عمله.
هو بس بيقول: يا جماعة، أنا محتاج حاجة بتعرف تعمل وظيفة المشي.
مين بقى اللي بيديهاله؟ وازاي؟ ده مش شغله هو.

- تغيير في طريقة التفكير نفسها.

طيب، عمليا بقى الحقن ده بيحصل إزاي؟ إزاي اللي برّه ده بيوصل الـDependencies للكلاس؟
الفصل بيشرح طريقتين أساسيتين للـDI:

الأولى: الحقن عن طريق الـConstructor.

- أيوة.
  دي الطريقة الشائعة، اللي الـObject ما ينفعش يشتغل من غيرها أصلًا، الإجباري يعني.

بدل ما يبقى فاضي أو يعمل new Walkable() جواه، لأ، بنخليه ياخد الـWalkable كـParameter.
يبقى حاجة زي كده:

Injection by Constructor

One way to *inject* the `Walkable` behavior is to create a constructor within the `Dog` class that, when invoked, will accept an argument from the main application as follows:

```
class Dog {
    Walkable walker;
    public Dog (Walkable w) {
    this.walker=w;
    }
}
```

In this approach, the application instantiates a `Walkable` object and *inserts* it into the `Dog` via the constructor.

```
Walkable walker = new Walkable();

Dog fido = new Dog(walker);
```

- آه، بياخدها جاهزة.
- بالضبط.

والكود اللي برّه بقى، اللي هو بينادي وبيعمل new Dog، هو اللي لازم الأول يعمل new Walkable، وبعدين يبعته للـDog وهو بيعمله.

- تمام.

والطريقة التانية: الحقن عن طريق Setter.

- تمام.
  دي بنستخدمها أكتر في الـDependencies اللي مش ضرورية قوي، الاختيارية، أو اللي ممكن تتغير بعد ما الـObject يتعمل.

هنا الـDog Object بيتعمل الأول، ممكن يكون فاضي عادي.
الكود اللي برّه بينادي على Public Method جواه – بنسميها Setter – عشان يديه له الـWalker.

مثلًا:

Injection by Setter

Although a constructor will *initialize* attributes when an object is instantiated, there is often a need to reset values during the lifetime of an object. This is where accessor methods come into play—in the form of setters. The `Walkable` behavior can be *inserted* into the `Dog` class by using a setter, here called `setWalker()`:

```
class Dog {
    Walkable walker;
    public void setWalker (Walkable w) {
        this.walker=w;
    }
}
```

As with the constructor technique, the application instantiates a `Walkable` object and inserts it into the `Dog` via the setter:

```
Walkable walker = new Walkable();
Dog fido = new Dog();
fido.setWalker(walker);
```

الكود الخارجي برضه هو اللي بيعمل new Walkable، وبعدين ينادي setWalker() عشان يبعته.

- دي بتدي مرونة، الـDependencies مش لازم تبقى موجودة من أول ثانية، أو لو عايزين نغيرها بعدين،

بعد ما عملنا كل ده واستخدمنا الـ DI سواء Constructor أو Setter،
إيه بقى الزتونة؟ إيه الفايدة الحقيقية؟ المحصلة النهائية إيه على تصميم الكود؟

الفايدة الأساسية والهدف الأكبر للفصل ده هي إننا نوصل لـ Decoupling أو فك الترابط بشكل كبير جدًا.
لما بنستخدم Dependency Injection كلاس Dog ما بقاش مربوط وما بقاش معتمد على الـ Class الفعلي اللي اسمه Walkable،
هو أصلًا ممكن ما يبقاش عارف إن فيه Class بالاسم ده.

إزاي يعني؟ هو مش لسه بيستخدم Walker؟
آه ما هي دي الحتة الأحلى بقى.

إن الأفضل كمان والأصح إننا نخلي Dog ما يعتمدش على Concrete Class خالص.
حتى في الـ Constructor or Setter بدل كده نخليه يعتمد على Abstraction.

يعني Interface مثلًا؟
بالظبط. نعمل Interface اسمها Walker وجواها Method اسمها walk()،
وبعدين نخلي Dog يعمل Inject لـ Interface دي.

وساعتها يبقى كده تمام.
دلوقتي كل اللي Dog يعرفه وكل اللي يهمه إنه محتاج أي Object بيعرف يعمل walk().

مين هو ده ومن أي Class؟ مش مهم خالص.
ممكن نبعت له Object من SlowWalker، أو ممكن نبعت له Object من Class تاني خالص اسمه FastWalker،
أو حتى Object وهمي اسمه TestWalker لما نكون بنعمل Testing.

يعني Dog بقى بيتعامل مع عقد أو وظيفة اللي هي الـ Interface،
مش مع موظف معين اللي هو الـ Concrete Class.

مية مية.

تخيلها زي ما بتتعامل مع فيشة الكهربا في الحيطة.
الفيشة دي هي الـ Interface.
إنت مش مهتم مين الشركة اللي أنتجت الكهربا اللي جوه السلك؟
يعني كل اللي يهمك إن الفيشة دي بتديك كهربا بالمواصفات اللي إنت عايزها.

مثال رائع.

ده بيخلي السيستم مرن لأقصى درجة ممكن تتخيلها.
نقدر نضيف سلوكيات جديدة أو نعدل السلوك الحالي من غير ما نلمس سطر كود واحد في Class Dog.
ونقدر نعمل Testing لكلاس Dog بسهولة خرافية لأننا نقدر نحقن له TestWalker وهمي بدل الحقيقي،
عشان نختبره لوحده بعيد عن أي Dependencies تانية.

كل ده بيرجعنا تاني للهدف الأساسي:
نتجنب الكود الـ Rigid والـ Fragile والـ Immobile.

وفهم الـ DI ده يعتبر تمهيد ممتاز لمبادئ تصميم تانية أهم وأشمل اسمها SOLID Principles.

الـ SOLID Principles.
بالظبط، والكتاب هيتكلم عنها بالتفصيل في الفصل اللي جاي، الفصل ال12.

يبقى لو حبينا نلخص بسرعة كده أهم أفكار الفصل الحادي عشر،
نقدر نقول إنه بيشرح مشكلة إدارة الاعتماديات والترابط بين الكلاسات.

وبيورينا إزاي لما نستخدمه صح ومعاه تقنية DI بيوصلنا لتصميمات مرنة وقابلة للتطوير والصيانة أحسن بكتير من إننا نعتمد على Inheritance بشكل كبير وخلاص.

صح.

والنقطة الجوهرية زي ما قلت هي فك الارتباط،
إننا نخلي الكلاسات تعتمد على Abstraction زي Interface بدل Concrete Class،
وإننا نحقن Dependencies دي من بره بدل ما الكلاس يعملها بنفسه.

في نقطة أخيرة بس مهمة لازم نقولها ونأكد عليها:
كل الكلام ده مش معناه أبدًا إن Inheritance وحش أو إننا ما بنستخدموش خالص ونستخدم بس Composition و DI في كل حتة.

آه طبعًا، ما فيش حل سحري واحد.

بالظبط.

لسه Inheritance ليه مكانه وليه دوره المهم جدًا خصوصًا في علاقات **الـ "is-a"** اللي بتكون واضحة ومستقرة.
لما بيكون فعلاً Child حقيقي للـ Parent،
مش مجرد إننا عايزين نعيد استخدام حتة كود معينة وخلاص.

الفكرة كلها إننا كمبرمجين وكمصممين لازم نبقى فاهمين كويس قوي الفروقات بين الأسلوبين،
مميزات وعيوب كل واحد،
وإمتى وليه نستخدم ده أو ده.

الهدف في الآخر هو إننا نختار الأداة الصح،
اللي تخلينا ندير Dependencies في السيستم بتاعنا بوعي وبشكل فعال حسب طبيعة المشكلة اللي قدامنا والسياق اللي شغالين فيه.

الـ DI هو أداة قوية جدًا في صندوق أدواتنا عشان نحقق ده.

لو حد حابب يتعمق اكتر Dependency Injection in Flutter | Mohamed Naif | 60 comments🌐

في الملف هتلاقي
يعني إيه Dependency و Injection؟
الفرق بين Tightly Coupled و Loosely Coupled
أنواع Dependency Injection: Constructor vs Setter
استخدام مكتبة get_it كـ Service Locator
الفرق بين:
registerSingleton
registerLazySingleton
registerFactory
إمتى استخدامهم ممكن يكون Over Engineering؟
دمج get_it مع clean architecture و injectable
شرح كامل لـ injectable وأنواع الـ @annotations

بس كده سلام.

الفصل الثاني عشر The SOLID Principles of Object-Oriented Design :

النهاردة هنتكلم شوية في موضوع مهم،
مهم قوي للمبرمجين،
خصوصاً اللي لسه بيبدأوا رحلتهم مع البرمجة الكائنية أو الـ OOP.
يعني بعد ما الواحد فهم الأساسيات، الكلاسات والحاجات دي، الفصل ده بينقلنا لمستوى تاني،
مستوى التصميم بقى.

تمام.
بنتكلم عن إزاي نبني الكود بشكل صح، مش مجرد كود شغال وخلاص.
وهنا بتيجي مجموعة مبادئ مهمة جداً اسمها **SOLID.** هدفنا النهاردة نفصص المبادئ دي ونبسطها.

وقبل ما نخش في **SOLID** نفسها، يمكن نسأل: ليه أصلاً محتاجين المبادئ دي؟ مش كفاية إننا بنوصل العالم بكلاسات؟
الكتاب وتحديداً كلام روبرت مارتن بيقول إن الهدف أعمق من كده.
الهدف هو إ**دارة الاعتماديات بين أجزاء الكود**.

أيوة، حتة الاعتماديات دي هي مربط الفرس زي ما بيقولوا.
لو الاعتماديات دي مش متظبطة الكود بيبقى صعب أوي في التعامل معاه بعد كده.

In the lecture I mentioned previously, Robert Martin, often referred to as Uncle Bob, defines these three terms to describe non-reusable code:

- **Rigidity**—When a change to one part of a program can break another part
- **Fragility**—When things break in unrelated places
- **Immobility**—When code cannot be reused outside its original context

طيب، ممكن تدينا فكرة عامة وسريعة كده عن SOLID قبل ما ندخل في تفاصيل كل حرف؟
أكيد.
SOLID ده اختصار، كل حرف بيمثل مبدأ تصميم من خمس مبادئ أساسية:

The five SOLID principles are

- **SRP**—Single Responsibility Principle
- **OCP**—Open/Close Principle
- **LSP**—Liskov Substitution Principle
- **IPS**—Interface Segregation Principle
- **DIP**—Dependency Inversion Principle

تمام.

ممكن نعتبرهم زي خريطة الطريق كده بيوجهنا علشان نعمل تصميمات أحسن،
مرنة أكتر وسهل إننا نصلح فيها ونطورها بعدين.
يعني أساسيات لشغل البرمجة الاحترافية.

كلام جميل.
الإطار ده مهم.
طيب، ياللا بينا نمسكهم واحدة واحدة.

نبدأ بالـ S، أول حرف: مبدأ المسئولية الواحدة SRP: Single Responsibility Principle.
ايه القصة هنا...
بُص، المبدأ ده يمكن يبان بسيط، لكن تطبيقه بيفرق جداً.
بيقول إن أي Class أو حتى أي وحدة برمجية عموماً لازم يكون لها سبب واحد بس يخلينا نحتاج نغيرها.

سبب واحد بس.
تغيير يعني إيه؟ يعني ده المفروض يكون مسؤول عن حاجة واحدة بس، مهمة محددة جداً.

لو لقيت نفسك بتشرح الـ Class ده بيعمل إيه؟ وبتقول كلمة **"و"** زي مثلاً: الـ Class ده بيحسب المرتبات **و** بيطبع التقارير **و** بيبعت إيميلات،
يبقى غالباً الـ Class ده شايل أكتر من مسئولية.

آه،
يعني لامم كذا شغلانة مع بعض.

بالظبط.
الهدف إن كل مسئولية تبقى متغلفة كده في مكانها، في الـ Class بتاعها، منعزلة عن الباقيين.
وده علشان لو غيرنا في تفاصيل صغيرة في مسئولية معينة، ده ما يأثرش على المسئوليات التانية، ما لهاش دعوة.

تمام جداً.
كده بتقلل خطر إن تعديل بسيط يبوظ الدنيا في حتت تانية خالص.

الكتاب بيشرحها بمثال كويس:
تخيل عندنا Class اسمه CalculateAreas،
بيحسب مجموع مساحات أشكال هندسية.

ماشي.
وفي نفس الوقت الـ Class ده هو هو بيطبع النتيجة دي على الشاشة، على الـ Console يعني.

أيوة،
يعني بيعمل الحاجتين مع بعض: الحساب **والعرض**.

أيوة.
المشكلة فين بقى؟ لو حبينا نغير طريقة العرض، بدل الـ Console عايزين نعرض النتيجة دي في صفحة ويب HTML مثلاً،

هنعمل إيه؟ هنضطر نروح نعدل في الـ Class نفسه،
ونعمله اختبار و.....

مع إن الجزء بتاع الحسابات نفسه ما اتغيرش خالص.
ده كده ربطنا حاجتين ببعض من غير لازمة.

طيب، والحل؟ حسب **الـ SRP** نفصلهم عن بعض.
بالظبط كده، الحل إننا نفصل المسئوليات: نعمل Class اسمه مثلاً Calculator شغلته بس الحسابات،
ونعمل Class تاني خالص نسميه مثلاً Printer أو أي اسم تاني، شغلته بس إنه يعرض النتيجة.

والـ Class التاني ده ممكن يعرض بكذا طريقة بقى.
ممكن جداً، ممكن يبقى جواه طريقة تعرض على الـ Console، وطريقة تعرض على Web Page، وطريقة تالتة تعرض على ملف، وهكذا.

كده لو حبينا نغير طريقة العرض أو نضيف طريقة جديدة، هنعدل فين؟ في الـ Class بتاع العرض بس.

بالظبط.
ومش هنلمس الـ Class بتاع الحسابات.
والعكس صحيح، كل واحد بقى مسؤول عن حتة، وسبب تغييره بقى واحد ومحدد.

فهمتِ؟ يعني بنعزل التغيير ونحدد المسئوليات.

ممتاز.
طيب...

النقطة اللي بعد كده بقى: إزاي نقدر نضيف إمكانيات للكود بتاعنا من غير ما نفضل نفتح الكود القديم اللي شغال ومستقر ونعدل فيه؟

هنا بيجي دور الحرف التاني: OCP: Open/Close Principle .
تمام، الـ O بترمز لمبدأ الفتح/الغلق.



المبدأ ده بيقول إن الـ Classes والـ Modules بتاعتنا لازم تكون مفتوحة للتوسع، ومغلقة للتعديل.
able to extend a class’s behavior, without modifying it.
مفتوحة ومغلقة... الكلام ده يبان عكس بعضه شوية. إزاي يعني؟ هو مش تناقض بالمعنى الحرفي.

الفكرة إن الكود الأساسي اللي كتبناه وتعبنا فيه واختبرناه،
المفروض إننا ما نرجعش نعدل فيه كل شوية عشان نضيف وظيفة جديدة.

ليه؟ علشان كل تعديل في كود شغال بيفتح باب لأخطاء جديدة ما كانتش موجودة.
ده الجزء بتاع **"مغلق للتعديل".**

تمام.
والجزء المفتوح؟
في نفس الوقت لازم التصميم بتاعنا يكون معمول بطريقة تسمح لنا نضيف وظايف جديدة أو سلوكيات جديدة للنظام بسهولة،
من غير ما نلمس الكود القديم ده خالص.
هو ده الجزء **"مفتوح للتوسع".**

طيب، وإزاي بنعمل المعادلة الصعبة دي؟
غالباً بنستخدم المفاهيم الأساسية بتاعة الـ OOP زي الـ Abstraction.

الكتاب بيرجع تاني لمثال الأشكال الهندسية.
بدل ما نعمل Class واحد كبير جواه كود لكل شكل هندسي ممكن (مستطيل، دايرة، مثلث...) ونضطر نعدل فيه كل ما نضيف شكل جديد،
الطريقة القديمة دي بتبقى صعبة جداً.

الأفضل بقى إننا نعمل تعريف عام للشكل الهندسي.
ده بيكون في شكل مجرد مثلا **abstract class Shape** .
في مثالنا، الـ **Shape** ده ممكن يكون جواه **Abstract Method** اسمها **()getArea**.

مجردة يعني إيه؟ يعني الـ Shape نفسه مش بيقول المساحة بتتحسب إزاي، هو بس بيقول: أي حد هيعتبر نفسه Shape لازم يعرف يقول مساحته كام.
زي عقد كده بيلزم اللي بعده.

بالظبط.
دلوقتي بقى لو حبينا نضيف شكل جديد زي الـ Rectangle أو Circle (دايرة)، نعمل إيه؟
بنخليهم يورثوا من Shape،
وكل واحد فيهم لازم يقدم التنفيذ بتاعه للوظيفة **()getArea**.

المستطيل هيقول: الطول × العرض.
الدايرة هتقول: π × نصف القطر^2.

والـ Class الـ Shape الأصلي مش بيتغير خالص.
هو ده "المغلق للتعديل".
لكن النظام كله بقى قادر يستقبل أشكال جديدة بسهولة،
مجرد نعمل Class جديد يورّث وينفذ العقد.
ده الجزء "المفتوح للتوسع".

والميزة الكبيرة كمان إن الكود التاني اللي كان بيستخدم الأشكال دي،
زي Class CalculateAreas بتاع المثال الأول، هو كمان مش محتاج يتغير لما نضيف شكل جديد، صح؟

مية في المية صح.
لأنه بيتعامل مع أي شكل على إنه Shape، ويطلب منه المساحة بتاعته.
مش فارق معاه بقى هو دايرة ولا مستطيل ولا شكل هييجي بكرة.

وده بسبب الـ Polymorphism أو تعدد الأشكال،
اللي اتكلمنا عنها قبل كده.
هي دي اللي بتخلي الموضوع ده يشتغل.

تماماً.
الـ Polymorphism هو البطل هنا.

بيخلينا نتعامل مع أنواع مختلفة بنفس الطريقة،
وكل نوع يرد بطريقته.
فا ال **OCP** بيعتمد بشكل كبير على الـ **Polymorphism**.

جميل قوي، يبقى بيخلينا نضيف الجديد بأمان.

نخش على المبدأ التالت، حرف الـ L، اللي هو مبدأ LSP: Liskov Substitution Principle.

ده ساعات باحسه إنه محتاج تركيز شوية في فهمه.
هو فعلاً محتاج تركيز،
لأنه مخالفته ساعات بتعمل مشاكل مش بتظهر على طول.

مبدأ الـ LSP بيقول ببساطة:
لو عندك Class S هو ابن الـ Class T (يعني S بيورث من T)،
لازم تقدر تستخدم Object من S مكان أي Object من T في أي حتة في الكود،
من غير ما الدنيا أو سلوك البرنامج يتغير بشكل غريب وغير متوقع.

يعني الابن لازم يبقى بديل حقيقي للأب.
بالظبط، بديل كامل مش مجرد شبه في الشكل.

طيب، ما ده يبان طبيعي مع الوراثة. مش الابن بياخد كل حاجة من الأب؟
نظرياً آه، لكن عملياً مش دايماً.

المشكلة بتظهر لما الابن علشان يعمل شغلانته،
يضطر يغير أو يقيد سلوك ورثه من الأب بطريقة الأب ما كانش عامل حسابها.

أشهر مثال على الحكاية دي (واللي الكتاب بيستخدمه برضه) هو العلاقة بين المستطيل (Rectangle) والمربع (Square).
رياضياً المربع حالة خاصة من المستطيل،
فممكن نفكر إننا نخلي الـ Square يورث من الـ Rectangle.
منطقي في الأول.

تمام.
عنده طول وعرض (Heightg Width) وممكن نغير كل واحد لوحده بوظايف زي setWidth و setHeight.

المربع بقى علشان يفضل مربع **لازم طوله يساوي عرضه دايماً.**
أكيد.

فممكن نعمل Constructor لل Square ياخد قيمة واحدة ويحطها في اللي ورثهم،
وممكن نعمل Override لل setWidth و setHeight إن لو غيرت واحدة، التانية تتغير معاها غصب عنها عشان يفضل مربع.

ماشي، لحد هنا ممكن تمشي.

المشكلة فين بقى؟
لو فيه كود تاني في حتة ما بيتعامل مع Object على إنه Rectangle وهو في الحقيقة Square.

الكود ده ممكن يكون متوقع إنه لما يعمل setHeight(10) وبعدها setWidth(5)،
يبقى عنده مستطيل 10×5، مساحته 50.

لكن لو هو مربع (Square)،
اللي هيحصل إن setWidth(5) هتخلي الـ Height كمان = 5.
فالمساحة هتبقى 25 مش 50.

هنا سلوك الابن (Square) كسر توقعات الكود اللي بيتعامل مع الأب (Rectangle).
الابن ما بقاش بديل مثالي للأب لأنه كسر العقد أو الافتراض بتاع الأب إن الطول والعرض ممكن يتغيروا

بشكل مستقل.

ده كده انتهاك للـ LSP.

يعني الوراثة هنا، رغم إنها منطقية رياضياً، عملت مشكلة برمجية في السلوك.
ده معناه إن مش أي علاقة "is-a" تنفع وراثة وخلاص. لازم نفكر في السلوك كمان.

بالظبط.
الـ LSP بتقول لنا إن الوراثة لازم تكون قوية من ناحية السلوك كمان.
الابن لازم يحترم كل قواعد اللعبة بتاعة الأب وما يغيرهاش بشكل مفاجئ.

لو العلاقة مش قوية كده، يبقى الوراثة يمكن مش أحسن حل.
وساعتها نفكر في بدائل زي Composition.

تجاهل الـ LSP ممكن يعمل Bugs قوي وصعب نلاقيها.
نقطة خطيرة ومهمة جداً.

طيب، نعدي على المبدأ الرابع، حرف الـ I في SOLID، هو IPS: Interface Segregation Principle
أو مبدأ فصل الواجهات.

ده بيتكلم عن الـ Interface.
إيه قصته؟ قصته برضه بسيطة ومفيدة جدًا، بيقول إن ما ينفعش نجبر أي Class إنه يعمل أو يستخدم Interface كبيرة ومليانة وظائف هو مش بيستخدمها كلها.
بدل ما نعمل Interface واحدة ضخمة بيسموها ساعات Fat Interface، الأفضل إننا نعمل Interfaces أصغر ومتخصصة أكتر.
وكل Class ياخد بس الـ Interface اللي هو محتاجه فعلًا.

طب وليه ده مهم؟ إيه المشكلة إن الـ Class ياخد Interface كبيرة حتى لو هيسيب حتت منها فاضية؟
المشكلة في الاعتمادية اللي مالهاش لازمة.
لو الـ Class بتاعك معتمد على Interface ضخمة، أي تغيير يحصل في الـ Interface دي، حتى لو في حتة الـ Class بتاعك مش بيستخدمها، ممكن يخليك تضطر تغيّر الـ Class بتاعك وتعمل له Test من جديد.
ده بيزود **Coupling** أو الارتباط بين الكلاسات من غير داعي.

الكتاب بيدي مثال ظريف:
تخيل عندنا Interface اسمها **"IMammal"**، وحطينا فيها كل حاجة ممكن الثدييات تعملها.

زي eat, sleep ,swim, makeNoise, fly

المشكلة بقى لو جينا نعمل **class Dog implements IMammal** . الكلب بيعمل Bark لكن مش بيعرف Swim أو Fly.
لو اضطر إنه يستخدم الـ Interface دي هيضطر يحط Methods فاضية أو يرمي Exceptions للوظايف اللي مش بتاعته.
ونفس الكلام لو حوت **Whale** مثلًا، هيعرف Swim بس مش Walk.

- بالظبط.

الحل حسب ISP إننا نقسم الـ Interface الكبيرة دي ونعمل Interfaces صغيرة.
وكل Class ياخد اللي على مقاسه بس.

- تمامًا.

وماحدش مضطر يتعامل مع وظائف مش بتاعته.

ده Design أنضف والاعتماديات أقل وأوضح.
كل Class بيعلن عن قدراته الفعلية بس.
هنا زي ما قلنا قبل كده، الـ Interface زي عقد بيحدد مجموعة وظائف لازم الـ Class اللي هيستخدمها ينفذها.

- مفهوم. فعلاً بيخلي الدنيا أوضح.

طيب وصلنا لآخر حرف في الـ SOLID حرف ال D
DIP: Dependency Inversion Principle
وهو يمكن أكتر واحد بيربط كل الخيوط ببعض، وبيتعامل مباشرة مع مشكلة الاعتماديات اللي بدأنا بيها.

- ممكن تشرحي لنا ده بقى؟
- فعلًا، الـ DIP ده يعتبر زي التاج اللي فوق المبادئ التانية، وبيركز قوي على هيكل الاعتماديات في السيستم كله.

المبدأ ده له شقين أساسيين:

- **high-level modules** الوحدات اللي في مستوى عالي (زي مثلًا الكود اللي بينسق العمليات الكبيرة)
  ما ينفعش تعتمد بشكل مباشر على الوحدات اللي في مستوى منخفض **low-level modules .**
- اللي هي بتعمل التفاصيل الدقيقة، زي الكتابة في File أو التعامل مع Database معينة.

يعني الكبير ما يعتمدش على الصغير مباشرة.

- بالظبط.

الاتنين، الكبير والصغير، لازم يعتمدوا على Abstractions.
حاجة في النص كده تفصلهم.
والشق التاني بيقول إن الـ Abstractions دي نفسها ما ينفعش تعتمد على التفاصيل.
العكس هو اللي صح.
التفاصيل هي اللي لازم تعتمد على الـ Abstractions أو تلتزم بيها.

الكلام ده شكله تقيل شوية.
يعني إيه تجريد أو Abstraction هنا؟ وازاي بنعكس الاعتمادية دي؟ خلينا نبسطها:
الـ Abstraction هنا غالبًا بتكون Interface، هي زي ما قلنا عقد أو فكرة عامة بتوصف إيه المطلوب يتعمل من غير ما تدخل في تفاصيل التنفيذ الفعلية.

عكس الاعتمادية بقى معناها إيه؟ بدل ما الكلاسات الكبيرة والمهمة (High-level) تبقى بتعتمد على الكلاسات الصغيرة اللي بتنفذ التفاصيل، بنعمل إيه؟
بنخلي الكلاسات الكبيرة دي تعتمد على الـ Interface العقد أو الفكرة العامة دي، والكلاسات الصغيرة اللي بتعمل التفاصيل هي كمان بتلتزم بنفس العقد ده.

كأننا حطينا عازل في النص، اللي هو الـ Abstraction، والاتنين بقوا بيبصوا عليه وبيعتمدوا عليه بدل ما يبقوا معتمدين على بعض بشكل مباشر.

- آه بدأت توضح شوية.

الكتاب بيشرحها بمثال بيوضح الفكرة خطوة بخطوة:

البداية: نفس المثال "Mammal".
Class بيرثه Cat وينفذه. هنا Cat معتمد على Mammal مباشرة بالوراثة.

```
abstract class Mammal
{
    public abstract String makeNoise();
}
```

The subclasses, such as `Cat`, use inheritance to take advantage of `Mammal`'s behavior, `makeNoise()`:

```
class Cat extends Mammal
{
    public String makeNoise()
    {
       return "Meow";
    }
}
```

فصل السلوك بالـ Composition:
ممكن نفكر بشكل مختلف. بدل ما Cat يورث السلوك، نعمل Class تاني مسؤول بس عن إصدار صوت القطط ونسميه MakingNoise.

```
abstract class MakingNoise
{
        public abstract String makeNoise();
}
```

```
class CatNoise extends MakingNoise
{
    public String makeNoise()
    {
        return "Meow";
    }
}
```

ونخلي Class Cat جواه Object من النوع ده.
لما حد يقول لـ Cat "اعمل صوت"، هو يروح يقول لـ Object اللي جواه "اعمل صوت".
ده اسمه Composition.
فصلنا السلوك شوية.

```
abstract class Mammal
{
    public abstract String makeNoise();
}
class Cat extends Mammal
{
    CatNoise behavior = new CatNoise();
    public String makeNoise()
    {
        return behavior.makeNoise();
    }
}
```

عكس الاعتمادية:
الخطوة اللي فاتت كويسة، بس لسه Cat هو اللي عارف إنه بيستخدم MakingNoise بالذات، وممكن يكون هو اللي بيعمل Object جواه بنفسه.
ده لسه فيه اعتمادية مباشرة على Class معين.

الـ DIP بقى بيقول: لأ.
المفروض Cat ما يعرفش التفاصيل.

المفروض يعتمد على Interface زي INoiseMaker فيها بس وظيفة makeNoise().
يعني Cat كل اللي يعرفه إنه محتاج حد بيعرف يعمل Noise وخلاص.

- بالظبط كده.

طيب، مين بقى اللي هيديله الـ Object الفعلي اللي بيعمل الشغلانة دي (زي MakingNoise أو أي Class غيره)؟
الإجابة: حد من برة.

الكود اللي بيستخدم Cat، زي مثلًا Main أو أي Client Code، هو اللي بيبقى مسئول إنه يعمل Object من نوع NoiseMaker، ويبعتُه أو يحقنه جوه Cat.
ممكن يحقنه عن طريق Constructor أو عن طريق Setter.
العملية دي بقى اسمها Dependency Injection.

وهي دي التطبيق العملي لمبدأ الـ DIP.

- آهـا.

يعني بدل ما القطة هي اللي بتعمل MakingNoise بتاعها بنفسها، بقت تستنى حد من برة يديها أي MakingNoise شغال.
يعني أي Object بينفذ الـ INoiseMaker، وهي تستخدمه وخلاص.

ده يخلي القطة نفسها مش مرتبطة بنوع معين، وممكن نغير فيها بسهولة من غير ما نفتح الكلاس ونعدل فيه.

- بالظبط كده.

مثال :

```java
// Abstraction (العقد أو الـ Interface)
interface INoiseMaker {
    void makeNoise();
}

 //التفاصيل (Implementation 1)
class MeowNoiseMaker implements INoiseMaker {
    @Override
    public void makeNoise() {
        System.out.println("Meow! Meow!");
    }
}

 //التفاصيل (Implementation 2)
class AngryCatNoiseMaker implements INoiseMaker {
    @Override
    public void makeNoise() {
        System.out.println("Hissss! Grrrr!");
    }
}

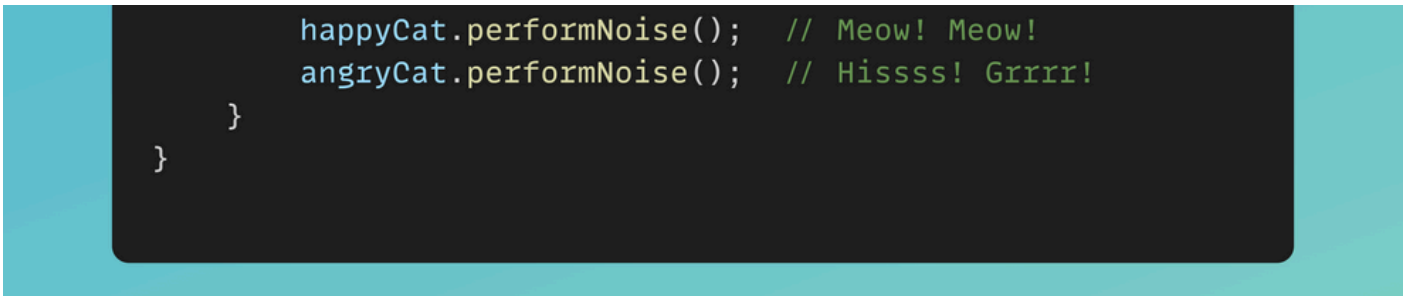
// High-level module (القط نفسه)
class Cat {
    private final INoiseMaker noiseMaker;

     //هنا بنعمل Dependency Injection عن طريق الـ Constructor
    public Cat(INoiseMaker noiseMaker) {
        this.noiseMaker = noiseMaker;
    }

    public void performNoise() {
        noiseMaker.makeNoise();
    }
}

// Client code (الكود اللي بيحقن الاعتمادية)
public class Main {
    public static void main(String[] args) {
         //نقدر نغير بسهولة نوع الـ NoiseMaker في من غير ما نعدل Class Cat
        INoiseMaker normalNoise = new MeowNoiseMaker();
        INoiseMaker angryNoise = new AngryCatNoiseMaker();

        Cat happyCat = new Cat(normalNoise);
        Cat angryCat = new Cat(angryNoise);
```



هنا اللي بيحصل:

- **Cat** ما بيعتمدش على تفاصيل (زي MeowNoiseMaker) بشكل مباشر.
- هو بيعتمد بس على (INoiseMaker) **Abstraction**.
- التفاصيل (زي MeowNoiseMaker و AngryCatNoiseMaker) هي اللي بتلتزم بالعقد ده.
- الاعتمادية اتعكست: التفاصيل هي اللي معتمدة على الـ Interface، مش العكس.
- وده اللي بنسميه **Dependency Injection** (حقن الاعتمادية).

و ده

- بيقلل الارتباط الجامد أو Coupling بين الكلاسات.
- بيخلي الكود أسهل بكتير في الاختبار. ممكن نحقن Object وهمي للاختبار.
- وبيزود قابلية إعادة الاستخدام والتعديل بشكل رهيب.
- رائع رائع، الصورة كده كملت فعلًا.

يبقى كده على المبادئ الخمسة لـ SOLID:

- **SRP**: مسئولية واحدة عشان التغيير يبقى سهل والخطر قليل.
- **OCP**: نوسع الكود من غير ما نعدل في القديم.
- باستخدام الـ Abstraction والـ Polymorphism.
- **LSP**: الابن لازم يبقى بديل بجد للاب في السلوك عشان الوراثة تبقى صح.
- **ISP**: واجهات صغيرة ومتخصصة عشان نقلل الاعتماديات اللي مالهاش لازمة.
- **DIP**: نعتمد على التجريدات مش التفاصيل، ونستخدم Dependency Injection.

والهدف الكبير من اللفة دي كلها، زي ما الكتاب بيقول وكلام مارتن بيوضح، إننا نوصل لكود نقدر نفهمه بسهولة ونعدل فيه وإحنا مطمئنين، ونعرف نطوره لسنين طويلة من غير ما نكره نفسنا.

المبادئ دي هي أسلحتنا ضد:

- **Rigidity**—When a change to one part of a program can break another part
- **Fragility**—When things break in unrelated places
- **Immobility**—When code cannot be reused outside its original context

ال SOLID هي نقلة في التفكير عشان نعمل برامج قوية بجد.

- طيب، لو حبينا نوجّه رسالة أخيرة للي لسه بيتعلم OOP وقرأ الفصل ده:
  إيه أهم حاجة المفروض تطلع بيها من المبادئ دي؟

أهم حاجة إن فهم وتطبيق المبادئ دي مش رفاهية أو حاجة للمشاريع الكبيرة بس.
دي أساسيات بتبني عادات برمجة سليمة من الأول.

ممكن في البداية تحس إنها بتعقد الدنيا شوية أو بتخليك تكتب كود أكتر، بس على المدى الطويل بتوفر عليك وقت ومجهود وصداع كبير جدًا.

الهدف مش بس نكتب كود بيشتغل.
الهدف نكتب كود مصمم صح يقدر يستحمل التغيير ويتطور.
دي خطوة عشان تبق مهندس برمجيات بتفكر في التصميم مش مجرد حد بيكتب أوامر يا سيد.

- ربنا يباركلك يا أبو السيد والله تعبتك معايا

لو حابب تتعمق اكتر و تعرف تفاصيل اكتر: 🌐 المختصر الفيد في شرح السوليد العنيد | Mohamed Naif

و بكده نكون خلصنا الكتاب كله بفضل الله الحمد لله الذي يسر و بس كده يا سيدي متنساش تديعيلي دعوة حلوة سلام.